zerotouch
ENTERPRISE ORCHESTRATOR

ZTF-Orchestrator

ZTF Orchestrator Administrator and User Guide



Table of Contents

How to use this guide 8
Changes covered in this revision 8
Start Here and guided cluster deployment 8
Follow the deployment journey 10
Continue to the post configuration pipeline 11
Architecture and operating boundaries 11
Installation and first login 12
Current Scope 12
Installation Options 13
Common First Login Flow 13
Option A One Command Linux macOS 14
Prerequisites 14
Install 14
Restart Later 14
Validate 15
Option B One Command Windows PowerShell 15
Prerequisites 15
Install 15
Restart Later 15
Validate 16
Option C Docker Compose with PostgreSQL 16
Prerequisites 16
Install 16
Validate 17
Stop and Restart 17
Option C2 Docker Compose File Backend 17
Install 17
Validate 18
Option D Appliance Deployment 18
Prerequisites 18
Install on a Fresh VM 18
Validate 19
TLS Recommendation 19
Pre-Built QCOW2 Summary 19
Option E Manual Install 20

Prerequisites 20
Install 20
Validate 21
Option F Kubernetes Starter Manifests 21
Prerequisites 21
Install 21
Validate 22
Air Gapped Installation 22
Connected Staging Environment 22
Connected Staging for a Portable AHV Appliance 23
Disconnected AHV Appliance Import 24
Disconnected Target Environment 26
Air Gapped Internal Mirrors 27
Air Gapped NKP Notes 28
Post Install Hardening Checklist 28
First login acceptance checks 28
Screen reference and daily operation 28
Roles And Access 29
Navigation Shell 29
Common Operating Flow 29
Overview Menu 30
Overview Dashboard 30
Setup Install 32
Workflows 1 x 33
Workflows 2 x 34
Configure Menu 35
Global Config 35
Config Files 37
YAML Studio 38
Workflow Catalogue 39
Scripts 1 x 43
Scripts 2 x 44
Execute Menu 46
Execution History 46
Jobs Queue 47
Pipelines 48

Schedules 50
Parallel Exec 51
NKP Framework 52
Govern Menu 54
Approvals 54
Appliance Ops 55
Upgrade Advisor 57
Inventory and saved assessment operations 58
Validation Evidence 59
Drift Detection 60
Audit Log 62
Admin Menu 63
Users 63
Settings 64
ZTF 2 x IaC workspace 66
Deploy an AHV HCI cluster with Foundation VM 67
Prepare the deployment 67
Complete the form 68
Validate and submit 69
How the workflow interacts with Foundation 70
Monitor and recover 70
Multiple clusters and remote sites 70
Accept the cluster 70
Initial Prism Element configuration and password changes 71
Procedure 71
Native Schema 72
Pipelines And Validation 73
Standalone PE admin password change 73
Build a safe post deployment pipeline 73
Verify requested storage container properties 75
Create named local PE administrators 75
Diagnose local user provisioning safely 76
Independently verify the requested accounts 77
CVM and Dell iDRAC credential rotation 77
Prepare and run 78
Behaviour and recovery 78

Vendor references and validation boundary 79
Live Prism Element drift monitoring 79
Configure a baseline 79
Trust and API compatibility 80
Interpret live connection diagnostics 80
Acceptance test 80
Establish persistent certificate trust 81
Validate the schedule end to end 82
Administration security and service operations 83
Security Administration 83
Backup Restore and Disaster Recovery 83
Disaster Recovery Cluster Rebuild Use Case 84
Backup and restore procedure 85
Metadata 85
Purpose 86
Scope 86
Preconditions 86
Required Role RBAC 86
Required Inputs 86
Dependencies 86
Risk Impact 86
Procedure 86
Validation 87
Expected Result 87
Failure Conditions 87
Recovery Rollback 87
Evidence To Capture 87
Audit Requirements 87
Escalation 87
References 87
Evidence Mapping 88
Notifications governance and build identity 88
Keep Docker rebuild identity aligned 92
Appliance update and rollback procedure 93
Apply and accept a consolidated RC appliance bundle 93
Rollback without reversing target changes 94

Daily weekly and maintenance checks 98
Troubleshooting and support 99
Native Foundation planning and post Foundation baselines 100
Workflow and script reference 100
Workflow catalogue 101
ZTF 1 x script catalogue 103
ZTF 2 x script catalogue 107
Acceptance checklists and source reference 107
Administrator handover checklist 107
Operator job checklist 108
Validation limits of this document 108
Source documents 108
Screenshot index 109
Cluster deployment process flow revision 3 112
Stages 01 to 05 preparation and deployment 113
Stages 06 to 10 configuration and acceptance 114

ZTF Orchestrator Administrator and User Guide

Installation Configuration Operations and Governance

A practical reference for administrators who deploy and maintain the console, operators who run approved Nutanix automation, and reviewers who need traceable results. This guide explains the application screens, the supported execution paths and the checks required before accepting infrastructure changes.

Important. ZTF-Orchestrator is an independent community project, not a Nutanix-supported or certified product. Validate vendor compatibility, local change controls and recovery procedures before operational use.

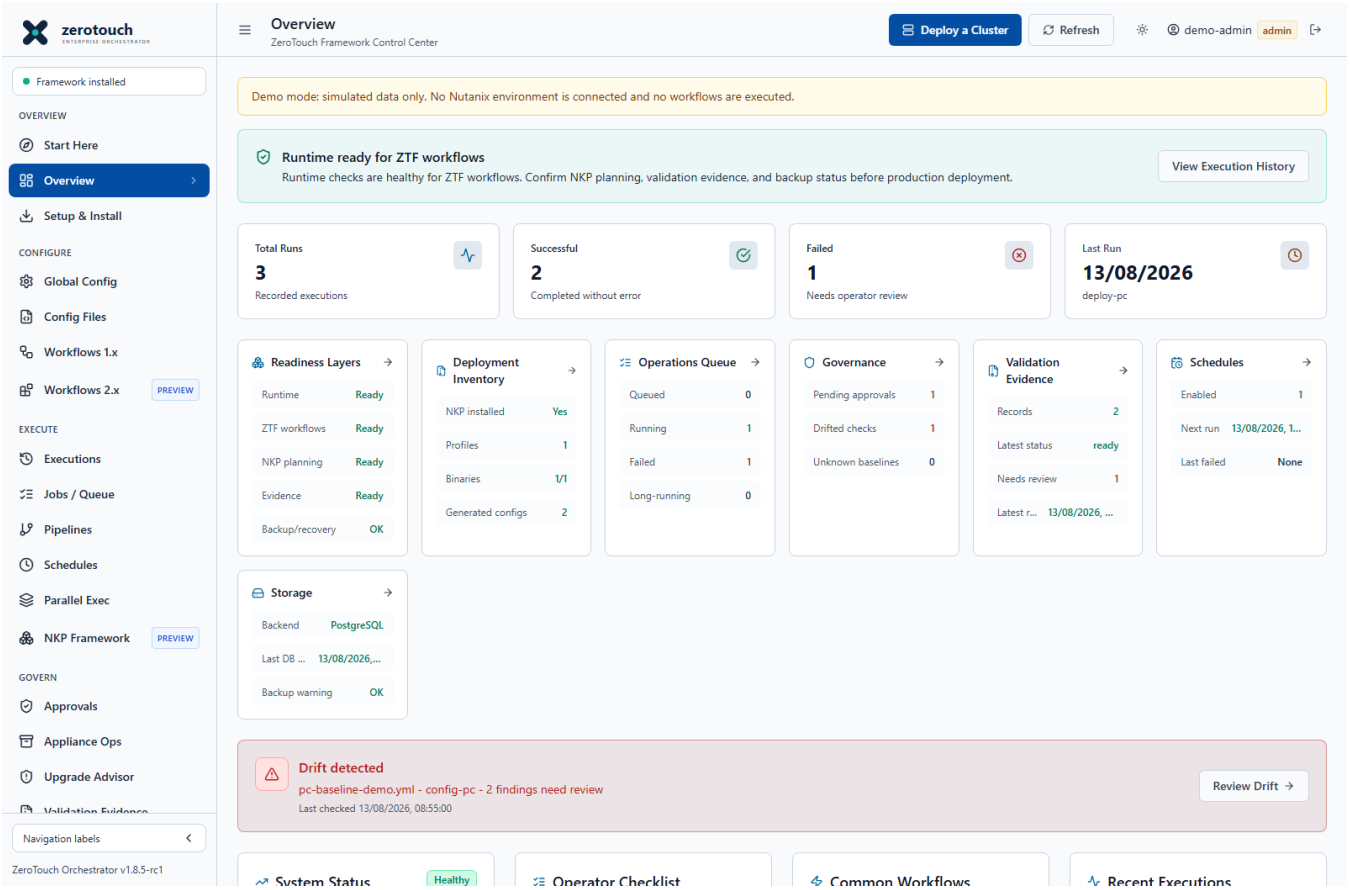


Figure 1. Operations dashboard and navigation. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

How to use this guide

Begin with Start Here to choose an outcome, then use the installation and detailed task chapters. The deployment journey opens the existing service-specific workflows; continue to post-configuration only after actual Foundation completion and independent PE health checks. Use the final chapters for support, backups, updates, recovery and catalogue identifiers.

Reader	Primary responsibilities
Administrator	Install, secure, back up, update and recover the service; manage users, runtimes, permissions and high-risk actions.
Operator	Prepare approved configuration, validate, submit permitted work, inspect results and capture target-side evidence.
Viewer or auditor	Read permitted operational records and evidence; request changes through an authorized operator.

Screenshot provenance. All application illustrations in this revision were recaptured on 27 October 2026 from merged frontend revision 4a010727d5db3496d17a52e4ce681818dcc56200 using an isolated static demo. They show simulated data and disabled live execution, not connected PE/PC or Foundation results. Source controls are current; example counts, identities and installed-build fields are demo fixtures rather than appliance evidence.

Changes covered in this revision

This revision adds Start Here and its seven-stage deployment journey, the post-configuration pipeline handoff, source-aligned Docker build identity, safe stage-specific PE local-user diagnostics and independent account verification. It retains the operational Upgrade Advisor, live drift scheduling and diagnostics, fresh password authentication, clean password skips, container readback and persistent trust procedures. Unfinished deployment-engine work remains excluded.

Use an approved, commit-specific image or bundle when testing post-candidate fixes. The original RC image, a new source commit, an imported update and the image actually running on a host are different states. Retain independent backups and recovery instructions before cutover.

Reading commands. Replace angle-bracket placeholders before execution. Display-wrapped code is explanatory; use the linked source procedure for long commands. Review scripts before running them, protect .env and credential files, and never paste passwords into public tickets or screenshots.

Revision 3 process flow. The appendix provides the complete deployment-to-operations sequence and enlarged stage views. The online process guide includes a readable text equivalent and full-resolution downloads.

Start Here and guided cluster deployment

After login, users without a saved landing preference open Start Here. Choose the outcome you want rather than searching through technical workflow identifiers. Existing approvals, exact confirmations and role restrictions still apply.

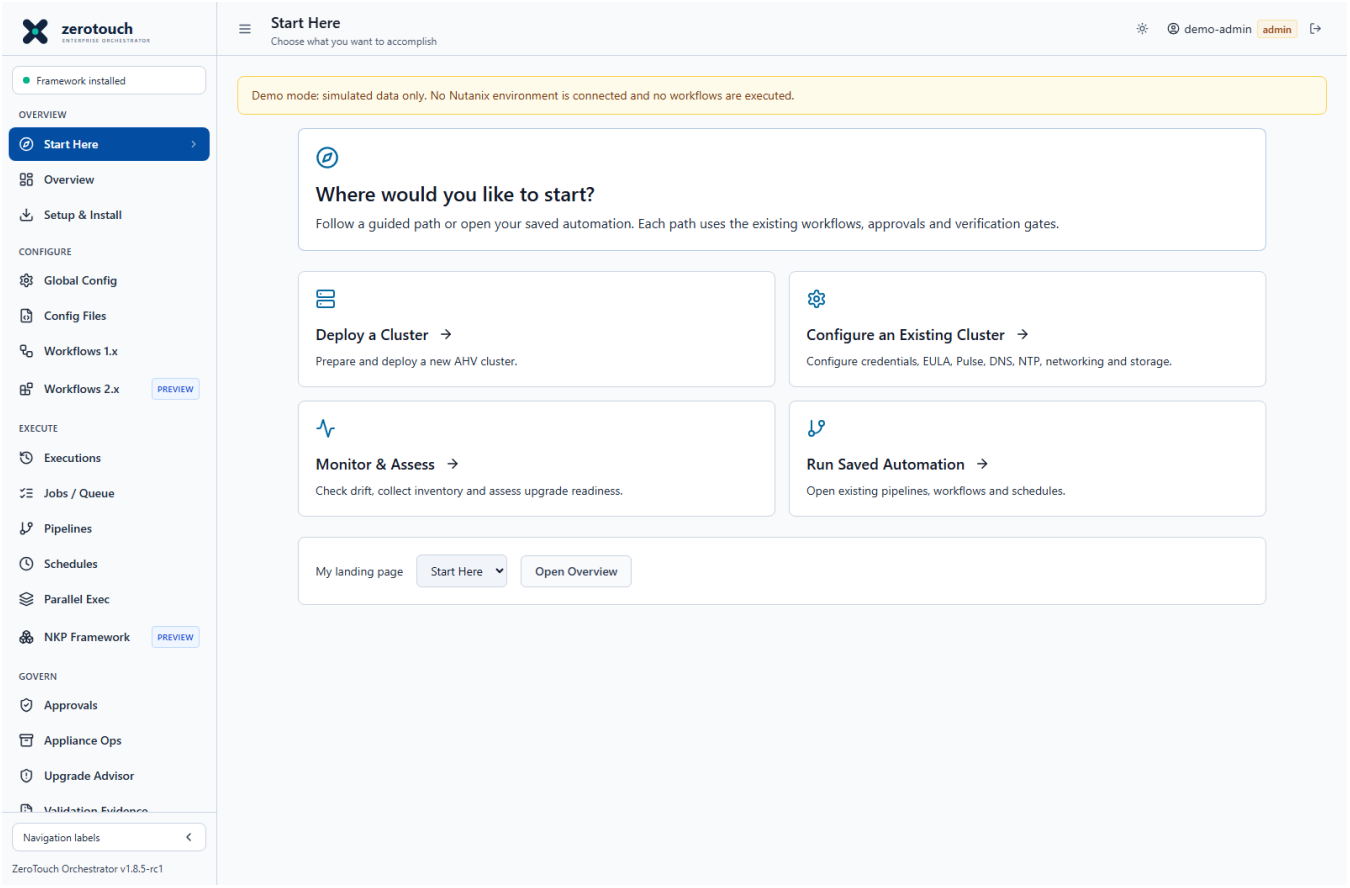


Figure 2. Start Here and the four operator choices. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Deploy a Cluster prepares a new AHV cluster. Configure an Existing Cluster prepares credentials, EULA, Pulse, DNS, NTP, supported VM networks, storage and optional controls. Monitor & Assess opens drift, inventory and upgrade-readiness tools. Run Saved Automation opens existing pipelines, schedules and jobs.

Use My landing page to choose Start Here or Overview for this user in this browser. Browser storage errors are reported; preferences are not centrally synchronized. The Overview dashboard remains available, including a prominent Deploy a Cluster action. Specialist tools remain under Advanced Automation.

Follow the deployment journey

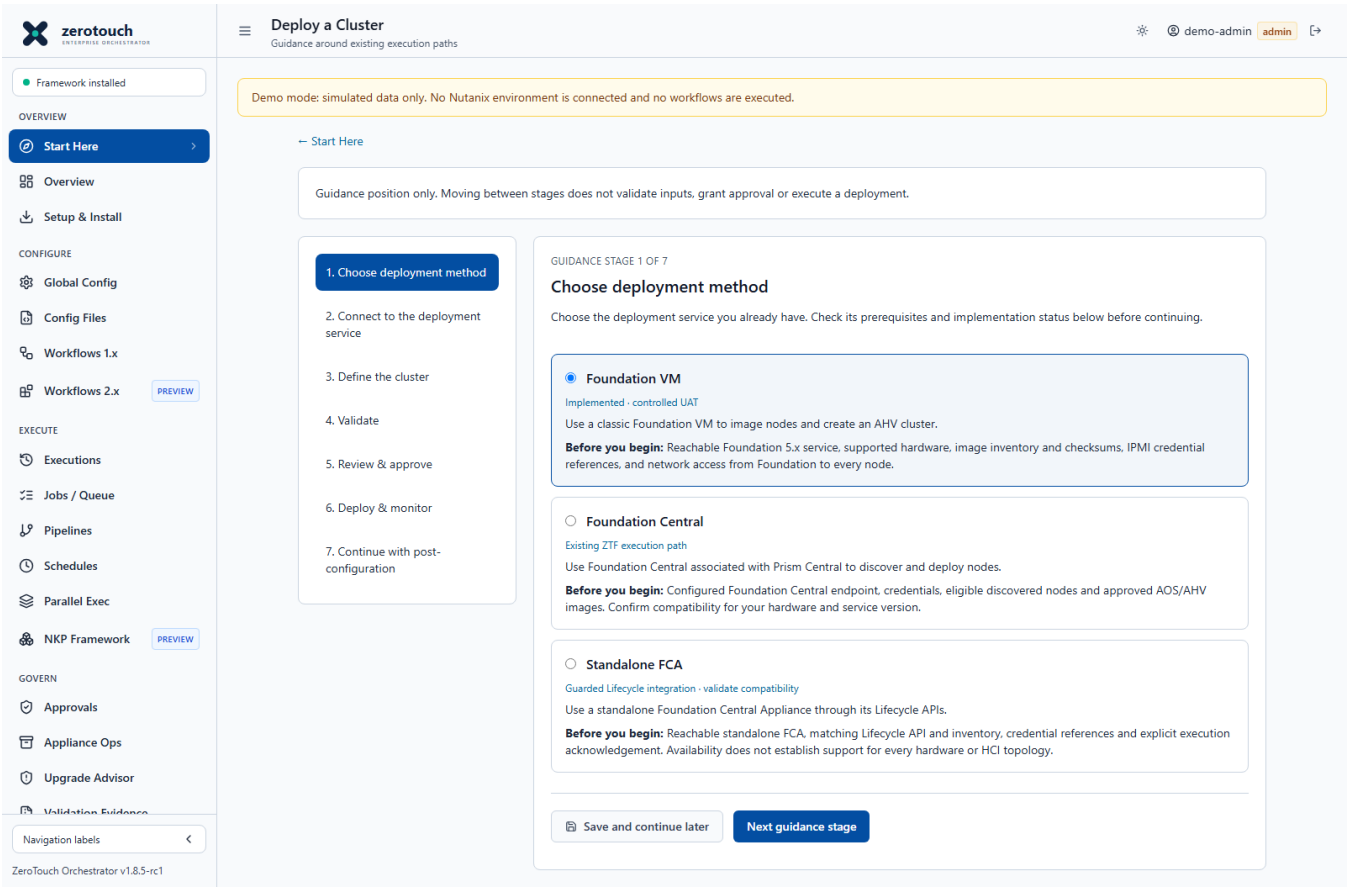


Figure 3. Deployment methods and seven guidance stages. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

1. Choose deployment method. Select classic Foundation VM, integrated Foundation Central or standalone FCA according to the service actually present. Review prerequisites and implementation status. A method selector does not extend hardware support.

2. Connect to the deployment service. Open the selected existing workflow, enter its endpoint and credential references and use available connection or inventory checks. Confirm target network reachability and required certificate trust.

3. Define the cluster. Enter nodes, addressing, image choices, checksums and cluster settings or Import Config. Use Download Config or Config Files to retain the actual YAML.

4. Validate. Run Dry Run and service-specific native validation. Resolve missing inputs, reference errors, image and connectivity failures. Coverage differs by workflow; a pass is not proof of hardware support or completed deployment.

5. Review and approve. Review exact serials, management addresses, cluster VIP, image versions and changes. Obtain approval for the exact YAML. Imaging can overwrite existing installations; changed content requires renewed review.

6. Deploy and monitor. Submit from the execution workflow after approval. Inspect Jobs, per-node progress and logs. Check actual Foundation and node state before abort, restart or retry.

7. Continue with post-configuration. Confirm Foundation completion and fresh PE health first, then follow the post-configuration handoff. Guidance position does not track verified infrastructure state.

Save and continue later saves only the selected method and guidance position in this browser. It does not save form inputs, approve work, execute anything or mark a stage complete. Keep the generated YAML separately.

Continue to the post configuration pipeline

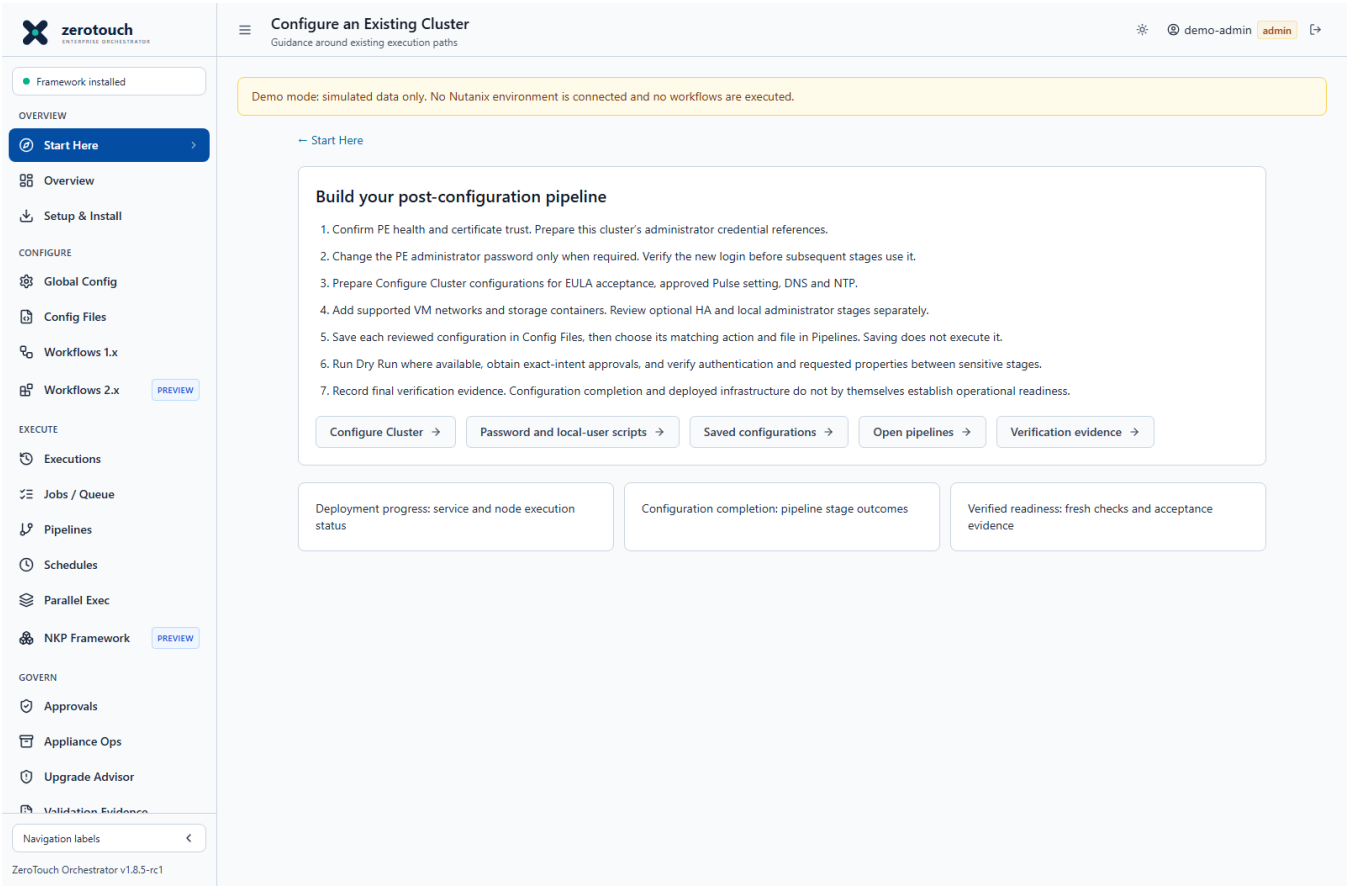


Figure 4. Post-configuration guidance and pipeline handoff. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Continue to Post-Configuration opens guidance with links to Configure Cluster, Scripts, saved configurations, Pipelines and Verification Evidence. It does not select a cluster, copy credentials, construct a pipeline or start work automatically.

Build a reviewed pipeline for each cluster: conditional PE admin password change with fresh authentication; EULA and approved Pulse; DNS/NTP; supported VM networks; storage containers with readback; optional HA and local users; PC registration and final independent verification. Keep sensitive stages narrowly scoped and use the matching saved file and action.

Separate deployment progress, configuration completion and verified readiness. A completed deployment or successful pipeline is not a substitute for fresh cluster-health, access and acceptance evidence. See docs/start-here.md and docs/pe-post-configuration-verification.md.

Architecture and operating boundaries

ZTF-Orchestrator is the configuration, execution-control and evidence layer. The target API or installed automation engine performs the infrastructure action. A green UI readiness indicator does not validate hardware support, prove successful imaging or establish cluster health.

Lane	Target and purpose	Boundary
ZTF 1.x	Installed legacy workflow and script CLI	Pin the compatible 1.x runtime separately from 2.x.
Classic Foundation VM	Foundation 5.x web service and native imaging payload	Administrator-only controlled mutation, image validation and explicit deployment confirmation.
Integrated Foundation Central	PC-hosted Foundation Central workflow	Different API, configuration and lifecycle from classic Foundation VM.
Standalone FCA	Foundation Central Appliance Lifecycle API	An accepted handoff is not proof of completed imaging.
Native Foundation	Planning, review and bounded provider adapters	Treat evidence gates and preview artifacts as planning unless an enabled adapter is independently validated.
ZTF 2.x IaC	Separate desired-state plan and apply runtime	Preview status and guarded execution do not establish production support.
NKP integration	Profiles, schema, binaries, readiness and safe phases	Preparation and evidence do not imply complete cluster provisioning.

Operational sequence: choose the correct backend; establish trusted connectivity and credentials; author the exact schema; validate; obtain approval; submit; monitor; verify the actual target; retain evidence. A successful subprocess exit is only one part of this sequence.

Installation and first login

This guide expands the installation options from the main README into step-by-step deployment procedures.

Release channels reviewed on 27 October 2026: v1.8.4 remains Latest stable and v1.8.5-rc1 remains the controlled-UAT prerelease. This revision covers merged application changes through 4a01072, including the consolidated v1.8.5-rc1-4a01072 appliance build. A fixed original RC tag does not automatically include later source changes.

Current Scope

Item	Current guidance
Current release baseline	v1.8.5-rc1 controlled-UAT candidate; v1.8.4 Latest stable
Existing appliance update method	Appliance update package plus host-side helper
Full QCOW2 appliance image	GitHub Actions artifact or durable internal artifact storage, not a GitHub Release binary
ZTF runtime	Legacy ZeroTouch Framework v1.5.2 for Workflows 1.x/Scripts 1.x, with a separate guarded ZTF v2.0.0 runtime for ZTF 2.x IaC
Historical docs	See Documentation Archive; archived procedures are traceability records, not current install paths

ZTF-Orchestrator keeps the legacy ZeroTouch Framework 1.x workflow/script CLI separate from the ZTF 2.x ztf plan/apply model. Keep ZTF_REF pinned to v1.5.2 for Workflows 1.x and Scripts 1.x. Use the separate ZTF 2.x runtime (ZTF2_REF, ZTF2_PATH, and ZTF2_COMMAND) for ZTF 2.x IaC, Workflows 2.x, and Scripts 2.x. ZTF 2.x checkouts remain blocked from the legacy workflow/script launcher.

For operator controlled/UAT-ready deployments, complete the relevant procedures in the runbook index and control matrix, capture evidence with the UAT evidence checklist, and keep the production readiness boundary with the deployment record.

Installation Options

Option	Best For	Storage	ZTF Location
One-command Linux/macOS	Quick host install	File backend	$ZTF_INSTALL_DIR/zerotouch-framework
One-command Windows	Quick workstation install	File backend	%USERPROFILE%\ztf\zerotouch-framework
Docker Compose	Recommended server install	PostgreSQL	/opt/zerotouch-framework and /opt/zerotouch-framework-2x inside the container
Docker Compose file backend	Local/simple Docker testing	File backend	/opt/zerotouch-framework and /opt/zerotouch-framework-2x inside the container
Appliance	VM/AHV deployment	PostgreSQL	/opt/zerotouch-framework and /opt/zerotouch-framework-2x inside the container
Manual	Development or custom layout	File backend by default	Configured by ZTF_PATH
Kubernetes	Starter cluster deployment	PostgreSQL	/opt/zerotouch-framework inside the container
Air-gapped	Disconnected environments	PostgreSQL or file	Prebuilt image or local clone

Common First Login Flow

1. Start ZTF-Orchestrator.
2. Watch the terminal, Docker logs, or service journal for the first-run credential block.
3. Open the app in a browser.
4. Sign in as admin with the generated password.
5. Go to **Users** and create named admin/operator/viewer accounts.
6. Store the generated password securely or rotate it immediately.

If the first-run password is missed, use an authorized account recovery procedure after preserving operational state. Do not delete users, volumes or database records to regain access. For PostgreSQL deployments, involve the trusted database or appliance administrator.

Before controlled UAT, create named users for the required admin, operator, and viewer roles, then rehearse RB-001 start/stop/restart, RB-002 backup/restore, and RB-006 emergency stop.

Option A One Command Linux macOS

Use this when you want a quick install directly on a Linux or macOS host.

Prerequisites

1. Python 3.10 or newer.
2. pip.
3. git.
4. Network access to GitHub and PyPI, unless using internal mirrors.
5. Access from the host to the Nutanix systems that ZTF workflows will target.

Install

1. Choose an install directory. The default is:

```
~/ztf
```

2. Run the installer:

```
curl -fsSLo install.sh https://raw.githubusercontent.com/VirtuArchitect/ZTF-Orchestrator/main/install.sh
# Review the script and approved source revision before execution.
bash install.sh
```

3. For a custom port or install directory, run:

```
ZTF_PORT=8080 ZTF_INSTALL_DIR=/opt/ztf ZTF_REF=v1.5.2 bash install.sh
```

4. Confirm the installer created:

```
<install-dir>/ZTF-Orchestrator
<install-dir>/zerotouch-framework
<install-dir>/venv
```

5. Open:

```
http://localhost:5001
```

Use the custom port if you set ZTF_PORT.

Restart Later

1. Activate the shared virtual environment:

```
source ~/ztf/venv/bin/activate
```

2. Start the server:

```
ZTF_PATH=~/ztf/zerotouch-framework \
ZTF_PYTHON=~/ztf/venv/bin/python3 \
python ~/ztf/ZTF-Orchestrator/server.py
```

Validate

1. Open /health:

```
curl http://localhost:5001/health
```

2. In the UI, open **Setup & Install** or **Settings** and confirm the runtime paths:

• **ZTF 1.x Legacy** points to the local zerotouch-framework checkout for Workflows 1.x and Scripts 1.x.

• **ZTF 2.x IaC** points to the separate ZTF 2.x checkout and ztf command before using ZTF 2.x IaC, Workflows 2.x, or Scripts 2.x.

Option B One Command Windows PowerShell

Use this when running ZTF-Orchestrator locally on a Windows workstation.

Prerequisites

1. Python 3.10 or newer on PATH.
2. pip.
3. Git for Windows on PATH.
4. PowerShell.
5. Network access to GitHub and PyPI, unless using internal mirrors.

Install

1. Open PowerShell as your normal user.
2. Optional: avoid the common Hyper-V reserved range around port 5001:

```
$env:ZTF_PORT = "8080"
```

3. Run:

```
   iex ((New-Object Net.WebClient).DownloadString('https://raw.githubusercontent.com/VirtuArchitect/ZTF-Orchestrator/main/install.ps1'))
```

4. The installer creates:

```
   %USERPROFILE%\ztf\ZTF-Orchestrator
   %USERPROFILE%\ztf\zerotouch-framework
   %USERPROFILE%\ztf\venv
```

5. Open:

```
http://localhost:5001
```

Use the custom port if you set ZTF_PORT.

Restart Later

1. Open PowerShell.
2. Activate the virtual environment:

```
   & "$env:USERPROFILE\ztf\venv\Scripts\Activate.ps1"
```

3. Start the server:

```
$env:ZTF_PATH = "$env:USERPROFILE\ztf\zerotouch-framework"
$env:ZTF_PYTHON = "$env:USERPROFILE\ztf\venv\Scripts\python.exe"
python "$env:USERPROFILE\ztf\ZTF-Orchestrator\server.py"
```

Validate

1. Open:

http://localhost:<port>/health

2. Confirm **Settings > ZTF Installation Path** points to:

```
C:\Users\<you>\ztf\zerotouch-framework
```

Option C Docker Compose with PostgreSQL

Use this for the recommended server deployment. The app container bakes in ZeroTouch Framework at build time under `/opt/zerotouch-framework`.

Prerequisites

1. Docker Engine or Docker Desktop.
2. Docker Compose plugin.
3. Network access to GitHub during image build, unless using an internal mirror.
4. Network access to Docker Hub for postgres:16-alpine, unless using an internal registry.

Install

1. Clone the repository:

```
git clone https://github.com/VirtuArchitect/ZTF-Orchestrator.git
cd ZTF-Orchestrator
```

2. Create .env:

```
cp .env.example .env
```

3. Edit .env and set a unique password:

```
POSTGRES_PASSWORD=<unique-password>
ZTF_DATABASE_URL=postgresql://ztf:<unique-password>@postgres:5432/ztf_orchestrator
ZTF_REF=v1.5.2
ZTF_HOST_PORT=15001
```

4. Build and start:

```
docker compose up -d --build
```

5. Watch logs and copy the first-run admin password:

```
docker compose logs -f ztf-orchestrator
```

6. Open:

```
http://localhost:15001
```

Validate

1. Check containers:

```
docker compose ps
```

2. Check health:

```
curl http://localhost:15001/health
```

3. Confirm the baked frameworks:

```
docker exec -it ztf-orchestrator ls -la /opt/zerotouch-framework
 docker exec -it ztf-orchestrator ls -la /opt/zerotouch-framework-2x
```

4. Confirm the baked framework runtime dependencies:

```
  docker exec -w /opt/zerotouch-framework ztf-orchestrator \
    /opt/ztf-python/bin/python -c "import rainbow_logging_handler; import ntnx_iam_py_client; import scrypt;
import framework.helpers.log_utils"
  docker exec -w /opt/zerotouch-framework ztf-orchestrator \
    /opt/ztf-python/bin/python main.py --help
  docker exec -w /opt/zerotouch-framework-2x ztf-orchestrator \
    /opt/ztf2-python/bin/ztf --help
```

5. Confirm persistent data volume:

```
docker volume ls | grep ztf
```

Stop and Restart

```
docker compose stop
docker compose start
```

To stop and remove containers while keeping volumes:

docker compose down

Do not remove volumes unless you intend to delete users, history, configs, and database data.

Option C2 Docker Compose File Backend

Use this for simple local Docker testing when PostgreSQL is not required.

Install

1. Clone the repository:

```
git clone https://github.com/VirtuArchitect/ZTF-Orchestrator.git
cd ZTF-Orchestrator
```

2. Start the file-backed stack:

```
ZTF_REF=v1.5.2 docker compose -f docker-compose.file.yml up -d --build
```

3. Read the generated admin password:

```
docker compose -f docker-compose.file.yml logs -f ztf-orchestrator
```

4. Open:

```
http://localhost:15001
```

Validate

```
docker compose -f docker-compose.file.yml ps
curl http://localhost:15001/health
docker exec -it ztf-orchestrator ls -la /opt/zerotouch-framework
```

Option D Appliance Deployment

Use this for a small Linux VM on AHV or another virtualization platform. The appliance Compose file pulls the published container image and starts a local PostgreSQL container.

For a pre-built AHV-importable QCOW2 workflow, see Appliance Kit. The repository includes a **Build AHV Appliance Image** GitHub Actions workflow and a local appliance/scripts/build-ahv-qcow2.sh wrapper for Packer/QEMU builds.

Prerequisites

1. Ubuntu Server 24.04 LTS VM for the reference appliance path.

2. 2 vCPU, 4-8 GB RAM, and 80-100 GB disk.

3. Network access to GitHub and a container registry, unless using internal mirrors or a preloaded image.

4. Network access from the VM to Prism Central, Prism Element, Foundation Central, DNS, NTP, and other deployment services.

5. `sudo` access.

Install on a Fresh VM

1. Install basic packages:

```
sudo apt-get update
sudo apt-get install -y git curl ca-certificates
```

2. Clone the source:

```
git clone https://github.com/VirtuArchitect/ZTF-Orchestrator.git /opt/ztf-orchestrator-source
```

3. Run first boot:

```
sudo bash /opt/ztf-orchestrator-source/appliance/scripts/firstboot.sh
```

4. To pin a released image:

```
sudo ZTF_ORCHESTRATOR_VERSION=<version> \
  bash /opt/ztf-orchestrator-source/appliance/scripts/firstboot.sh
```

5. Open:

```
http://<vm-ip>:5001
```

Validate

1. Check service status:

```
sudo systemctl status ztf-orchestrator
```

2. Watch logs:

```
sudo journalctl -u ztf-orchestrator -f
```

3. Check Compose:

```
cd /opt/ztf-orchestrator
sudo docker compose --env-file /opt/ztf-orchestrator/.env \
  -f /opt/ztf-orchestrator/appliance/docker-compose.appliance.yml ps
```

4. Confirm health:

```
curl http://localhost:5001/health
```

TLS Recommendation

For shared team access, place nginx, a load balancer, or another TLS reverse proxy in front of the appliance. Do not expose port 5001 directly to untrusted networks.

Pre-Built QCOW2 Summary

1. Run **Actions > Build AHV Appliance Image** in GitHub or run the local Packer wrapper from a Linux build host. The default QCOW2 build locally builds the ZTF-Orchestrator container image inside the appliance and bakes ZeroTouch Framework v1.5.2 into /opt/zerotouch-framework plus ZeroTouch Framework v2.0.0 into /opt/zerotouch-framework-2x inside that container.

The GitHub **Run workflow** button is only visible to repository users with permission to manually dispatch workflows, typically write-level access or higher. Public and read-only users should download published artifacts, ask a maintainer to run the workflow, or open the **AHV appliance build request** issue template. This is a GitHub permission rule rather than a ZTF-Orchestrator workflow setting.

Maintainers can run the workflow from the GitHub CLI:

```
gh workflow run ahv-appliance-image.yml \
  -f artifact_profile=standard \
  -f source_ref=main \
  -f image_version=latest \
  -f ztf_framework_ref=v1.5.2
```

Pushing a version tag matching v* automatically builds all appliance profiles and publishes release checksum/manifest metadata.

2. Download the. qcow2 and checksum artifact from the workflow run. GitHub Releases publish checksum and manifest metadata only because QCOW2 files can exceed the GitHub Release 2 GiB per-file asset limit.

3. Upload the. qcow2 into Prism Central or Prism Element image management.
4. Create a VM with 2 vCPU, 4-8 GB RAM, and 80-100 GB disk.
5. Attach a management VLAN and provide cloud-init or image-process credentials.
6. Boot the VM and wait for `ztf-orchestrator-firstboot` to complete.
7. Retrieve the generated admin password from:

```
sudo journalctl -u ztf-orchestrator -n 200 --no-pager
```

Option E Manual Install

Use this for development, custom service managers, or environments where Docker is not available.

Prerequisites

1. Python 3.10 or newer.
2. pip.
3. Git.
4. A ZeroTouch Framework 1.x checkout, preferably v1.5.2.

Install

1. Clone ZTF-Orchestrator:

```
git clone https://github.com/VirtuArchitect/ZTF-Orchestrator.git
cd ZTF-Orchestrator
```

2. Clone ZeroTouch Framework:

```
git clone --depth 1 --branch v1.5.2 https://github.com/nutanixdev/zerotouch-framework.git ../zerotouch-framework
```

3. Create a virtual environment:

```
python3 -m venv venv
source venv/bin/activate
```

On Windows:

```
python -m venv venv
.\venv\Scripts\Activate.ps1
```

4. Install ZTF-Orchestrator dependencies:

```
pip install -r requirements.txt
```

5. Install ZeroTouch Framework dependencies. The exact requirements file depends on the checked-out ZTF version:

```
pip install -r ../zerotouch-framework/requirements/prod.txt
```

If that file is not present, inspect the framework requirements/ directory and install the appropriate requirements file for your workflow set.

6. Start the server:

```
ZTF_PATH="$(pwd)/../zerotouch-framework" \
ZTF_PYTHON="$(pwd)/venv/bin/python3" \
python server.py
```

On Windows:

```
$env:ZTF_PATH = (Resolve-Path ..\zerotouch-framework).Path
$env:ZTF_PYTHON = (Resolve-Path .\venv\Scripts\python.exe).Path
python server.py
```

7. Open:

```
http://localhost:5001
```

Validate

```
curl http://localhost:5001/health
```

In the UI, confirm **Settings > ZTF Installation Path** points to the cloned framework.

Option F Kubernetes Starter Manifests

Use this only as a starter deployment. Execution workers currently run inside the Flask process, so the starter manifests intentionally use one replica.

Prerequisites

1. A Kubernetes cluster.
2. A namespace where you can create Deployments, Services, Secrets, and PVCs.
3. A container image available to the cluster.
4. PostgreSQL from the included manifest or a managed PostgreSQL endpoint.

Install

1. Build and publish an image accessible to the cluster or use the published image if your cluster can reach it.
2. Edit k8s/secret.example.yaml and set production secrets.
3. Apply manifests:

```
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/secret.example.yaml
kubectl apply -f k8s/configmap.yaml
kubectl apply -f k8s/postgres.yaml
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
```

4. If using managed PostgreSQL, omit postgres.yaml and set `ZTF_DATABASE_URL` to the managed endpoint.
5. Expose the service through your preferred internal ingress or port-forward for testing:

kubectl -n ztf-orchestrator port-forward svc/ztf-orchestrator 5001:5001

6. Open:

```
http://localhost:5001
```

Validate

```
kubectl -n ztf-orchestrator get pods
kubectl -n ztf-orchestrator logs deploy/ztf-orchestrator
curl http://localhost:5001/health
```

Air Gapped Installation

Use this when the target site cannot reach GitHub, GHCR, Docker Hub, or PyPI. The most reliable approach is to build and verify artifacts in a connected staging environment, then transfer them into the disconnected site.

Connected Staging Environment

1. Clone the repository:

```
git clone https://github.com/VirtuArchitect/ZTF-Orchestrator.git
cd ZTF-Orchestrator
```

2. Build the application image with ZTF 1.x v1.5.2 and ZTF 2.x v2.0.0 baked in:

```
docker build \
  --build-arg ZTF_REPO_URL=https://github.com/nutanixdev/zerotouch-framework.git \
  --build-arg ZTF_REF=v1.5.2 \
  --build-arg ZTF2_REPO_URL=https://github.com/nutanixdev/zerotouch-framework.git \
  --build-arg ZTF2_REF=v2.0.0 \
  -t ztf-orchestrator:airgap-<version> .
```

3. Pull the PostgreSQL image if using the PostgreSQL Compose deployment:

```
docker pull postgres:16-alpine
```

4. Export images:

```
docker save ztf-orchestrator:airgap-<version> -o ztf-orchestrator-airgap-<version>.tar
docker save postgres:16-alpine -o postgres-16-alpine.tar
```

5. Export the repository content needed for Compose files and docs:

```
git archive --format=tar.gz --output ZTF-Orchestrator-source.tar.gz HEAD
```

6. If using NKP features, stage the NKP framework and NKP bundle separately. Large NKP bundles should be copied to persistent storage on the target VM and registered by path in **NKP Framework > Binaries**.

7. Transfer these files using your approved removable-media or artifact process:

```
ztf-orchestrator-airgap-<version>.tar
postgres-16-alpine.tar
ZTF-Orchestrator-source.tar.gz
NKP framework archive, if used
NKP bundle/artifacts, if used
```

Connected Staging for a Portable AHV Appliance

Use this path when the disconnected site should receive a ready-to-import QCOW2 rather than container tar files.

Current AHV appliance builds use named artifact profiles. Older single-image workflow lines are historical and should not be used for current appliance builds.

1. Choose the QCOW2 build path.

GitHub Actions inputs:

```
artifact_profile: standard, airgap, minimal, or all
source_ref: main or a release tag
image_version: latest or the release tag
qemu_accelerator: tcg
build_container_image: true
pull_container_images: true
ztf_framework_ref: v1.5.2
bake_nkp_framework: true
nkp_framework_ref: main
nkp_bundle_urls: optional comma-separated bundle URLs
```

Profiles:

standard: connected staging appliance with baked app, PostgreSQL, ZTF, and NKP framework
airgap: portable disconnected-site appliance with baked app, PostgreSQL, ZTF, and NKP framework
minimal: smaller bootstrap appliance; defers image pulls and NKP staging
all: builds standard, airgap, and minimal artifacts in one workflow run

The workflow installs QEMU plus xorriso; Packer uses xorriso to create the temporary cloud-init ISO attached during image customization.

2. Optional: include NKP bundle artifacts before a local build.

Copy local files into:

```
appliance/preload/bundles/
```

Or provide connected staging URLs:

```
ZTF_NKP_BUNDLE_URLS="https://mirror.example/nkp-bundle.tar.gz"
```

If you need to use a reviewed local NKP framework checkout instead of cloning from Git, copy it into:

```
appliance/preload/nkp-zerotouch-framework/
```

3. Build the QCOW2 from GitHub Actions or a connected Linux build host.

A local Linux build host needs Packer, QEMU, and an ISO creation tool such as xorriso or mkisofs.

Local Linux build host:

```
cd appliance
VERSION=<version> \
ZTF_ORCHESTRATOR_VERSION=<version> \
ZTF_BUILD_CONTAINER_IMAGE=true \
ZTF_PULL_CONTAINER_IMAGES=true \
ZTF_FRAMEWORK_REF=v1.5.2 \
ZTF_BAKE_NKP_FRAMEWORK=true \
ZTF_NKP_FRAMEWORK_REF=main \
QEMU_ACCELERATOR=kvm \
scripts/build-ahv-qcow2.sh
```

4. Verify the checksum:

```
cd appliance/packer/output
sha256sum -c SHA256SUMS
```

5. Transfer only the appliance artifacts required by the site:

```
ztf-orchestrator-appliance-<version>.qcow2
SHA256SUMS
```

6. Archive the generated QCOW2 packages outside GitHub Actions.

GitHub Actions artifacts are retained only for the workflow artifact retention period. For a durable release archive, download the profile ZIPs and release metadata, verify checksums, and store them together in Nutanix Files, an internal artifact repository, object storage, OneDrive/SharePoint release storage, or offline transfer media.

Retain:

```
ztf-orchestrator-ahv-qcow2-standard-<version>.zip
ztf-orchestrator-ahv-qcow2-airgap-<version>.zip
ztf-orchestrator-ahv-qcow2-minimal-<version>.zip
AHV-APPLIANCE-ARTIFACTS-<version>.md
SHA256SUMS-standard-<version>.txt
SHA256SUMS-airgap-<version>.txt
SHA256SUMS-minimal-<version>.txt
```

7. Record archive status in **Appliance Ops**.

After login, open **Appliance Ops** from the sidebar and create one archive record for each retained profile. Record the durable archive location, checksum file, SHA-256 digest, workflow/release links, and artifact expiry timestamp. Mark each record verified after checksum validation.

The same page also provides first-boot path checks, NKP profile readiness review, and ZeroTouch Framework compatibility mode status.

The appliance includes the ZTF-Orchestrator source checkout and a Docker image with ZTF 1.x baked into `/opt/zerotouch-framework` and ZTF 2.x baked into `/opt/zerotouch-framework-2x` inside the container. The ZTF 2.x command path is /opt/ztf2-python/bin/ztf. The first boot can therefore start without GitHub Container Registry access for the application image. PostgreSQL is also preloaded when ZTF_PULL_CONTAINER_IMAGES=true succeeds during the connected build. When ZTF_BAKE_NKP_FRAMEWORK=true, the NKP framework is staged on the appliance host and mounted into the container at:

```
/var/lib/ztf-orchestrator/nkp-zerotouch-framework
```

Preloaded NKP bundles are mounted into:

```
/var/lib/ztf-orchestrator/bundles
```

Disconnected AHV Appliance Import

1. Download and extract the GitHub Actions artifact from the successful **Build AHV Appliance Image** run.

The matching GitHub Release contains checksum files and an artifact manifest, but the QCOW2 images are downloaded from the Actions run artifacts because GitHub Release assets have a 2 GiB per-file limit.

Artifact names include the selected appliance profile:

```
ztf-orchestrator-ahv-qcow2-standard-<ref>
ztf-orchestrator-ahv-qcow2-airgap-<ref>
ztf-orchestrator-ahv-qcow2-minimal-<ref>
```

Expected files inside the artifact:

```
ztf-orchestrator-appliance-<profile>-<ref>.qcow2
SHA256SUMS-<profile>-<ref>.txt
```

2. Verify the checksum.

```
cd <artifact-directory>
sha256sum -c SHA256SUMS-<profile>-<ref>.txt
```

3. Upload the QCOW2 into Prism Central or Prism Element image management.

Use **Image Configuration** or **Infrastructure > Compute & Storage > Images**, depending on the Prism version.

Recommended settings:

```
Image type: Disk
Source: uploaded QCOW2
Storage container: site-approved container
```

4. Create a VM from the image with 2 vCPU, 4-8 GB RAM, the imported QCOW2 disk, and the target management network.

5. Provide site-approved administrator access.

The sealed image locks the temporary Packer ubuntu build account. Inject an administrator SSH key or account using cloud-init, AHV guest customization, or your image process.

Minimal cloud-init example:

```
#cloud-config
users:
  - name: ztfadmin
    groups: sudo
    shell: /bin/bash
    sudo: ALL=(ALL) NOPASSWD:ALL
    ssh_authorized_keys:
      - ssh-ed25519 <public-key> <owner>
ssh_pwauth: false
```

6. Boot the VM and wait for first boot:

```
sudo systemctl status ztf-orchestrator-firstboot
sudo systemctl status ztf-orchestrator
cd /opt/ztf-orchestrator
sudo docker compose --env-file /opt/ztf-orchestrator/.env \
  -f /opt/ztf-orchestrator/appliance/docker-compose.appliance.yml ps
```

The first-boot script creates /opt/ztf-orchestrator/.env and the systemd service passes it explicitly to Docker Compose with --env-file. Do not copy this secret-bearing file into /opt/ztf-orchestrator/appliance/.

7. Retrieve the generated application admin password:

```
sudo journalctl -u ztf-orchestrator -n 300 --no-pager
```

If needed, inspect the container logs:

```
cd /opt/ztf-orchestrator
sudo docker compose --env-file /opt/ztf-orchestrator/.env \
  -f /opt/ztf-orchestrator/appliance/docker-compose.appliance.yml logs ztf-orchestrator
```

8. Open the UI and complete initial configuration:

```
http://<appliance-ip>:5001
```

Sign in as admin, rotate credentials, create named users, and restrict access to the management network or a TLS reverse proxy.

9. Validate the baked app image, ZTF framework, and NKP preload:

```
cd /opt/ztf-orchestrator
sudo docker image ls | grep ztf-orchestrator
sudo docker image ls | grep postgres
sudo docker compose --env-file /opt/ztf-orchestrator/.env \
  -f /opt/ztf-orchestrator/appliance/docker-compose.appliance.yml exec ztf-orchestrator \
  ls -la /opt/zerotouch-framework
sudo docker compose --env-file /opt/ztf-orchestrator/.env \
  -f /opt/ztf-orchestrator/appliance/docker-compose.appliance.yml exec ztf-orchestrator \
  ls -la /var/lib/ztf-orchestrator/nkp-zerotouch-framework
sudo docker compose --env-file /opt/ztf-orchestrator/.env \
  -f /opt/ztf-orchestrator/appliance/docker-compose.appliance.yml exec ztf-orchestrator \
  ls -la /var/lib/ztf-orchestrator/bundles
curl http://localhost:5001/health
```

10. Confirm the NKP Framework UI can see the preloaded paths:

```
/var/lib/ztf-orchestrator/nkp-zerotouch-framework
/var/lib/ztf-orchestrator/bundles/
```

11. In the UI, register the NKP framework path and any NKP binary or bundle paths under **NKP Framework > Binaries**. Use **Check CLI Compatibility** after registration.

12. Configure site integrations:

Global Config: Prism Central, Foundation Central, DNS/NTP/IPAM, registry
Config Files: site YAML profiles and environment files
Users: named operators and admin accounts
Schedules/Approvals: maintenance windows and approval gates
Validation Evidence: deployment evidence retention

13. Troubleshoot first boot if needed:

```
sudo journalctl -u ztf-orchestrator-firstboot -n 300 --no-pager
sudo journalctl -u ztf-orchestrator -n 300 --no-pager
cd /opt/ztf-orchestrator
sudo docker compose --env-file /opt/ztf-orchestrator/.env \
  -f /opt/ztf-orchestrator/appliance/docker-compose.appliance.yml ps
sudo docker compose --env-file /opt/ztf-orchestrator/.env \
  -f /opt/ztf-orchestrator/appliance/docker-compose.appliance.yml logs --tail=200
sudo ls -l /opt/ztf-orchestrator/.env
sudo grep -E '^(ZTF_ORCHESTRATOR_VERSION|ZTF_HOST_BIND|ZTF_LOG_LEVEL)=' /opt/ztf-orchestrator/.env
```

Keep `/opt/ztf-orchestrator/.env` local to the appliance because it contains the generated database password. Do not copy it into `/opt/ztf-orchestrator/appliance/.`

Disconnected Target Environment

1. Install Docker Engine and the Compose plugin using your site-approved offline package process.

2. Load images:

```
docker load -i ztf-orchestrator-airgap-<version>.tar
 docker load -i postgres-16-alpine.tar
```

3. Extract the source archive:

```
mkdir -p /opt/ztf-orchestrator
tar -xzf ZTF-Orchestrator-source.tar.gz -C /opt/ztf-orchestrator
cd /opt/ztf-orchestrator
```

4. Create .env:

```
cp .env.example .env
```

5. Edit .env:

```
POSTGRES_PASSWORD=<unique-password>
ZTF_DATABASE_URL=postgresql://ztf:<unique-password>@postgres:5432/ztf_orchestrator
ZTF_REF=v1.5.2
```

6. Update docker-compose.yml to use the loaded image tag, or retag the loaded image as the Compose default:

```
docker tag ztf-orchestrator:airgap-<version> ztf-orchestrator:latest
```

7. Start:

```
docker compose up -d
```

8. Check logs:

```
docker compose logs -f ztf-orchestrator
```

9. Validate the baked frameworks:

```
docker exec -it ztf-orchestrator ls -la /opt/zerotouch-framework
docker exec -it ztf-orchestrator ls -la /opt/zerotouch-framework-2x
docker exec -w /opt/zerotouch-framework-2x ztf-orchestrator \
  /opt/ztf2-python/bin/ztf --help
curl http://localhost:5001/health
```

10. Open:

```
 http://<host-ip>:5001
```

Air Gapped Internal Mirrors

If your disconnected environment has internal mirrors rather than no network at all, use these settings:

ZTF_REPO_URL=<internal Git mirror for zerotouch-framework>
ZTF_REF=v1.5.2
ZTF2_REPO_URL=<internal Git mirror for zerotouch-framework>
ZTF2_REF=v2.0.0

For manual installs, use a local PyPI mirror or pre-downloaded wheelhouse:

pip install --no-index --find-links /path/to/wheelhouse -r requirements.txt

For the in-app Setup page, use an internal Git mirror URL. The backend allowlist for install sources must include that mirror before operators can clone from it.

Air Gapped NKP Notes

1. Stage the NKP framework under a persistent path such as:

   /var/lib/ztf-orchestrator/nkp-zerotouch-framework

2. Set **Settings > NKP Framework Path** or ZTF_NKP_PATH to that path.
3. Stage the NKP CLI/bundle on the VM or inside a mounted persistent volume.
4. In **NKP Framework > Binaries**, register the server-visible path.
5. Run **Check CLI Compatibility** before generating or submitting NKP phases.

Post Install Hardening Checklist

1. Create named admin/operator/viewer users and rotate the initial admin password.
2. Keep the app on an internal management network.
3. Put nginx or another TLS reverse proxy in front for team access.
4. Restrict access to admins and operators who are authorized to run Nutanix workflows.
5. Configure ZTF_DATA_DIR backups or PostgreSQL backups.
6. Validate /health.
7. Confirm ZTF_PATH points to a ZeroTouch Framework 1.x checkout.
8. Run a non-production smoke workflow before using live Nutanix targets.
9. Record the deployment version, ZTF ref, storage backend, and backup plan.

First login acceptance checks

1. Verify /health and record the full Installed Build from Settings > About.

2. Sign in using the generated first-run password through a trusted connection. Rotate it and create named administrators, operators and viewers.

3. Configure HTTPS at the reverse proxy and restrict access to the management network. The localhost Compose binding is deliberate; do not expose an unauthenticated backend publicly.

4. Confirm PostgreSQL and persistent volumes, then create and download a backup. Record the location of configs and logs as well as database state.

5. Verify runtime paths, pinned framework refs and credentials with a non-mutating action. Confirm that an unauthorized role cannot submit a privileged action.

6. Rehearse restart and restore before onboarding a team. Record ownership, maintenance windows, retention and emergency stop arrangements.

Screen reference and daily operation

For controlled-UAT CVM SSH and Dell iDRAC password changes, see the credential rotation guide. The workflow is scoped to AOS 7.5.1.8 and requires manual Nutanix host IPMI credential synchronization.

This guide explains the ZTF-Orchestrator user interface, sidebar menu items, major features, and the operational purpose of each screen.

ZTF-Orchestrator is an internal operations console for preparing, validating, executing, and auditing Nutanix ZeroTouch Framework 1.x workflows/scripts, separate governed ZTF 2.x IaC plan/apply jobs, and guarded NKP preparation phases. It is the control plane and evidence layer. ZeroTouch Framework, NKP ZeroTouch Framework, Prism Central, Prism Element, and Foundation Central remain the systems that perform the underlying infrastructure work.

The static demo uses simulated data. Treat it as a product tour, not as proof that a deployment has validated live infrastructure.

Roles And Access

The visible menu depends on the signed-in user's role.

Role	Intended user	Access summary
Admin	Platform owner or trusted operator	Full access, including user management, audit log, settings, backup restore, approvals, and destructive record cleanup.
Operator	Infrastructure operator	Can prepare and run permitted governed operations and read evidence. Cannot run administrator-only PE local-user creation or other restricted actions, manage users or access admin-only audit screens.
Viewer	Reviewer, auditor, or stakeholder	Can view operational state, configs, jobs, schedules, evidence, drift results, appliance state, NKP state, and upgrade assessments. Viewer actions are read-only where execution or mutation would occur.

All protected pages require login. If a user opens a page their role cannot access, the app shows an access-denied page instead of the requested feature.

Navigation Shell

Start Here is the default landing page when this user has no saved browser preference. Overview provides the existing operations dashboard. Advanced Automation keeps Scripts, YAML Studio and specialist tools directly accessible. Menu visibility and server permissions still depend on role; navigation guidance never grants execution access.

The logo at the top returns to the Dashboard. The status indicator below the logo shows whether the ZeroTouch Framework path is currently detected. The sidebar can be expanded with labels or collapsed to compact icon navigation. When collapsed, hover over an icon to see the menu label.

Common Operating Flow

Most production-style work follows this path:

1. Configure runtime paths and connection defaults in Settings.
2. Define credentials, vault behaviour, and IPAM in Global Config.
3. Generate, upload, or edit YAML in YAML Studio, Config Files, Workflows, NKP Framework, or Scripts.
4. Validate the YAML and run a dry run where available.

5. Request approval for governed workflows or NKP phases.
6. Submit the work as a durable job through Workflows, Scripts, Pipelines, Schedules, Parallel Exec, or NKP Framework.
7. Monitor Jobs / Queue and Execution History.
8. Capture Validation Evidence, review Drift Detection, and consult Audit Log for governed records.

Overview Menu

Overview Dashboard

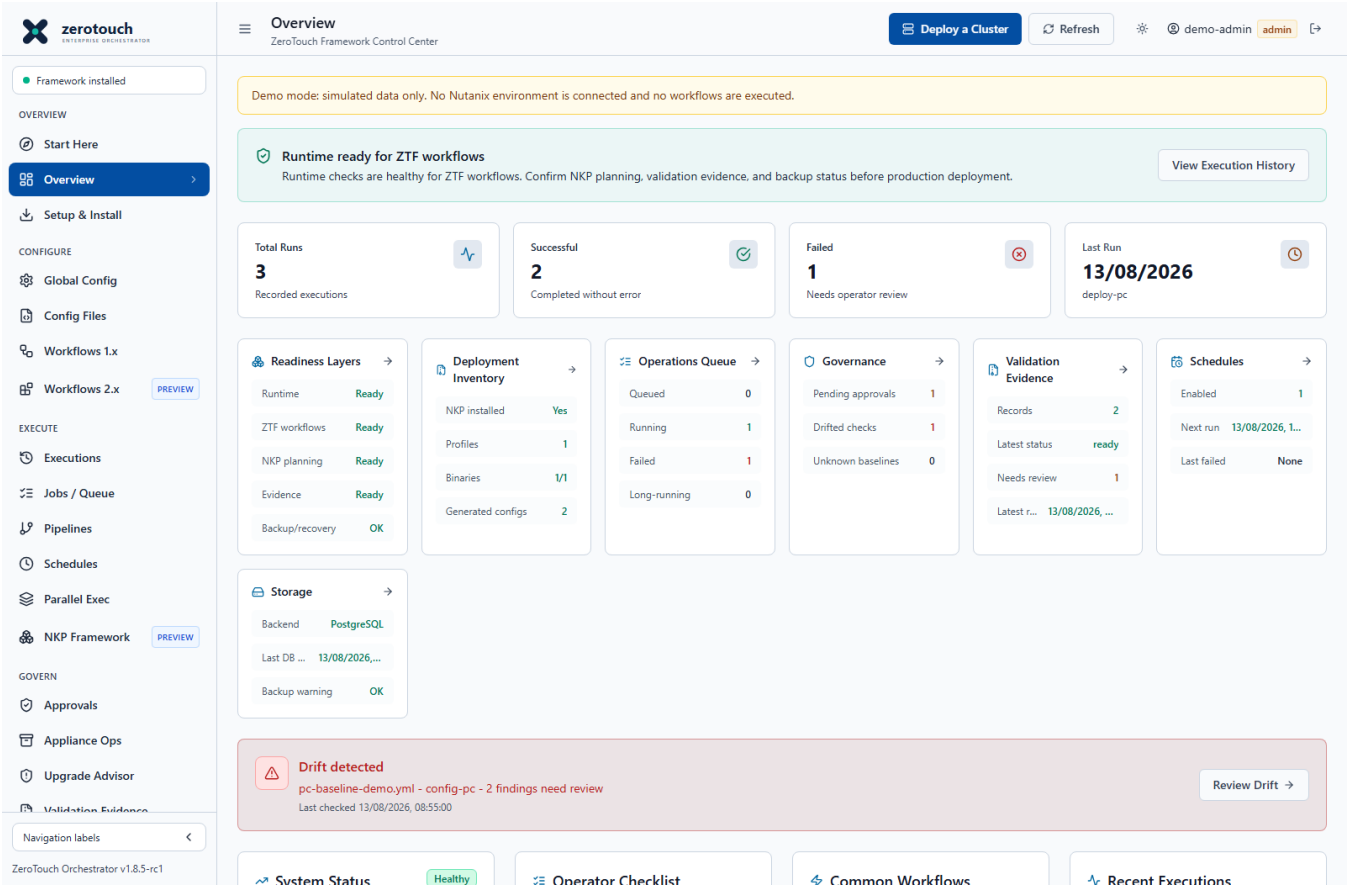


Figure 5. Dashboard. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Operating procedure

1. Read runtime readiness and Installed Build before launching work.
2. Review queued, failed and interrupted jobs, pending approvals and schedule warnings.
3. Follow the relevant quick link, inspect its detailed record and assign an owner.
4. Refresh after remediation; do not infer infrastructure health from console health alone.

Use Dashboard as the daily operations cockpit.

The Dashboard summarizes framework readiness, deployment inventory, queue pressure, governance state, validation evidence posture, schedules, storage, database backup state, and recent activity. It also provides quick links into common actions such as creating configuration, opening workflow launchers, checking jobs, and exporting evidence.

Key functions:

Area	Function
Refresh	Manually reloads dashboard data. The screen also refreshes periodically.
Readiness Layers	Shows ZTF and NKP readiness signals so operators can see whether the required framework pieces are configured.
Deployment Inventory	Summarizes saved profiles, configs, executions, and discovered operational records.
Operations Queue	Highlights active, queued, running, failed, interrupted, or long-running jobs that need attention.
Governance	Shows pending approvals and policy pressure.
Validation Evidence	Indicates whether evidence packages have been captured for recent work.
Schedules	Shows enabled schedules, next-run information, and recent schedule failures.
Storage	Shows backend mode and backup posture, including PostgreSQL backup signals when applicable.
Quick Actions	Routes operators to common starting points without hunting through the full sidebar.

Use Dashboard first when you need a fast answer to "is the orchestrator ready, busy, blocked, or waiting on governance?"

Setup Install

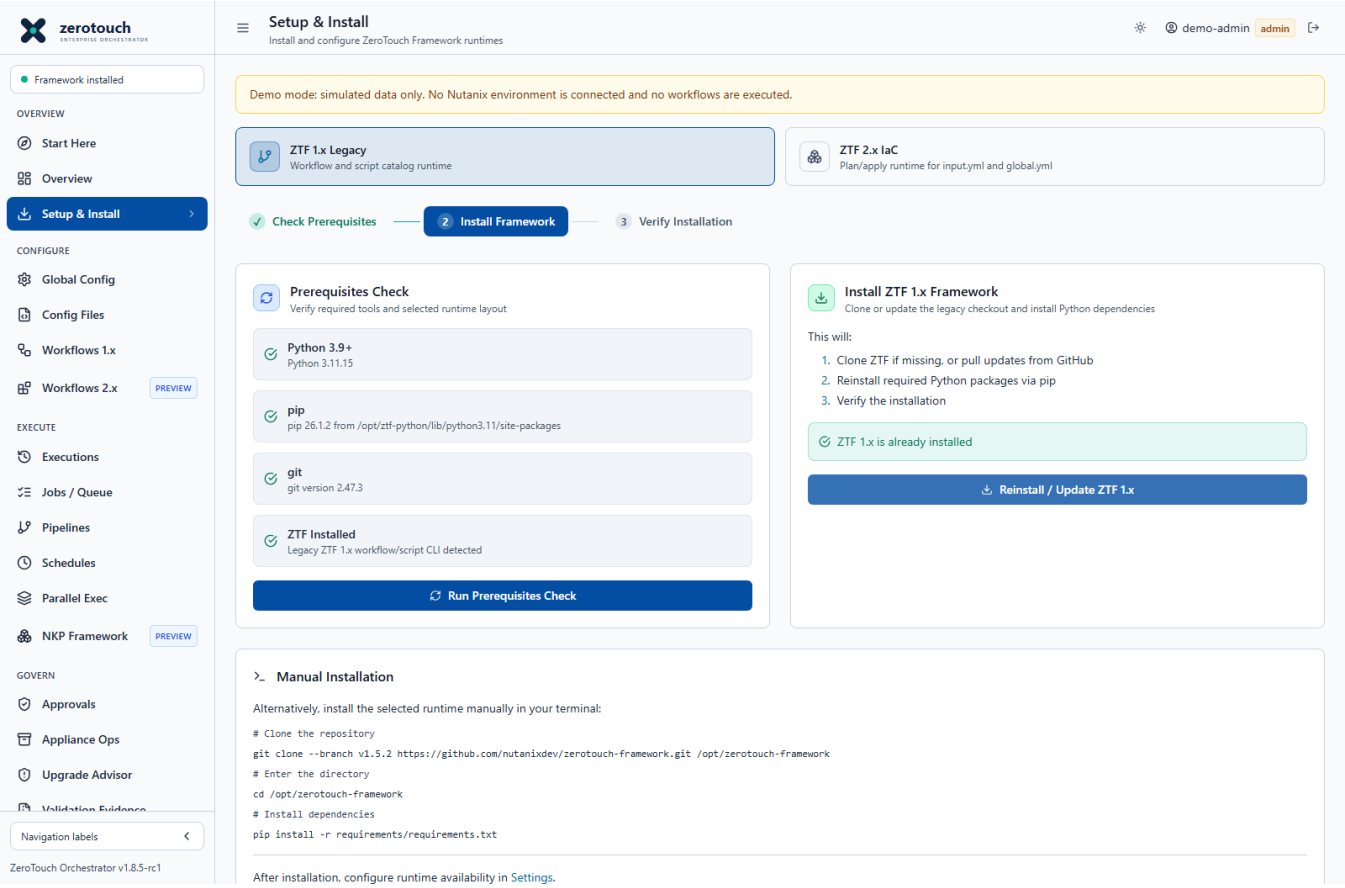


Figure 6. Setup & install. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Operating procedure

1. Choose the ZTF 1.x Legacy or separate ZTF 2.x IaC runtime.
2. Run the system check and correct missing prerequisites or paths.
3. Install only the approved repository and pinned framework ref.
4. Review installation output, then verify Runtime settings and a non-mutating action.

Use Setup & Install to install or detect ZeroTouch Framework runtimes from the UI. Select **ZTF 1.x Legacy** for the existing workflow/script catalog, or **ZTF 2.x IaC** for the separate plan/apply runtime.

Key functions:

Function	Purpose
Runtime selector	Chooses the ZTF 1.x or ZTF 2.x setup lane.
System check	Verifies that required local tools and selected runtime paths are available.
Install framework	Clones or installs the selected ZeroTouch Framework runtime from the configured source.
Installation output	Shows install progress and command output for troubleshooting.
Settings handoff	Directs operators to Settings when paths or framework location need adjustment.

ZTF 2.x installs into its own checkout and CLI runtime. After installation, an admin still controls whether the runtime is available for jobs from **Settings > Runtime**. This screen is for bootstrap and remediation. Once the framework is detected, most day-to-day work happens in Configure, Execute, and Govern pages.

Workflows 1 x

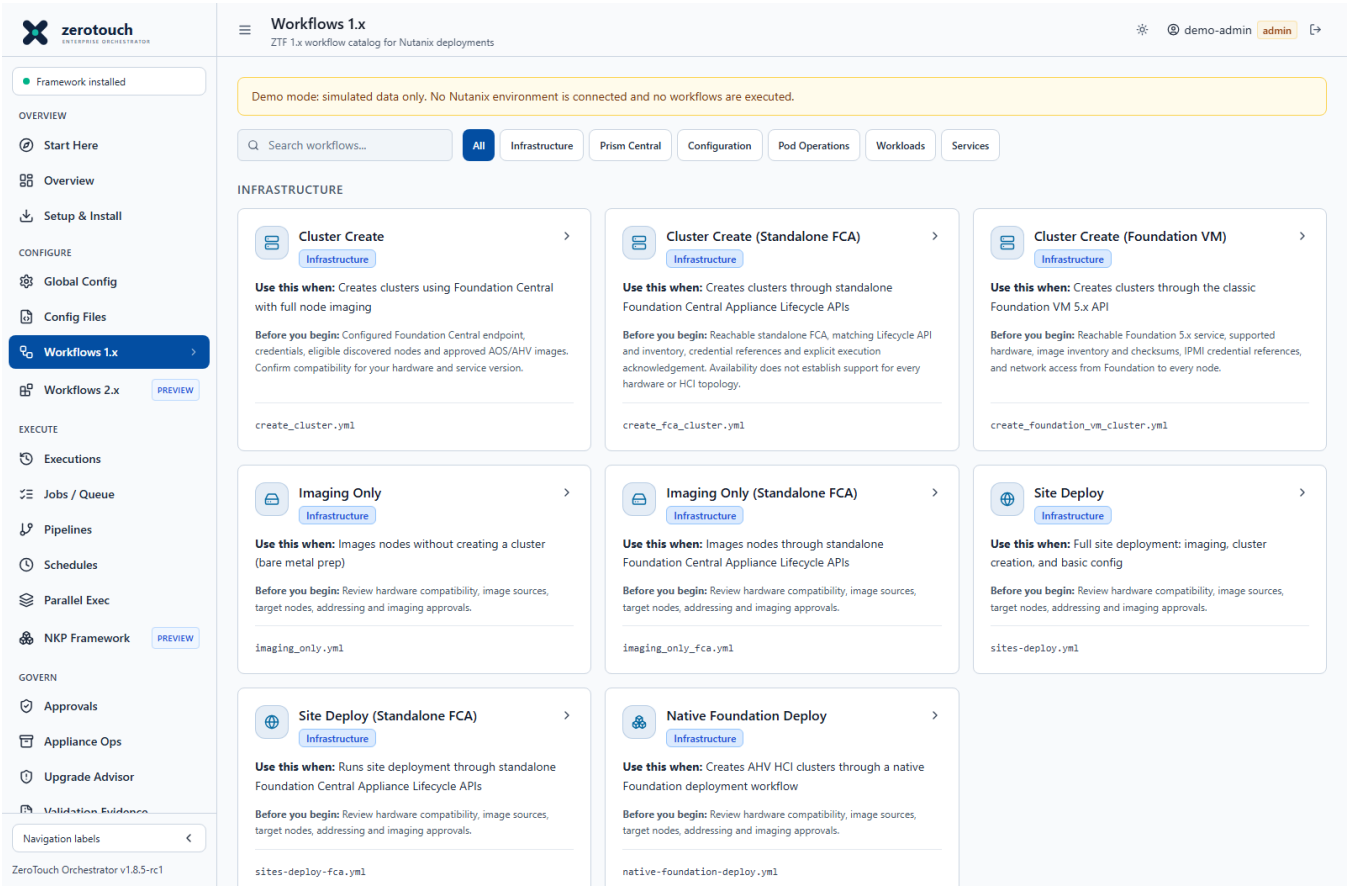


Figure 7. Workflows 1.x. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Operating procedure

1. Search or filter the catalogue and read the workflow description.
2. Open the correct execution backend; Foundation Central, FCA and Foundation VM are distinct.
3. Import or complete the form, review generated YAML and save a named configuration.
4. Run Dry Run where provided, resolve blockers and obtain any required approval.
5. Submit with the required confirmation, then verify Jobs and the target system.

Use **Workflows 1.x** for the existing ZTF 1.x workflow/script catalogue. These workflows generate legacy config YAML, support dry-run preflight where defined, and execute through the guarded python main.py --workflow path.

Workflows 2 x

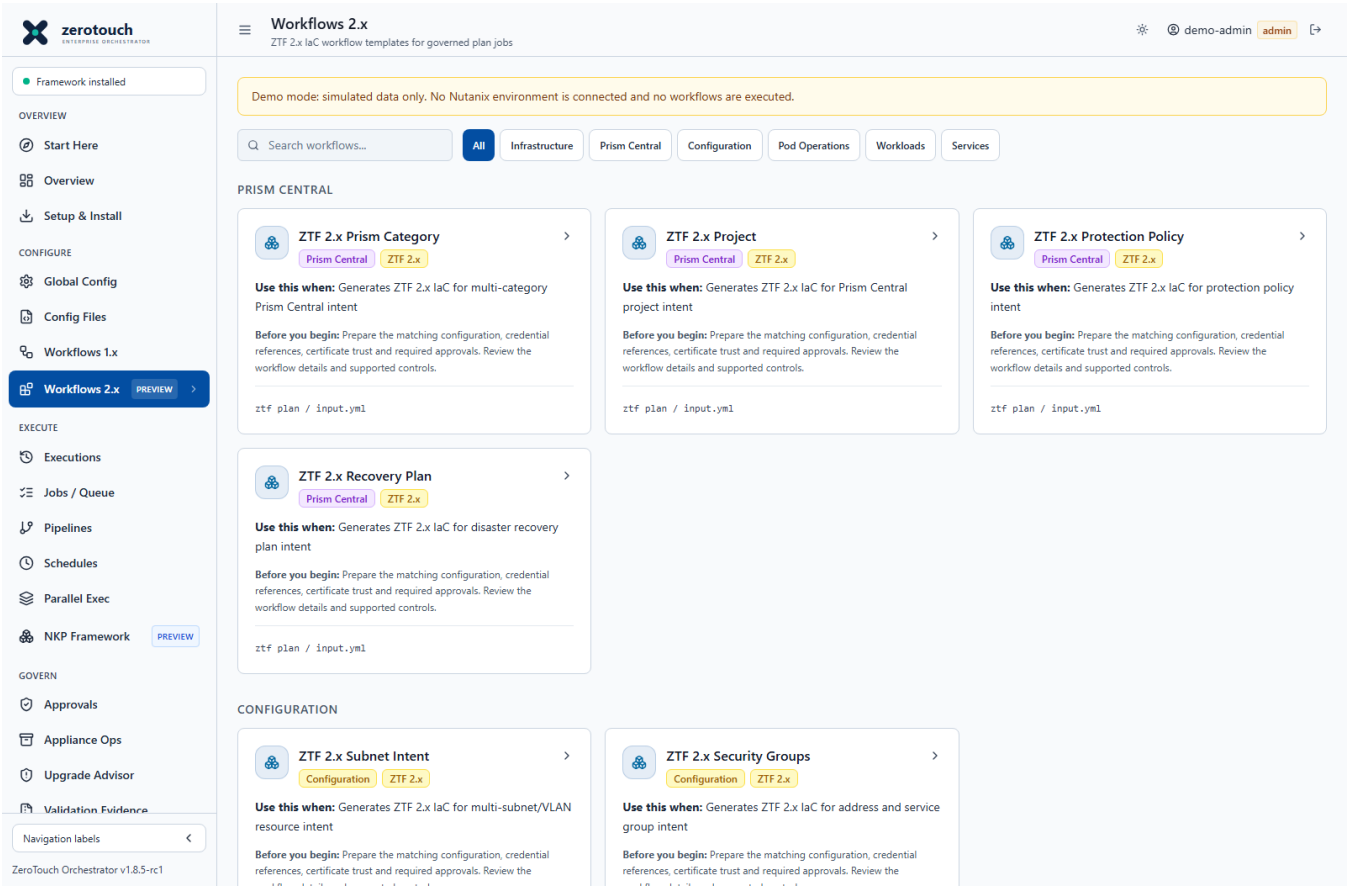


Figure 8. Workflows 2.x. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Operating procedure

1. Select a 2.x template and verify the separate 2.x runtime.
2. Prepare input, global configuration and a unique state filename.
3. Generate and review a plan; resolve schema or provider errors.
4. Apply only through the guarded lane when enabled, approved and validated for the environment.

Use **Workflows 2.x** for ZTF 2.x IaC templates. These workflows generate input.yml, an editable global.yml, and a state file name. Selecting **Run Plan** submits a governed ztf2:plan job and stores plan evidence on the job record. Apply and destroy remain separate approval-bound actions from the ZTF 2.x IaC page.

Available starter templates:

Workflow	Purpose
ZTF 2.x Prism Category	Multi-category Prism Central intent.
ZTF 2.x Project	Project intent with cluster, subnet, and account references.
ZTF 2.x Subnet Intent	Multi-subnet/VLAN intent.
ZTF 2.x Image Registration	Disk, ISO, or cloud-init image registration intent.
ZTF 2.x VM Deployment	VM intent with sizing, image, subnet, categories, and power state.
ZTF 2.x Security Groups	Address and service group intent.

Workflow	Purpose
ZTF 2.x Protection Policy	Protection policy intent with schedule and RPO.
ZTF 2.x Recovery Plan	Recovery plan intent with VM, subnet, and recovery location references.

The templates are operational starters. Validate generated plans against the installed ZTF 2.x resource schema before using approval-bound apply.

Configure Menu

Global Config

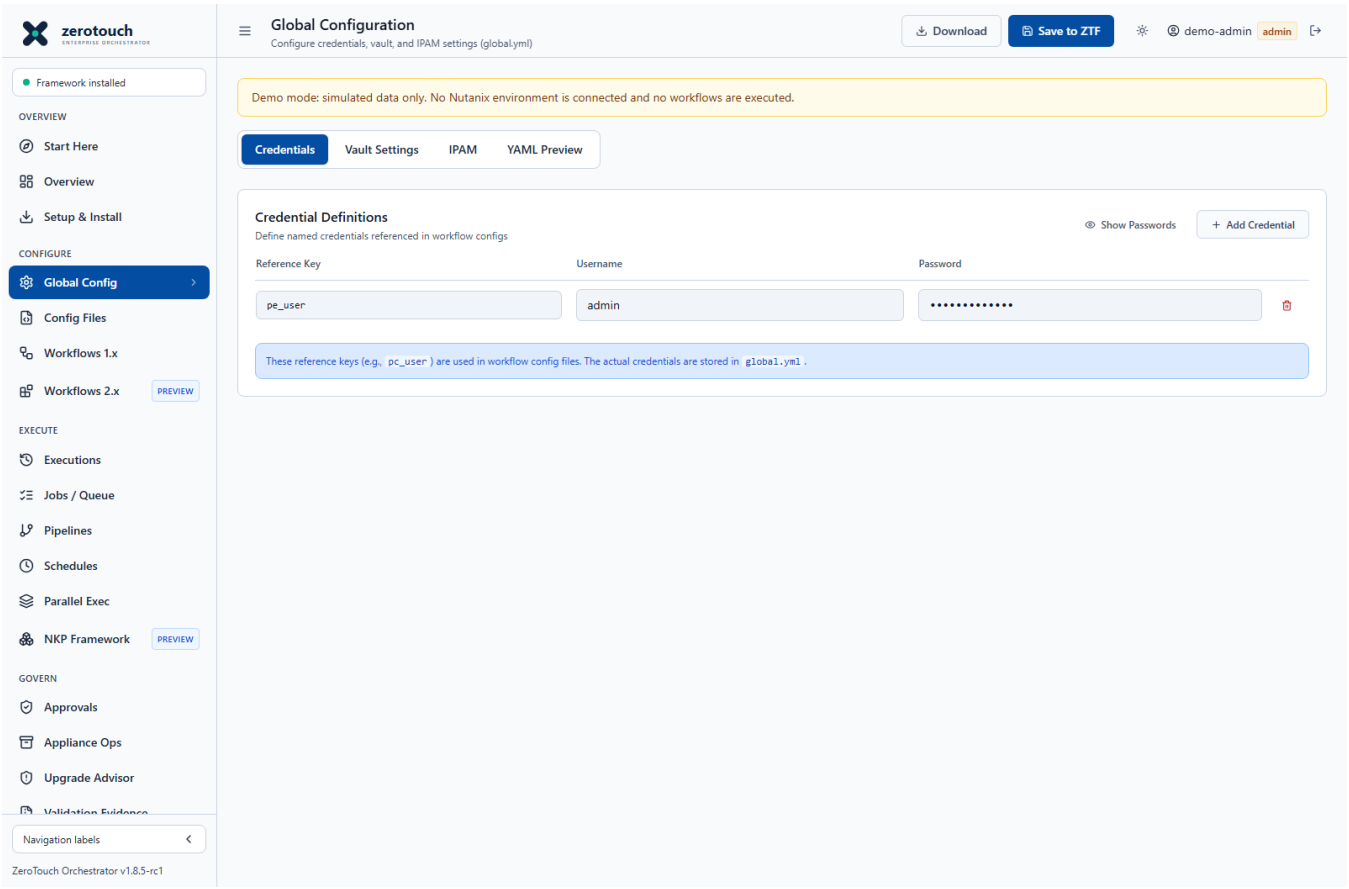


Figure 9. Global Config. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Operating procedure

1. Choose the appropriate vault and IP allocation method.
2. Create named credentials for each target and purpose; avoid shared administrator references.
3. Enter the required vault or IPAM metadata and save.
4. Use the exact reference names in workflow YAML and test only the necessary target permissions.
5. Protect local-vault configuration, backups and exports as sensitive data.

Use Global Config to build and save global.yml, the shared ZeroTouch Framework configuration file.

Tabs and functions:

Tab	Function
Credentials	Defines named credential references such as pc_user, foundation_central, pe_user, ncm_user, and cvm_credential. Workflow YAML uses these references instead of embedding usernames everywhere.
Vault Settings	Selects the credential source. Built-in options include local and environment-based resolution; configured and adapter-target options include HashiCorp Vault, CyberArk, Azure Key Vault, AWS Secrets Manager, Delinea, BeyondTrust, and a custom API profile.
IPAM	Selects the address source of truth. Built-in options include static and CSV/reservation-file allocation; configured and adapter-target options include NetBox, Nautobot, phpIPAM, Infoblox, BlueCat, EfficientIP, Microsoft DHCP/IPAM, and a custom API profile.
YAML Preview	Shows the generated global.yml content before saving or downloading.

Actions:

Action	Function
Show/Hide Passwords	Temporarily reveals or masks credential passwords in the browser.
Add Credential	Adds another named credential reference.
Delete credential row	Removes an unused credential reference from the generated config.
Download	Downloads the generated global.yml.
Save to ZTF	Saves the generated global configuration through the backend.

Use this page before running workflows that reference credentials, vaults, or IPAM settings.

Provider selections marked as configuration targets write provider metadata into global.yml; they do not imply that live credential retrieval or mutating IP allocation is enabled. Live provider operations depend on the corresponding adapter being configured, permitted, and validated for the environment.

Config Files

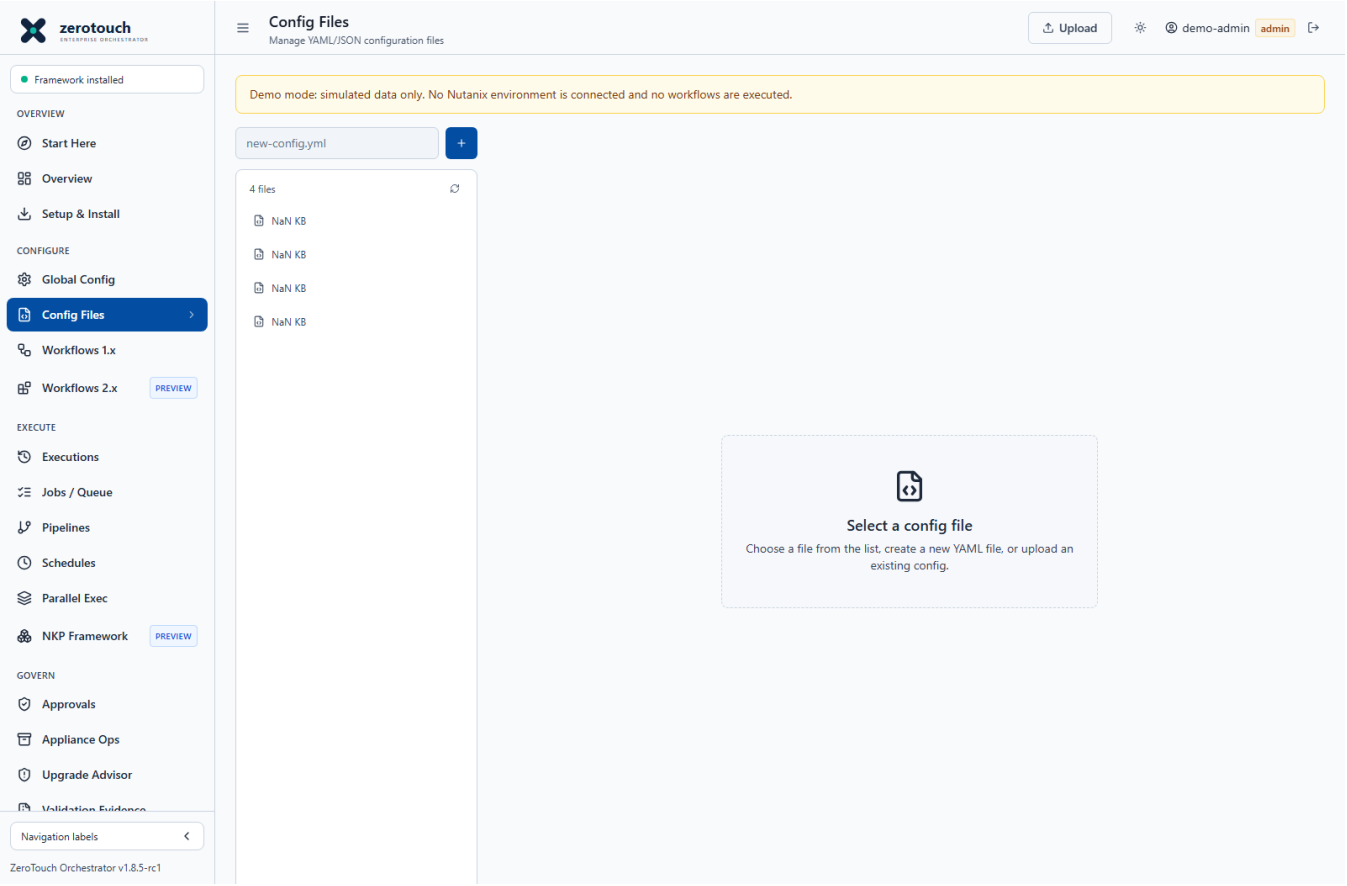


Figure 10. Config Files. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Operating procedure

1. Open a named file or upload approved YAML.
2. Review its target addresses, reference names and workflow schema before saving.
3. Use download or retained versions for review and recovery; restrict access to secret-bearing exports.
4. Select this exact saved file when building a pipeline or submitting a job.

Use Config Files to manage saved YAML and JSON configuration files used by workflows, scripts, schedules, pipelines, NKP phases, drift checks, and evidence records.

Key functions:

Function	Purpose
File list	Shows available saved config files.
New file	Creates an empty YAML or JSON config file.
Upload	Imports an existing config from the workstation.
Editor	Lets operators edit selected config content directly.
Save	Writes the current content. Existing files are backed up before overwriting.
Download	Downloads the selected config.

Function	Purpose
Delete	Removes a selected config after confirmation.
History	Shows recent automatic backups, including timestamp and size.
Restore	Restores a prior version. The current file is backed up first.
Open YAML Studio	Routes to YAML Studio for guided generation.

Use Config Files as the shared config library for repeatable, reviewable work.

YAML Studio

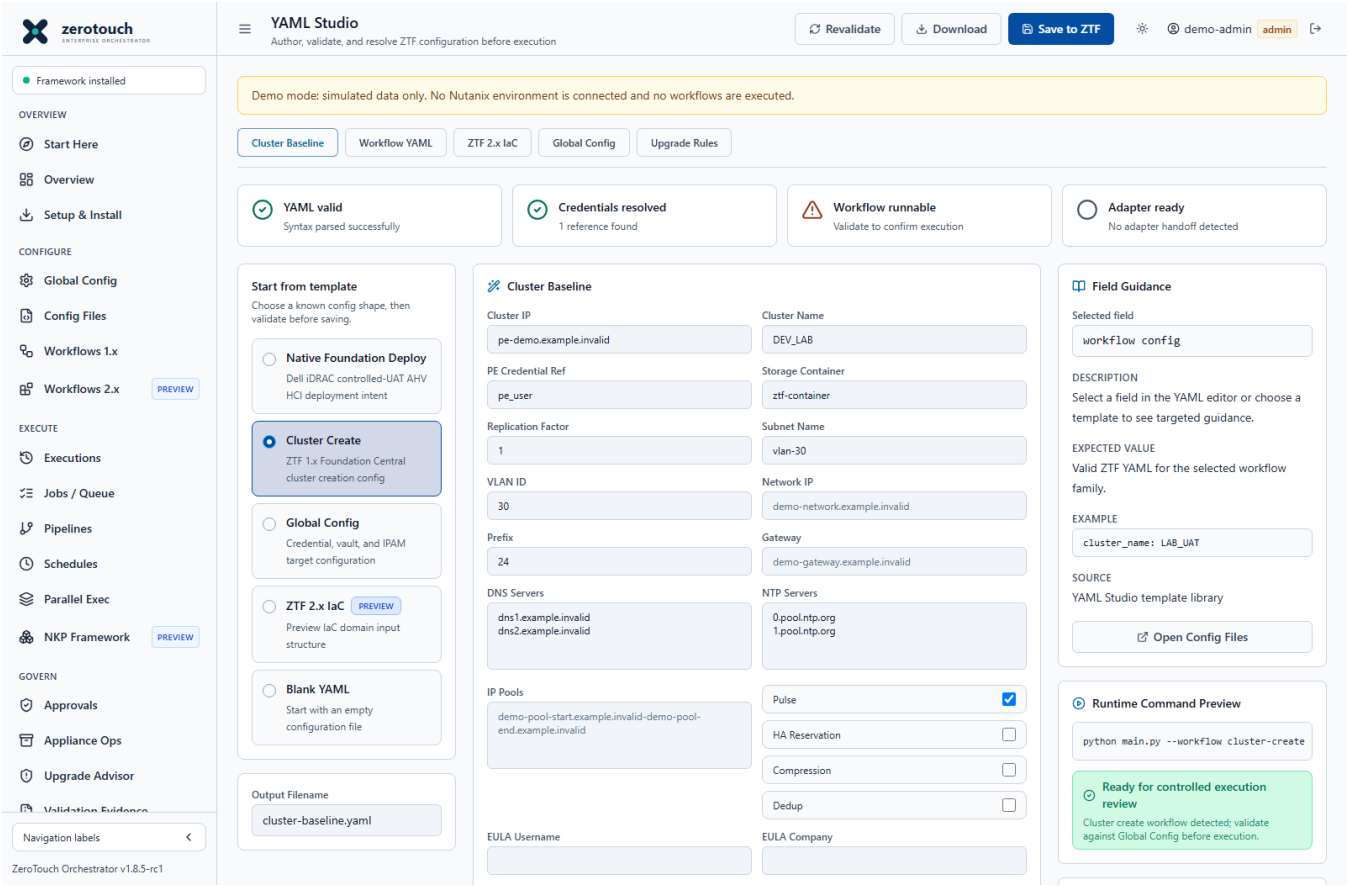


Figure 11. YAML Studio. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Operating procedure

1. Select the intended configuration family and generator rather than an unrelated blank template.
2. Complete the wizard and generate YAML.
3. Review syntax, credentials, workflow runnability and adapter readiness independently.
4. Resolve blocking findings and save a descriptive filename.
5. Open the corresponding workflow to validate execution; a successful save is not a successful deployment.

Use YAML Studio to create, edit, validate, and save ZTF-compatible Nutanix YAML without executing anything. The page is organized as a guided authoring workspace: choose a template, edit YAML, review field guidance, confirm readiness, and inspect the runtime command preview before saving.

Modes:

Mode	Function
Native Foundation Deploy	Starts a Dell iDRAC controlled-UAT deployment intent template and highlights credential refs, adapter readiness, and runtime handoff.
Cluster Create	Starts a ZTF 1.x Foundation Central cluster creation config.
Global Config	Produces starter credential, vault, and IPAM target configuration content.
ZTF 2.x IaC	Produces preview input.yml content for the domains model.
Blank YAML	Starts from an empty file while still using validation, field guidance, and command preview.

Actions:

Action	Function
Generate YAML	Builds YAML from the selected workflow/script wizard fields.
Revalidate	Sends YAML to the backend for parser and shape validation.
Save to ZTF	Saves generated YAML into Config Files using existing backup behavior.
Download	Downloads a bundle containing YAML and validation metadata.

Readiness indicators:

Indicator	Meaning
YAML valid	The current editor content parses as YAML.
Credentials resolved	Referenced credential keys were found in Global Config when available.
Workflow runnable	The YAML maps to a known workflow or script command after validation.
Adapter ready	Native Foundation adapter handoff is detected when relevant.

YAML Studio is intentionally non-mutating. It does not run ZTF, NKP, Prism, or Foundation Central operations. Execution remains behind Workflows, Scripts, NKP Framework, approvals, confirmations, and durable jobs.

Workflow Catalogue

Operating procedure

1. Select a catalogue entry based on the target API and installed framework.
2. Review its form and operation-specific support boundary before entering credentials.
3. Use the workflow reference appendix for identifiers; do not substitute a similarly named backend.

Use Workflows 1.x and Workflows 2.x to browse and launch prebuilt ZTF workflow families. Workflows 1.x runs guarded legacy workflow jobs. Workflows 2.x generates governed ZTF 2.x IaC plan jobs that can later be used as the source for approval-bound apply or destroy requests.

The catalogues are grouped by category and include Infrastructure, Configuration, Prism Central, Pod Operations, Workloads, Services, and System workflows where templates exist. Selecting a workflow opens its detail page.

Workflow detail functions:

Function	Purpose
Configure tab	Presents guided fields for the selected workflow. Specialized workflows use specialized forms. Generic workflows use a shared config form.
YAML Preview tab	Shows the active generated or imported YAML.
Import Config	Imports an existing YAML or JSON file for the selected workflow, with basic shape checks.
Download Config	Downloads the active YAML.
Dry Run	Validates the config and performs non-destructive readiness or connectivity checks where supported. Approval is not required for dry run.
Approved Request selector	Let's operators bind a matching approved request when the workflow is approval mandatory.
Run Workflow	Submits a Workflows 1.x item through the governed legacy execution path. Standalone FCA workflows require an exact destructive acknowledgement phrase before submission. Classic Foundation VM cluster-create requires an administrator, an exact DEPLOY FOUNDATION VM confirmation, normal approval policy, and a valid Foundation VM intent.
Run Plan	Submits a Workflows 2.x item through the ZTF 2.x IaC plan path. Apply and destroy remain approval-bound from the ZTF 2.x IaC page.

Native Foundation review controls are grouped below. These reviews expose plans and blockers; they do not establish permission or completed target execution. Read the returned readOnly and execution-enabled flags and use the dedicated task chapter for operational boundaries.

Review group	Typical controls	What to verify
Intent and discovery	Discovery preview, reconcile, network manifest and secret refs	Node identity, complete network data and named credentials; supplied facts are not live discovery.
Images and topology	Image sources, imaging plan, formation plan and topology support	Supported provider and topology, approved image references and required evidence.
Planning and policy	Generate Plan, execution graph, deployment policy and wave reviews	Deterministic hashes, dependencies, maintenance windows, scope and failure policy.
Admission and authorization	Execution readiness, approval binding, UAT entry and persistence reviews	Explicit blockers and exact reviewed intent; a review artifact is not a real permit or admission.
Adapter and runtime	Provider adapters, allow-list, package and runtime reviews	Configured adapter, trusted execution environment and controlled UAT evidence before mutation.

Review group	Typical controls	What to verify
Recovery and evidence	Job state, restart/resume, backup/restore and retained export reviews	Real recovery ownership and independently retained evidence; read-only plans do not perform recovery.

Workflows 1.x catalogue:

Workflow	Category	Function
Cluster Create	Infrastructure	Creates clusters using Foundation Central with node imaging and cluster formation through the legacy ZTF workflow lane.
Cluster Create (Standalone FCA)	Infrastructure	Builds and submits standalone Foundation Central Appliance Lifecycle cluster-create requests after read-only inventory validation and explicit acknowledgement.
Cluster Create (Foundation VM)	Infrastructure	Builds the classic Foundation VM native payload, inventories/stages images, validates node and network input, tests IPMI credentials read-only, and can submit controlled ipmi_config and image_nodes operations when explicitly enabled.
Imaging Only	Infrastructure	Images nodes without forming a cluster. Useful for bare-metal preparation or re-imaging.
Imaging Only (Standalone FCA)	Infrastructure	Images nodes through standalone Foundation Central Appliance Lifecycle APIs after guarded validation.
Pod Imaging	Pod Operations	Runs a pod-oriented imaging and cluster creation flow.
Pod Imaging (Standalone FCA)	Pod Operations	Runs pod imaging through standalone Foundation Central Appliance Lifecycle APIs.
Site Deploy	Infrastructure	Handles multi-site deployment input for imaging, cluster creation, and basic configuration.
Site Deploy (Standalone FCA)	Infrastructure	Builds standalone FCA site deployment intents and submits guarded Lifecycle requests.
Native Foundation Deploy	Infrastructure	Plans Orchestrator-owned multi-site, heterogeneous cluster deployment intents for HCI, compute-only, storage-only, and mixed topology design. Execution is disabled until native adapters are validated.
Configure Cluster	Configuration	Selected PE configuration with optional initial admin password change, verified replacement authentication, explicit password skips, EULA/Pulse and requested container readback.
Post-Foundation Baseline	Configuration	Plans and applies verified post-foundation Prism Element baseline operations where safe mappings exist.
PE Monitoring Baseline	Configuration	Plans monitoring, alerting, and validation evidence checks. Some controls are evidence-only or blocked until mappings are verified.
AHV Security Hardening	Configuration	Builds a checklist-driven hardening plan for Prism Element, CVM, and AHV controls. Only verified operations execute.
PE Network Baseline	Configuration	Plans and applies guarded VM network creation where mapped scripts are verified.

Workflow	Category	Function
PE Certificate Baseline	Configuration	Plans certificate validation, CSR, and replacement work. Validation can be evidence-only where mutation is not mapped.
Hardware OOB Baseline	Configuration	Plans hardware out-of-band inventory and vendor API baseline tasks. Mutation remains blocked until vendor-supported mappings are confirmed.
Deploy Prism Central	Prism Central	Deploys Prism Central VM instances on target clusters.
Configure Prism Central	Prism Central	Configures Prism Central services such as identity, SAML, Objects, NKE, Flow, DR, and policies.
Pod Config	Pod Operations	Configures pod-level or edge-site infrastructure at scale.
Deploy Management PC	Pod Operations	Deploys Prism Central and NCM management-plane components.
Configure Management PC	Pod Operations	Initializes and configures management Prism Central and NCM.
Calm VM Workloads	Workloads	Deploys VM workloads through Calm DSL blueprint inputs.
Edge AI Workload	Workloads	Deploys Edge AI application workloads using workload-specific Calm inputs.
NDB Deploy	Services	Deploys and configures Nutanix Database Service profiles and registrations.
LCM Update	System	Runs lifecycle management update workflows with reviewed YAML and approval-gated execution.

Post-foundation baseline workflows classify controls as apply, evidence, manual, or blocked. Only verified apply mappings execute. Evidence-only, manual, and blocked items are retained for review without silently mutating the environment.

Scripts 1 x

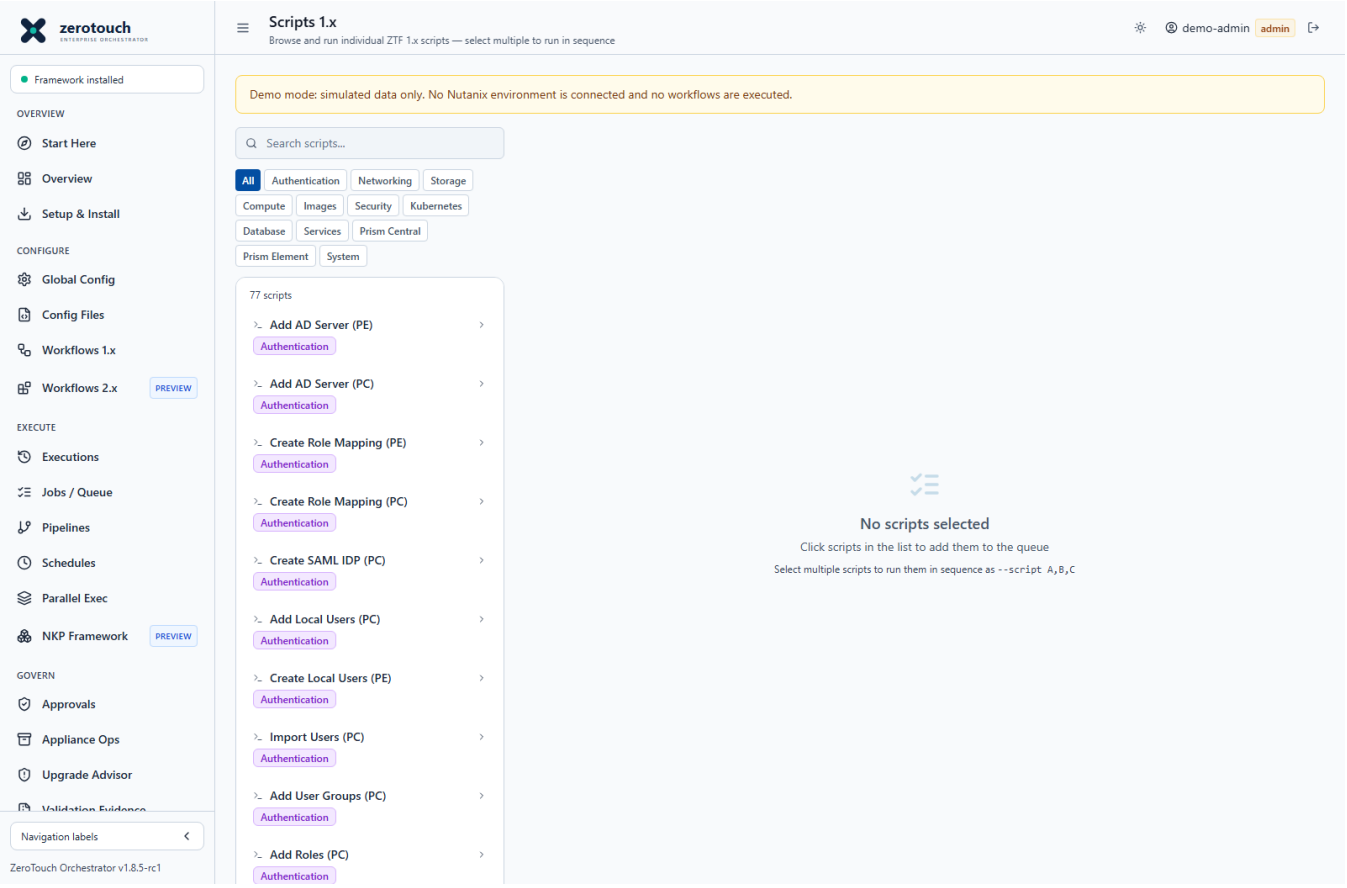


Figure 12. Scripts 1.x. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Operating procedure

1. Select a category and script, then open its parameter wizard.
2. Supply target and credential references using the installed script schema.
3. Generate and review YAML; save a file if a pipeline will reuse it.
4. Run with any explicit confirmation and inspect the deepest API or authentication error.
5. Verify the resulting target state independently.

Use Scripts 1.x to browse and run allowlisted atomic ZeroTouch Framework 1.x scripts.

Key functions:

Function	Purpose
Search	Finds scripts by name or description.
Category filter	Limits the list to categories such as Authentication, Networking, Storage, Compute, Images, Security, Kubernetes, Database, Services, Prism Central, Prism Element, or System.
Select script	Adds a script to the queue. Selecting it again removes it.
Script Queue	Shows the ordered script list that will be passed to ZTF.
Move up/down	Reorders queued scripts before launch.

Function	Purpose
Clear	Removes all queued scripts.
Shared Configuration	Supplies one YAML or JSON config file to all queued scripts.
Script Config Wizard	Generates shared config content for selected scripts where schema support exists.
Run	Submits the selected script or ordered script sequence as one execution. Destructive scripts require an exact confirmation phrase.
Terminal output	Shows streaming execution output during the browser-bound run path.

Use Scripts 1.x when a full workflow is too broad and an operator needs a specific legacy ZTF script or a carefully ordered script sequence.

Scripts 2 x

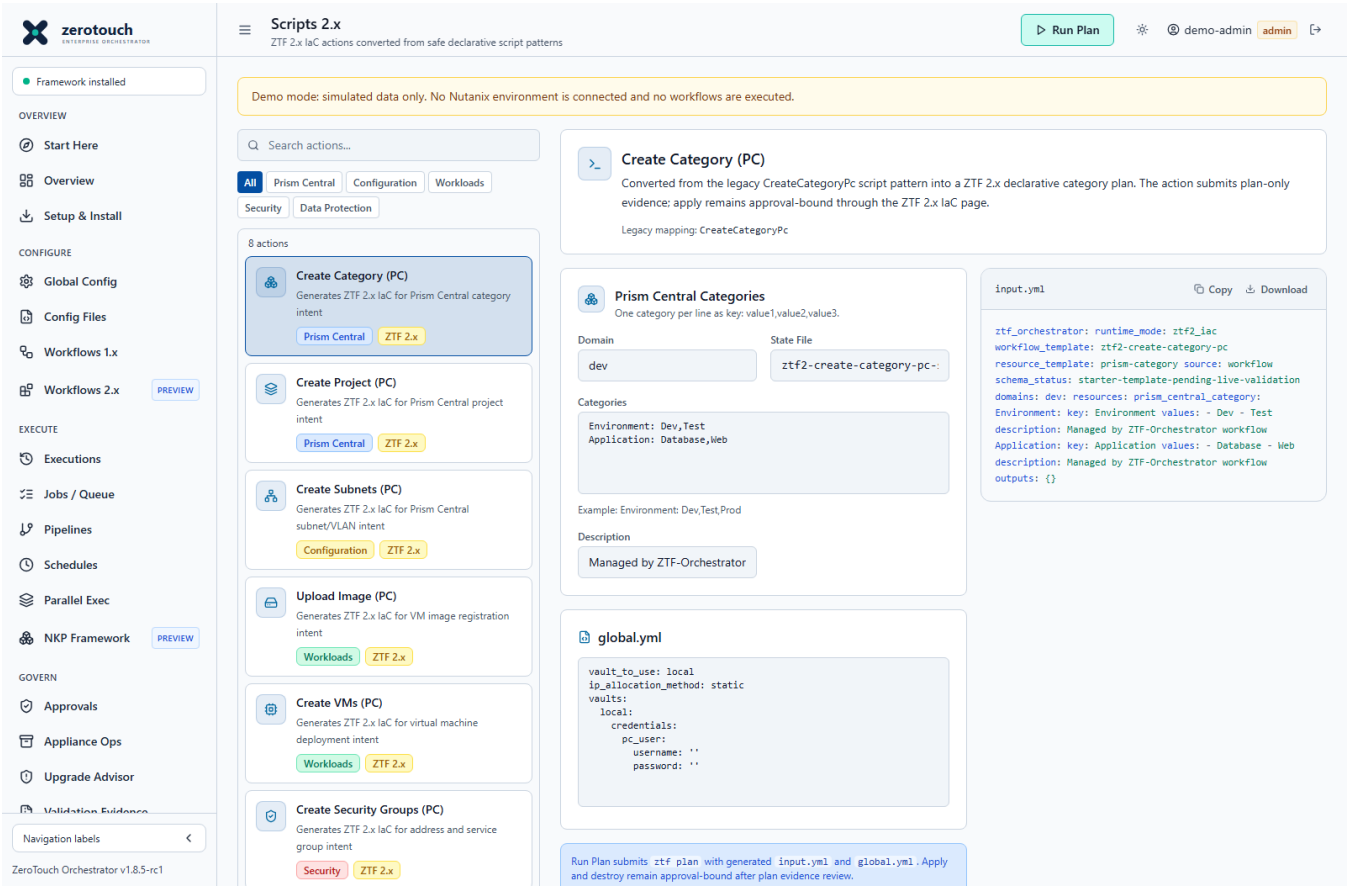


Figure 13. Scripts 2.x. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Operating procedure

1. Choose the appropriate 2.x resource intent.
2. Generate input and review the plan with the separate runtime.
3. Keep state filenames and approved input aligned.
4. Apply only if the environment and guarded lane permit it; preview availability is not support certification.

Use Scripts 2.x for converted ZTF 2.x IaC actions. These actions are not legacy --script commands. They generate input.yml, editable global.yml, and a state file name, then submit a governed ztf2:plan job. Apply and destroy remain approval-bound through the ZTF 2.x IaC page.

Available converted actions:

Action	Legacy mapping
Create Category (PC)	CreateCategoryPc
Create Project (PC)	New ZTF 2.x declarative action
Create Subnets (PC)	CreateSubnetsPc
Upload Image (PC)	PcImageUpload
Create VMs (PC)	CreateVmsPc
Create Security Groups (PC)	CreateAddressGroups, CreateServiceGroups
Create Protection Policy (PC)	CreateProtectionPolicy
Create Recovery Plan (PC)	CreateRecoveryPlan

PE-only, CVM/Foundation, delete, and power actions stay in Scripts 1.x until a reviewed ZTF 2.x resource contract is available.

Execute Menu

Execution History

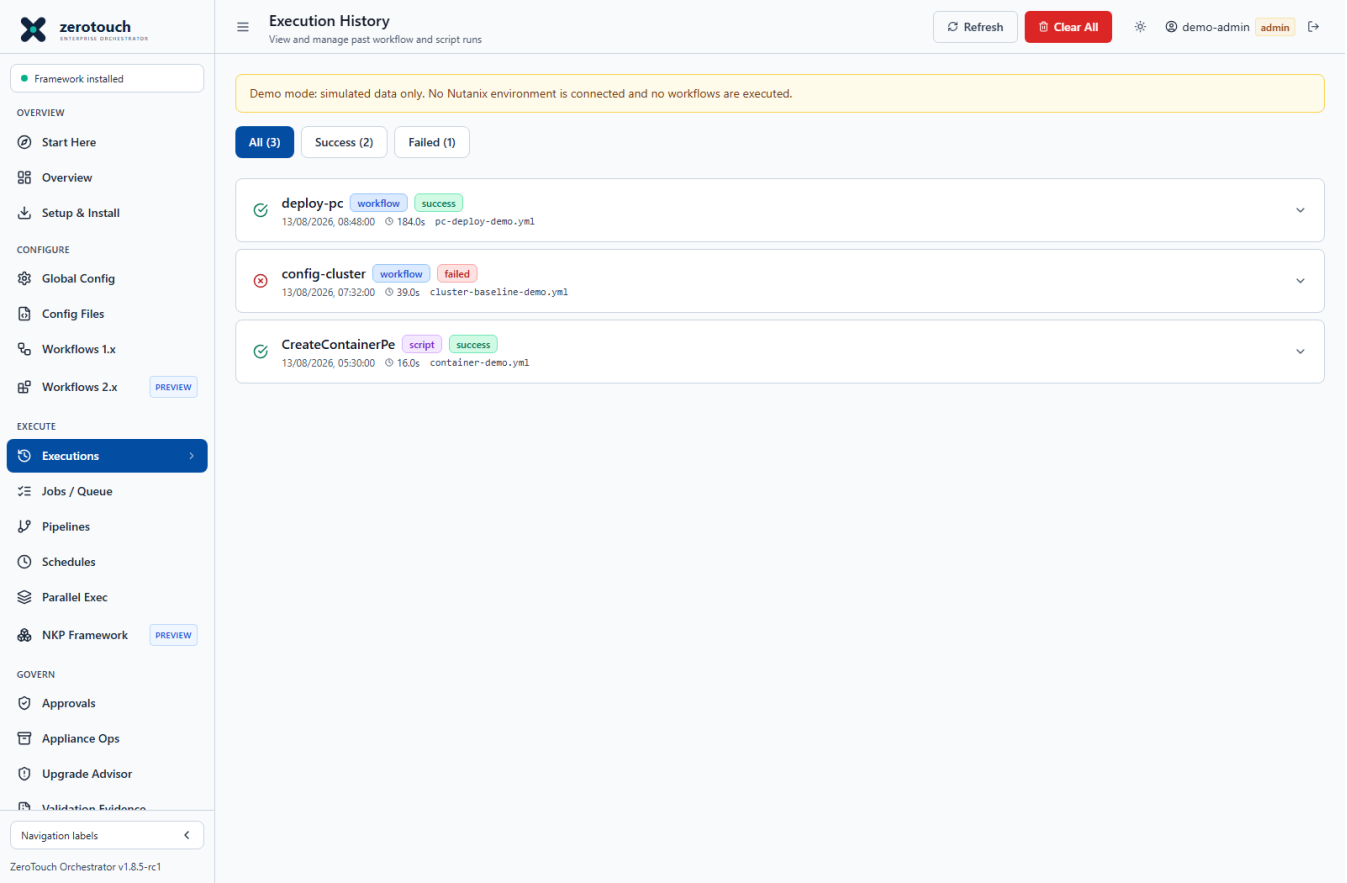


Figure 14. Execution History. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Operating procedure

1. Filter by result, workflow or date to locate the run.
2. Expand the execution and review return code, stdout, stderr and diagnostic context.
3. Correlate the configuration and job identifiers with target-side tasks.
4. Export relevant evidence before retention or cleanup removes records.

Use Execution History to review previous workflow and script submissions.

Key functions:

Function	Purpose
Refresh	Reloads execution records.
Filters	Switches between all, successful, and failed executions.
Expand row	Shows command, config path, return code, failure diagnostics, stderr, and stored config details.
Re-run	Starts a new run using the stored config content from a prior execution.

Function	Purpose
Clear All	Clears execution history after confirmation. Use carefully, because this is an operational record.

Execution History is best for completed-run review, troubleshooting, and repeat execution from known-good or known-failed configs.

Jobs Queue

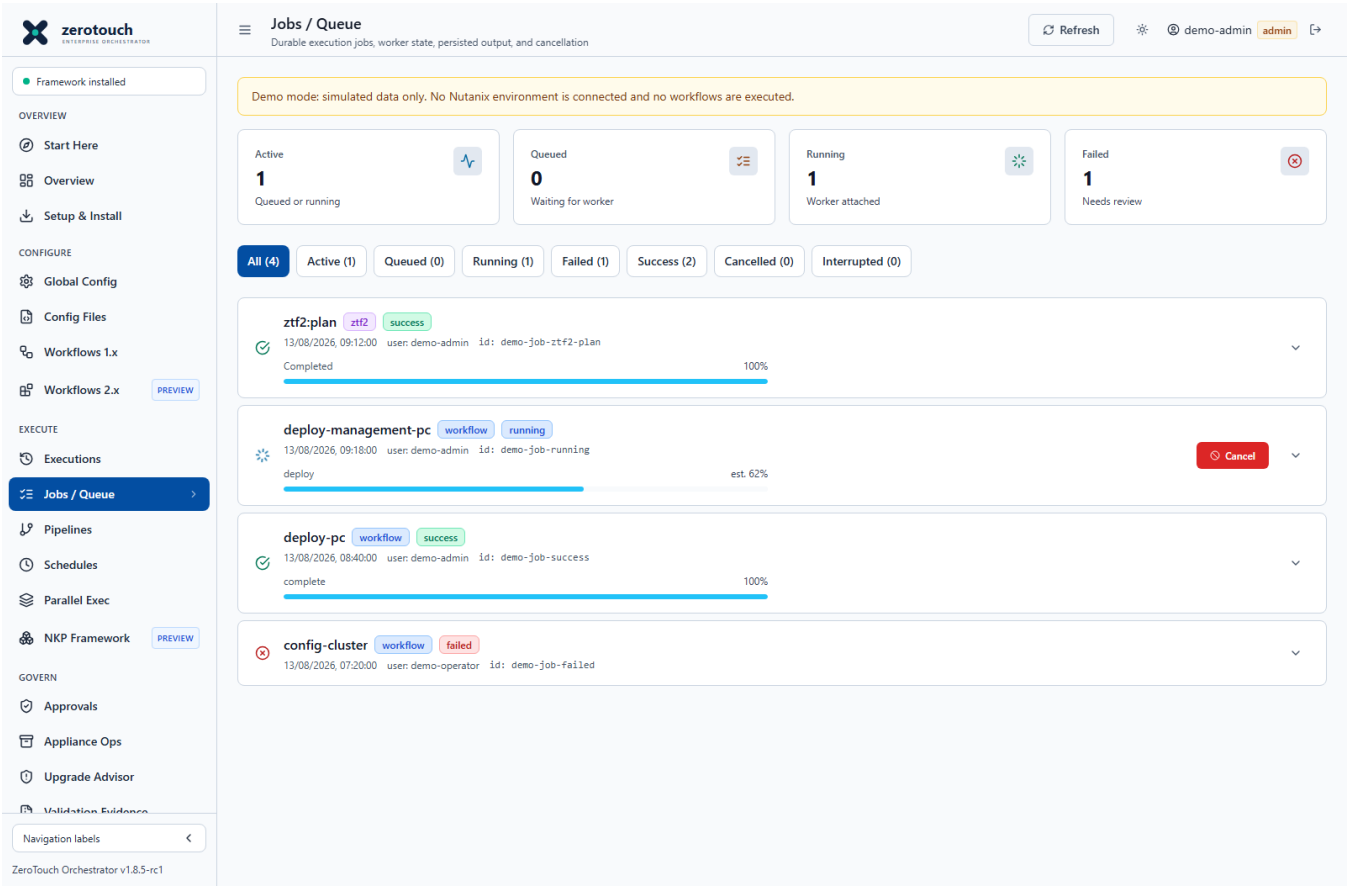


Figure 15. Jobs / Queue. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Operating procedure

1. Filter queued, running, failed or interrupted jobs.
2. Expand the job and inspect progress timestamps, logs, diagnostics and linked approval.
3. Cancel only after reviewing the target operation and its cancellation semantics.
4. For interrupted deployment jobs inspect Foundation or Prism before using a guarded restart.
5. Delete terminal queue records only under retention policy, after preserving evidence.

Use Jobs / Queue to monitor durable background jobs.

Key functions:

Function	Purpose
Metrics	Shows active, queued, running, failed, and interrupted job counts.

Function	Purpose
Filters	Switches between all, active, queued, running, failed, success, cancelled, and interrupted jobs.
Auto-refresh	Refreshes job state periodically.
Expand row	Shows worker timestamps, return code, log event count, progress, diagnostics, trace metadata, schema status, linked profile/config/approval IDs, and detected Nutanix task IDs.
Persisted Job Log	Displays saved stdout, stderr, start, error, and progress events.
Cancel	Lets admins and operators cancel queued or running jobs.
Delete Queue Record	Lets admins remove terminal queue records after review. Execution history and audit logs are retained.

Use Jobs / Queue while work is running or when investigating why a job is queued, stuck, cancelled, interrupted, or failed.

Pipelines

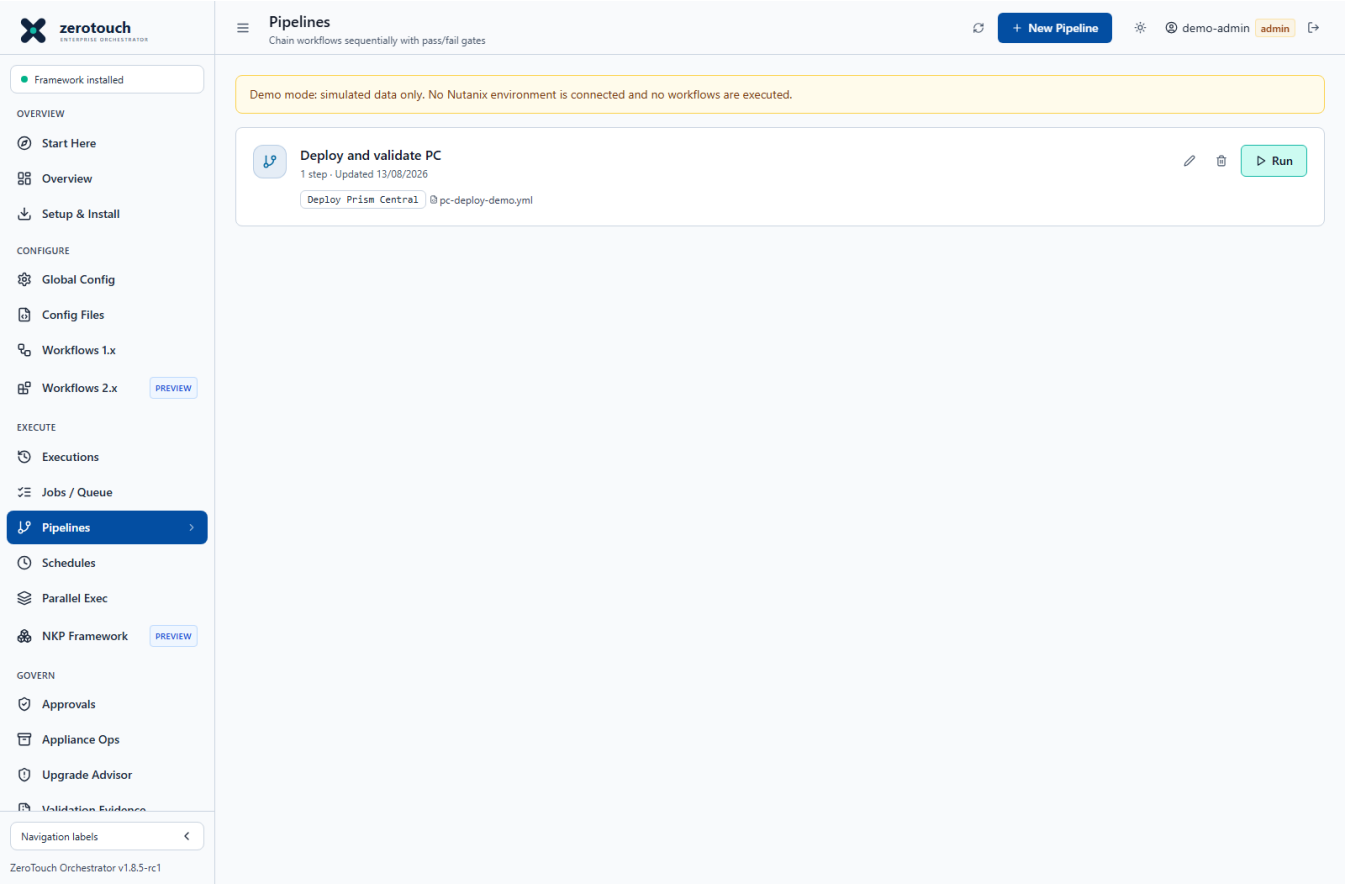


Figure 16. Pipelines. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Operating procedure

1. Create a named pipeline and add only supported workflow or allowlisted script steps.
2. Bind a separate reviewed configuration file to each step and put prerequisites first.

3. Verify credential transitions and approval restrictions before saving.
4. Run and review each step; a failure halts subsequent steps and is not a rollback.
5. Verify target state before retrying password changes or other non-idempotent actions.

Use Pipelines to chain workflows into a named, sequential deployment process.

Key functions:

Function	Purpose
New Pipeline	Opens the pipeline builder.
Pipeline Name	Names the reusable sequence.
Add Step	Adds a workflow and optional config-file pair.
Workflow selector	Chooses the workflow for a step.
Config selector	Binds a saved config file to the step.
Move up/down	Reorders steps.
Edit	Modifies an existing pipeline definition.
Delete	Removes a pipeline definition.
Run	Starts the pipeline execution modal.

Pipeline steps run in order. A step starts only after the prior step succeeds. Failures halt the pipeline and remaining steps are skipped.

Schedules

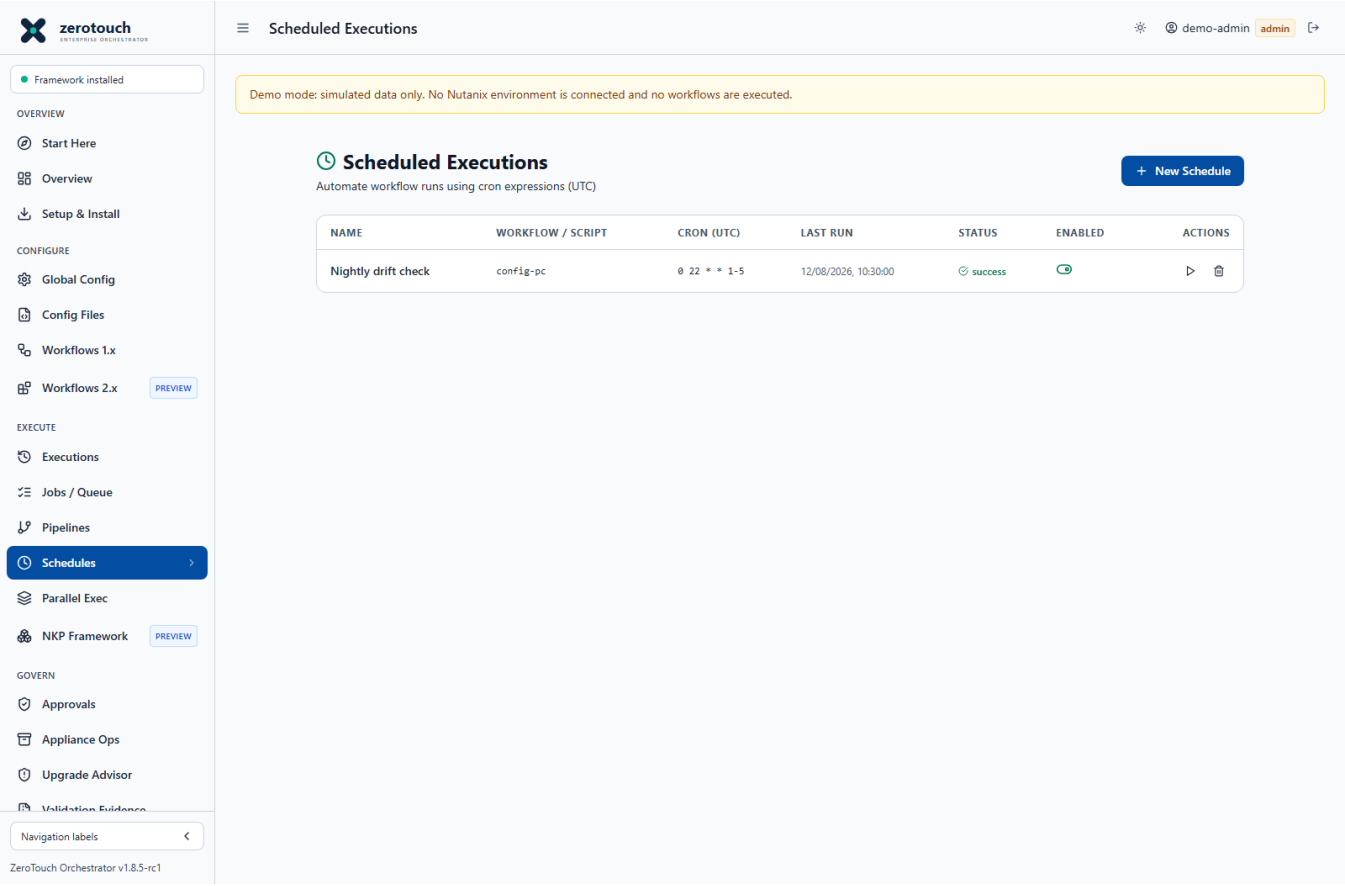


Figure 17. Schedules. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Operating procedure

1. Choose a repeatable workflow or script with a supported automation path.
2. Enter a five-field UTC cron expression and reviewed YAML.
3. Save enabled only after a successful manual test and governance review.
4. Use Run now, inspect Last Run and the resulting job, then verify the next scheduled execution.
5. Pause during maintenance; do not assume missed downtime runs are replayed.

Use Schedules to automate recurring workflow or script runs.

Key functions:

Function	Purpose
New Schedule	Creates a named recurring run.
Workflow / Script	Selects the workflow or script to run.
Cron Expression	Defines the schedule using a 5-field cron expression in UTC.
Presets	Inserts common cron expressions such as hourly or daily at 02:00.
Config	Stores YAML content to pass to the scheduled run.
Enabled	Turns a schedule on or off without deleting it.

Function	Purpose
Run now	Triggers the schedule immediately.
Delete	Removes the schedule.
Last Run / Status	Shows recent execution timestamp and result.

Scheduled runs are useful for repeatable health checks, periodic validation, or controlled recurring workflows. Workflows that require mandatory approvals may be rejected from automation paths until a governance binding exists for that surface.

Parallel Exec

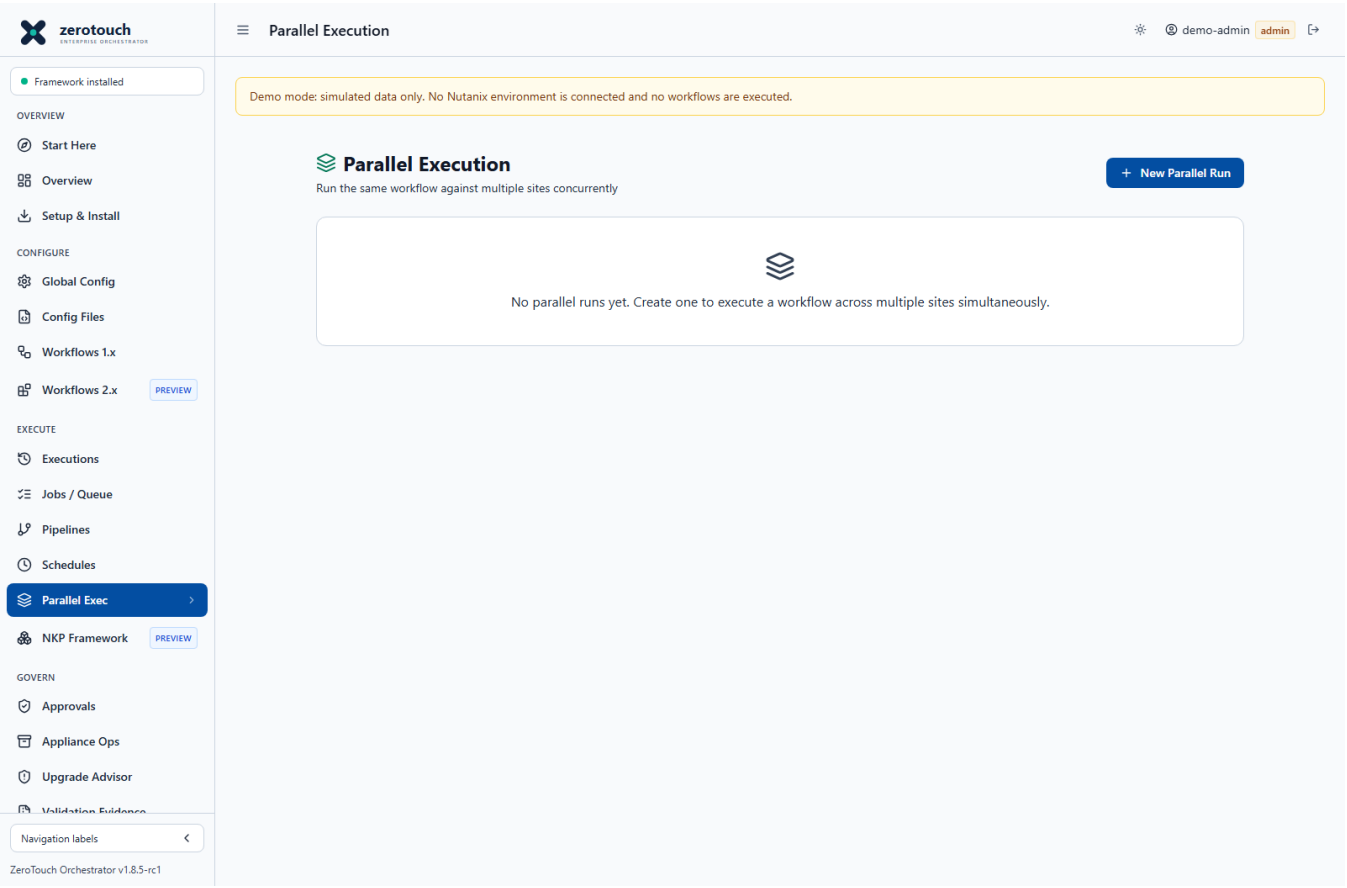


Figure 18. Parallel Exec. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Operating procedure

1. Confirm the selected workflow uses the supported legacy parallel execution path.
2. Add a distinct label and complete reviewed YAML for every site.
3. Check credential, image, IP and resource isolation across sites.
4. Run and inspect each site result separately; overall partial success needs investigation.
5. Do not use this screen as proof of support for concurrent Foundation VM sessions.

Use Parallel Exec to run the same workflow against multiple sites at the same time.

Key functions:

Function	Purpose
Workflow selector	Chooses the workflow each site will run.
Site entries	Captures labels and per-site YAML config content.
Add/remove site	Adjusts the set of target sites.
Run Parallel	Starts concurrent execution across the configured sites.
Per-site output	Shows terminal output for each target site independently.
Overall status	Summarizes the run as success, partial, or failed.

Parallel execution is intended for repeatable multi-site work where each target has its own reviewed config. It does not prove production completion by itself; site-level validation and evidence remain required.

NKP Framework

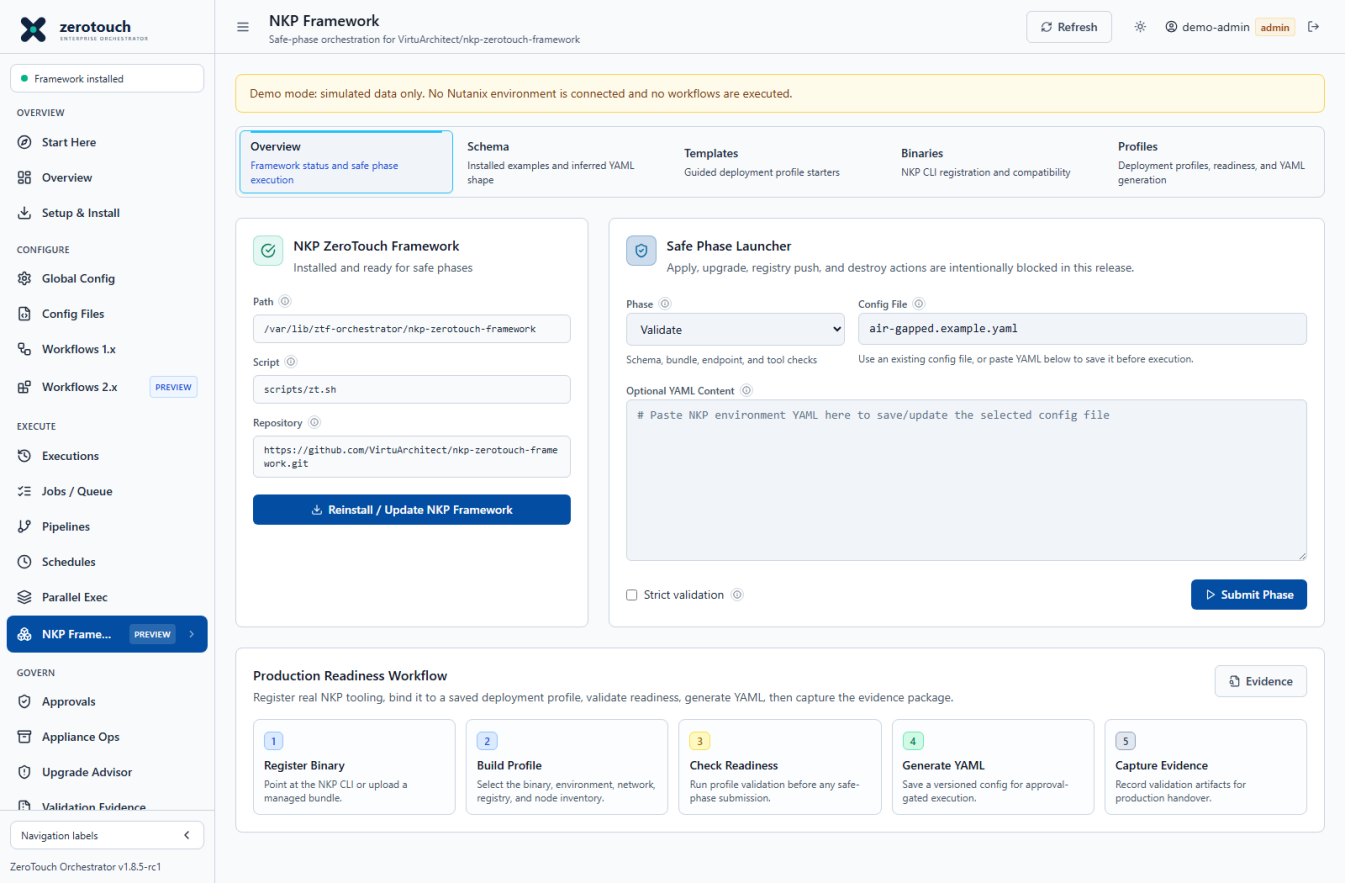


Figure 19. NKP Framework. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Operating procedure

1. Confirm the optional NKP framework and compatible binary inventory.
2. Choose a template and complete deployment profile, registry and network inputs.
3. Generate configuration and review Schema and Readiness findings.
4. Submit only the supported safe phase and permitted environment.

5. Retain readiness/evidence outputs; a generated file is not an NKP cluster deployment.

Use NKP Framework for optional NKP ZeroTouch Framework integration, deployment profile management, readiness checks, binary registration, generated YAML, and guarded safe-phase submission.

Tabs and functions:

Tab	Function
Overview	Shows NKP framework status, configured path, safe phases, config inventory, and phase launcher.
Schema	Discovers installed NKP example YAML and infers expected schema shape from the configured framework path.
Templates	Provides guided deployment starters such as Management Cluster, Workload Cluster, and Air-Gapped / Local Registry.
Binaries	Registers or uploads NKP binaries and checks CLI compatibility.
Profiles	Creates, edits, validates, versions, restores, and renders NKP deployment profiles into YAML.

Safe phases:

Phase	Function
Validate	Performs schema, bundle, endpoint, and tooling checks.
Prepare	Stages NKP tools and workspace metadata. Approval may be required.
Generate	Creates cluster values, environment content, or helper files. Approval may be required.
Registry Plan	Generates private registry planning output. Approval may be required.
Deploy Plan	Generates deployment planning output. Approval may be required.
Verify	Collects local state and kubeconfig-based checks when available.
Runs	Summarizes NKP ZeroTouch run artifacts.

Important boundaries:

Boundary	Meaning
Safe-phase model	NKP actions are allowlisted and constrained. Apply, registry push, upgrade, and destroy actions are blocked server-side unless explicitly implemented later.
Approval-required phases	Prepare, Generate, Registry Plan, and Deploy Plan require governance approval where policy applies.
Profile revisions	Saved profiles are versioned. Restoring an older profile creates a new revision instead of erasing history.
Readiness scoring	Profiles can be ready, needs attention, or blocked based on required fields, endpoint shape, IP checks, binary paths, and YAML validity.

Boundary	Meaning
Generated YAML	The YAML is transparent and editable in Config Files so teams can align it to the exact NKP schema they adopt.

Use NKP Framework when preparing Kubernetes platform deployment evidence or safe-phase automation. Use Jobs / Queue to monitor submitted NKP jobs.

Govern Menu

Approvals

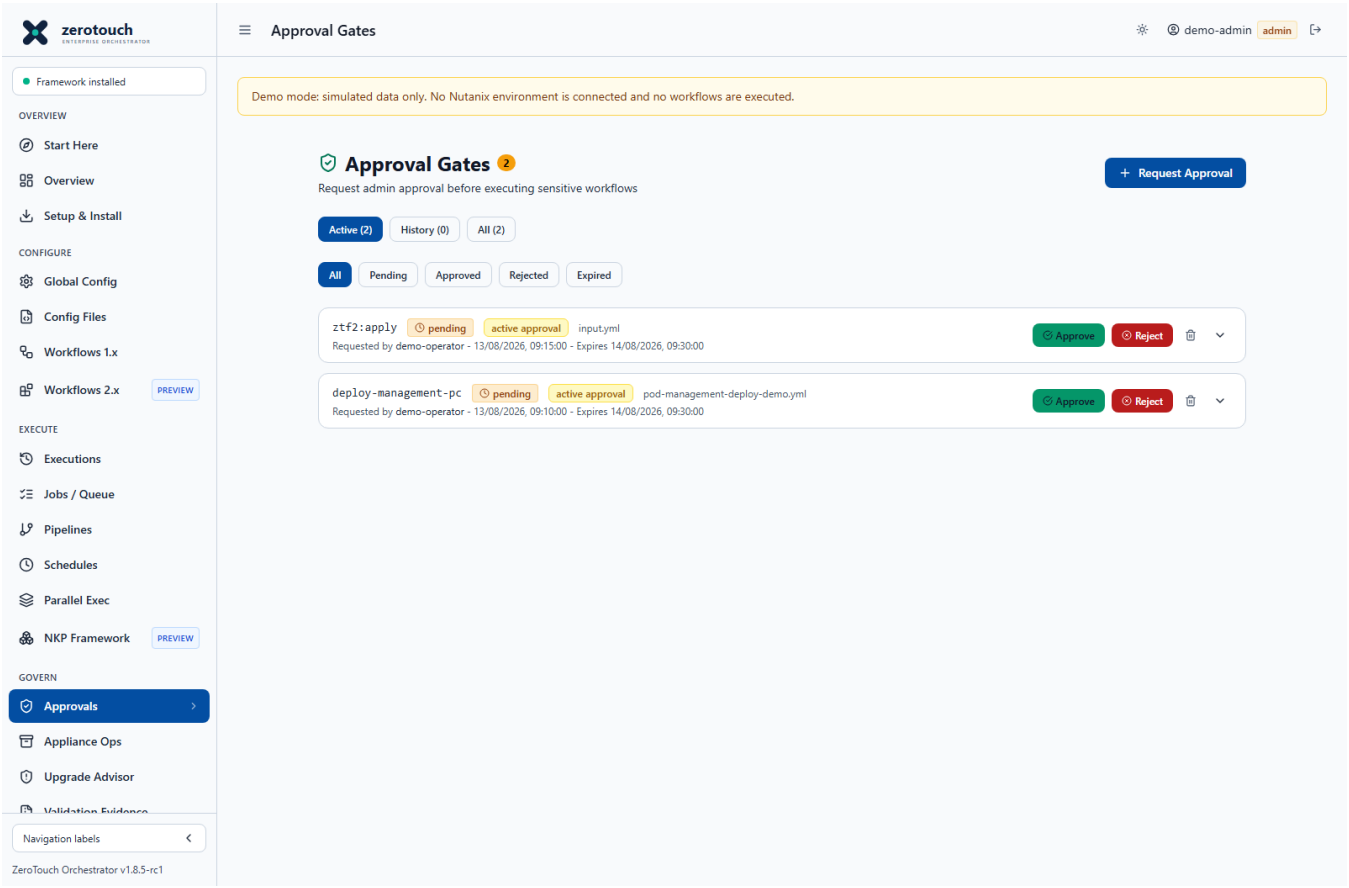


Figure 20. Approvals. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Operating procedure

1. Review the workflow, target scope, submitted YAML and requester identity.
2. Approve or reject only after checking change authorization and validation evidence.
3. Have the operator submit the exact approved configuration through the supported path.
4. Re-request approval if the configuration changes; inspect expiry and consumption state.
5. Keep audit and change-ticket evidence outside the console when required.

Use Approvals to create, approve, reject, and review governance requests.

Key functions:

Function	Purpose
New Request	Creates an approval request with workflow, config, requester notes, and context.
Active	Shows pending approval requests.
History	Shows approved, rejected, expired, or completed decisions.
All	Shows every approval record.
Status filters	Narrows the list by request state.
Expand row	Shows full request details and decision notes.
Approve	Admin decision action that allows matching governed execution.
Reject	Admin decision action that blocks the request.
Delete	Removes an approval request after confirmation.

Approval requests are used by workflow detail pages and controlled NKP phases when mandatory approval policy is enabled.

Appliance Ops

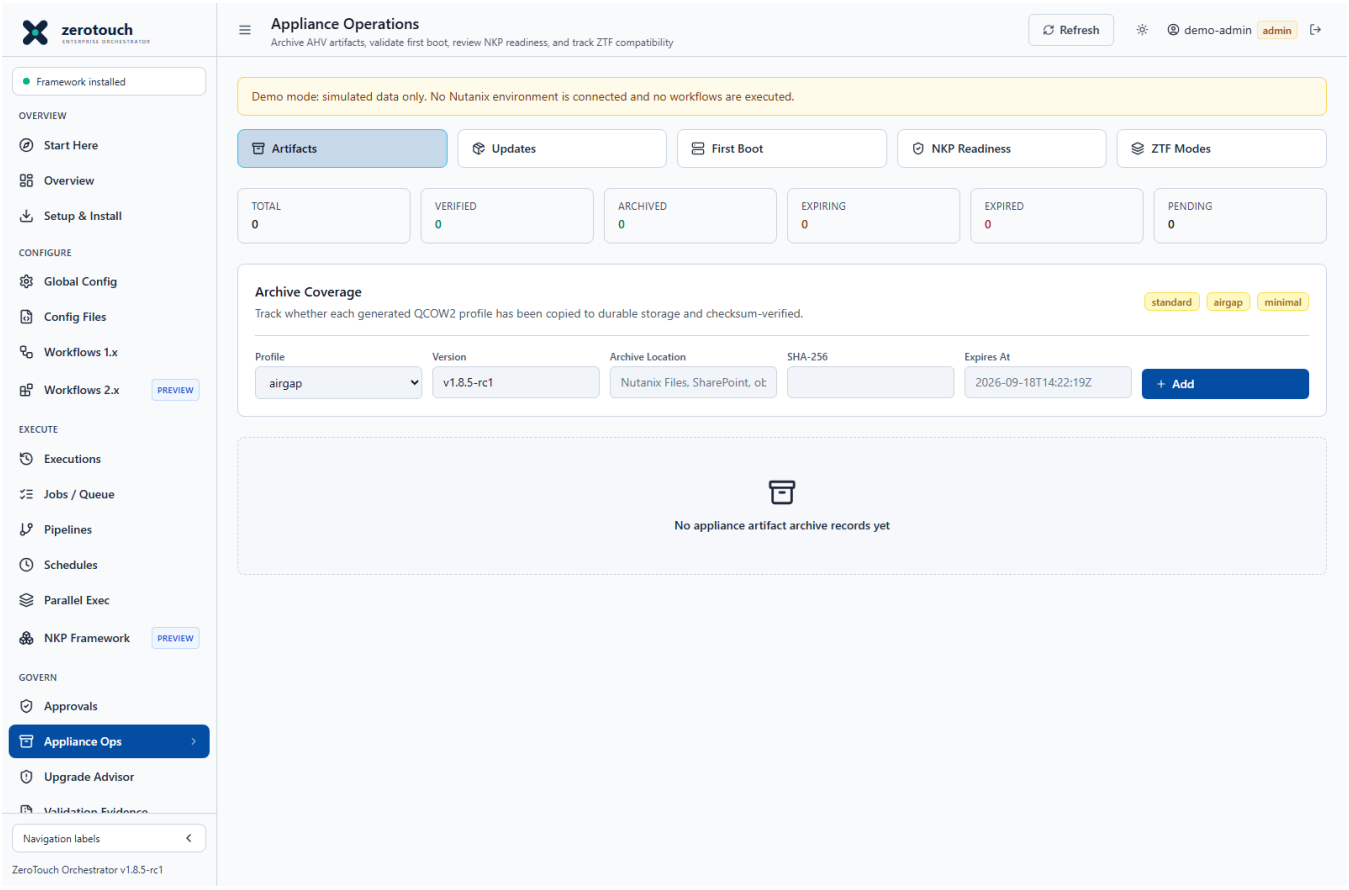


Figure 21. Appliance Ops. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Operating procedure

1. Select Artifacts, Updates, First Boot, NKP Readiness or ZTF Modes.
2. Review exact build identity and readiness findings, not only the footer version.
3. For updates, import and verify metadata or a package, then stage the intended request.
4. Have a host administrator apply the update with backups enabled.
5. Verify the running image, health and application data before accepting completion.

Use Appliance Ops to track appliance artifacts, first-boot/runtime state, compatibility, NKP readiness, and connected or air-gapped update packages.

Major areas:

Area	Function
Runtime and first boot	Shows appliance runtime checks, host layout state, data paths, and compatibility status.
Artifact archive	Records AHV appliance artifact metadata such as profile, version, checksum, and storage location.
Update manager	Checks GitHub update metadata, imports offline manifests, imports offline update packages, verifies package integrity, stages host-side update requests, marks updates applied, and deletes update records.
NKP readiness	Checks whether selected appliance profiles and framework paths are ready for NKP usage.
Compatibility	Displays whether configured ZTF and NKP framework paths match the supported orchestration mode.

Actions:

Action	Function
Refresh	Reloads appliance status and update inventory.
Create Artifact	Adds an archive record for a versioned appliance artifact.
Verify Artifact	Verifies an archive record's referenced metadata or checksum where available.
Delete Artifact	Admin cleanup for obsolete artifact records.
Check GitHub Update	Looks for connected update metadata from GitHub.
Import Manifest	Imports an offline update manifest.
Import Package	Imports an offline update package ZIP.
Verify Update	Validates update package metadata and integrity before staging.
Stage	Writes an appliance update request for host-side application.
Mark Applied	Records that the host-side update was applied and reviewed.
Delete Update	Admin cleanup for update records.

Staging an update is not the same as applying it on the host. Host-side update application remains an appliance operation that must be validated after the service restarts.

Upgrade Advisor

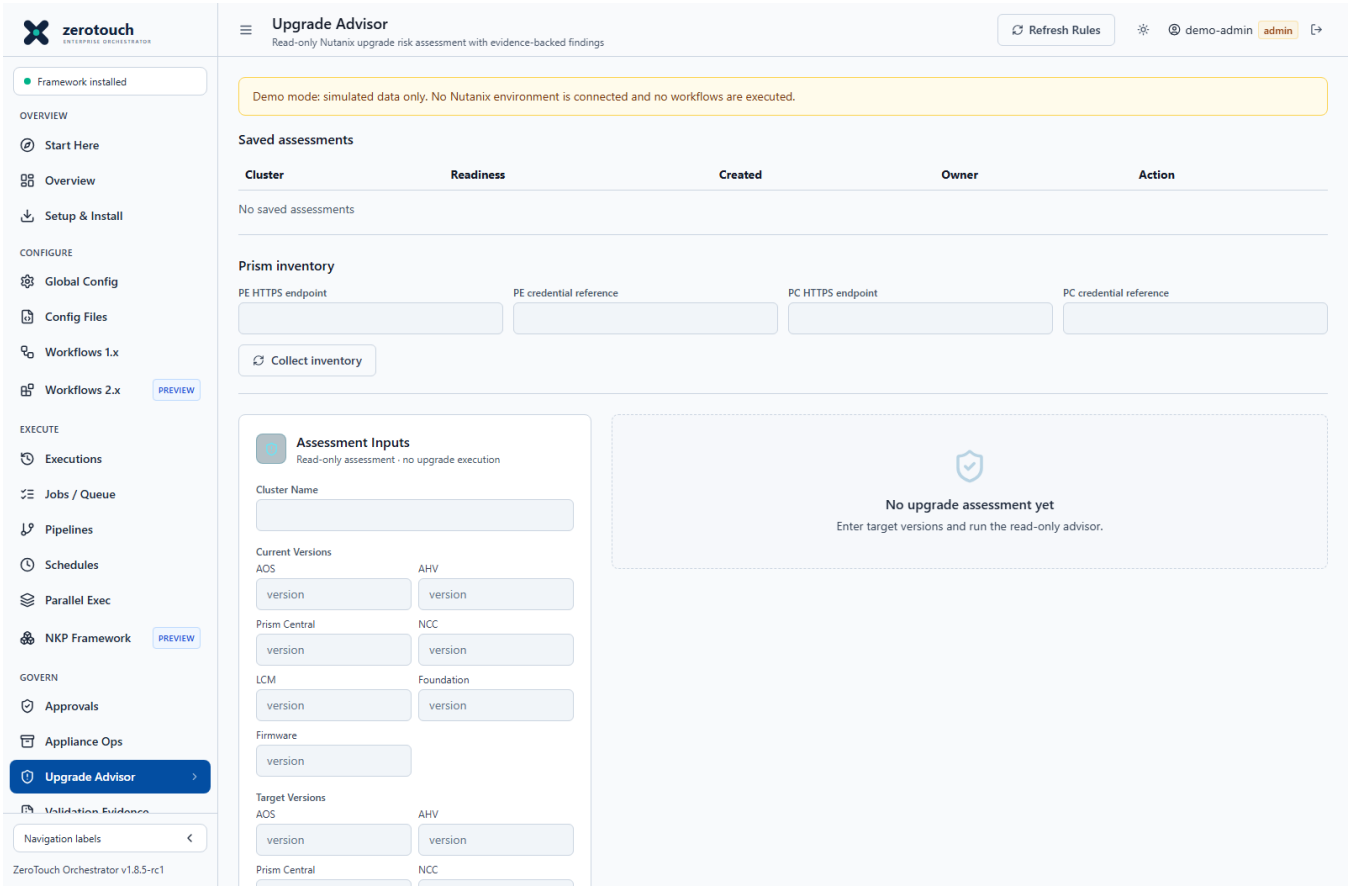


Figure 22. Upgrade Advisor. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Operating procedure

1. Create read-capable PE and optional PC credential references in Global Config. Establish the approved CA trust chain and choose HTTPS endpoint identities covered by the certificate SAN.
2. Enter PE HTTPS endpoint, PE credential reference and optional PC connection details. Select Collect inventory; inspect the collection timestamp, observations, Unknown components and errors before choosing target versions.
3. Enter verified remaining current versions and intended target versions. Complete only evidence flags supported by actual review. Run Assessment, then inspect Readiness actions, severity, sources, missing evidence and guidance.
4. Validate the proposed upgrade path using vendor compatibility information, release notes and actual LCM/NCC evidence. A clear advisory is not permission to execute an upgrade.
5. Select Save assessment snapshot after assessment. Reopen the immutable saved record for review or export it. Changed inputs require another assessment before saving; exports retain the displayed assessment rather than silently rerunning newer rules.

Use Upgrade Advisor for read-only Nutanix upgrade risk assessment.

Key functions:

Function	Purpose
Current Versions	Captures the versions currently running in the environment.
Target Versions	Captures proposed target versions.
Evidence toggles	Records whether LCM prechecks, release notes, compatibility review, Prism Central context, dark-site bundle review, or other supporting evidence has been completed.
Run Assessment	Produces findings classified as blocked, warning, , unknown, or clear.
Source Packs	Lets operators enable, disable, import, or delete curated rule packs.
Export	Downloads assessment output as an evidence bundle.
Collect inventory	Reads bounded PE cluster/host inventory and optional PC product header over verified HTTPS; unresolved components remain Unknown.
Save assessment snapshot	Creates an immutable record of exact reviewed inputs, findings, owner and timestamp.
Readiness actions	Shows blockers, missing evidence, sources and guidance; does not authorize or execute an upgrade.

Upgrade Advisor performs no LCM action or cluster mutation. PE inventory uses fixed authenticated GET requests; PC version detection uses an HTTPS product header and does not establish PC API permissions. NCC, LCM, firmware and Foundation versions still require manual evidence where no reader is implemented.

Inventory and saved assessment operations

Collect inventory uses verified HTTPS with fixed GET routes, ten-second timeouts, a 1 MiB response limit, no redirect following and no environment proxies. PE cluster/host reads populate AOS and homogeneous AHV versions. Mixed versions, paginated hosts and unsupported responses require manual review. Refresh clears earlier current-version entries rather than silently keeping stale values.

Collected observations, manual overrides and assessment inputs remain distinguishable. Saved snapshots retain their exact inputs, server-computed result, creator and timestamp; Save again creates a new record. The archive holds up to 200 records, requires administrator retention review at that limit and does not automatically delete records.

Admin and operator roles can collect inventory and save assessments; viewers can review shared saved assessments with endpoint redaction. The archive uses the configured file/PostgreSQL document backend and the single-process writer model. Do not load-balance writable instances against it.

For each readiness action, record an owner, evidence source and disposition in the change record. Resolve blocked and unknown findings, validate compatibility and prechecks independently, rerun the assessment after inputs change, and export the displayed assessment for review. No automatic KB scraping, firmware discovery or upgrade execution is implied.

Validation Evidence

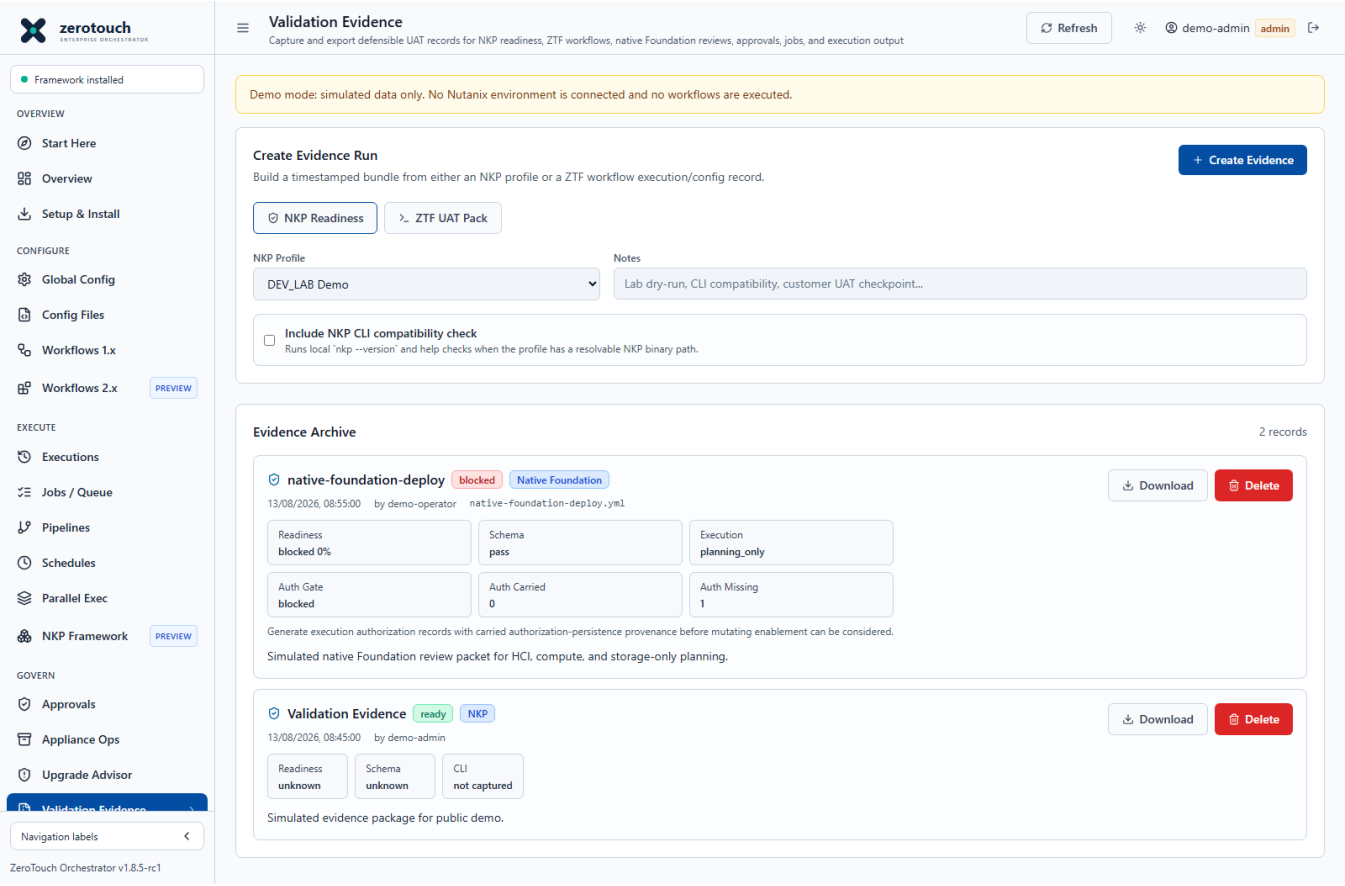


Figure 23. Validation Evidence. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Operating procedure

1. Select the intended profile, environment and validation target.
2. Complete checks and record the actual source and scope of evidence.
3. Capture or import the evidence and review blocked or needs-review findings.
4. Export a sanitized bundle with the matching job, configuration and change record.
5. Do not relabel simulated or operator-declared checks as live verification.

Use Validation Evidence to create and download defensible evidence records for UAT, handover, review, and change records.

Key functions:

Function	Purpose
Create Evidence Run	Builds an evidence record from selected source data.
Evidence source	Can reference an NKP profile, completed workflow/script execution, or saved config file.
Saved Config	Binds a config file to the evidence record.
Workflow / Script	Names the workflow or script under review.
Notes	Adds operator context for handover or UAT narrative.

Function	Purpose
Create Evidence	Saves the evidence archive.
Download	Downloads a ZIP bundle with generated summary and supporting records.
Delete	Admin/operator cleanup for evidence records after exact confirmation.
Create NKP Profile	Routes to NKP Framework when evidence needs a saved NKP profile first.

Evidence bundles can include readiness scoring, generated YAML, config hashes, approval/job/task references, parse results, redacted output, and Markdown summaries depending on the selected source.

Drift Detection

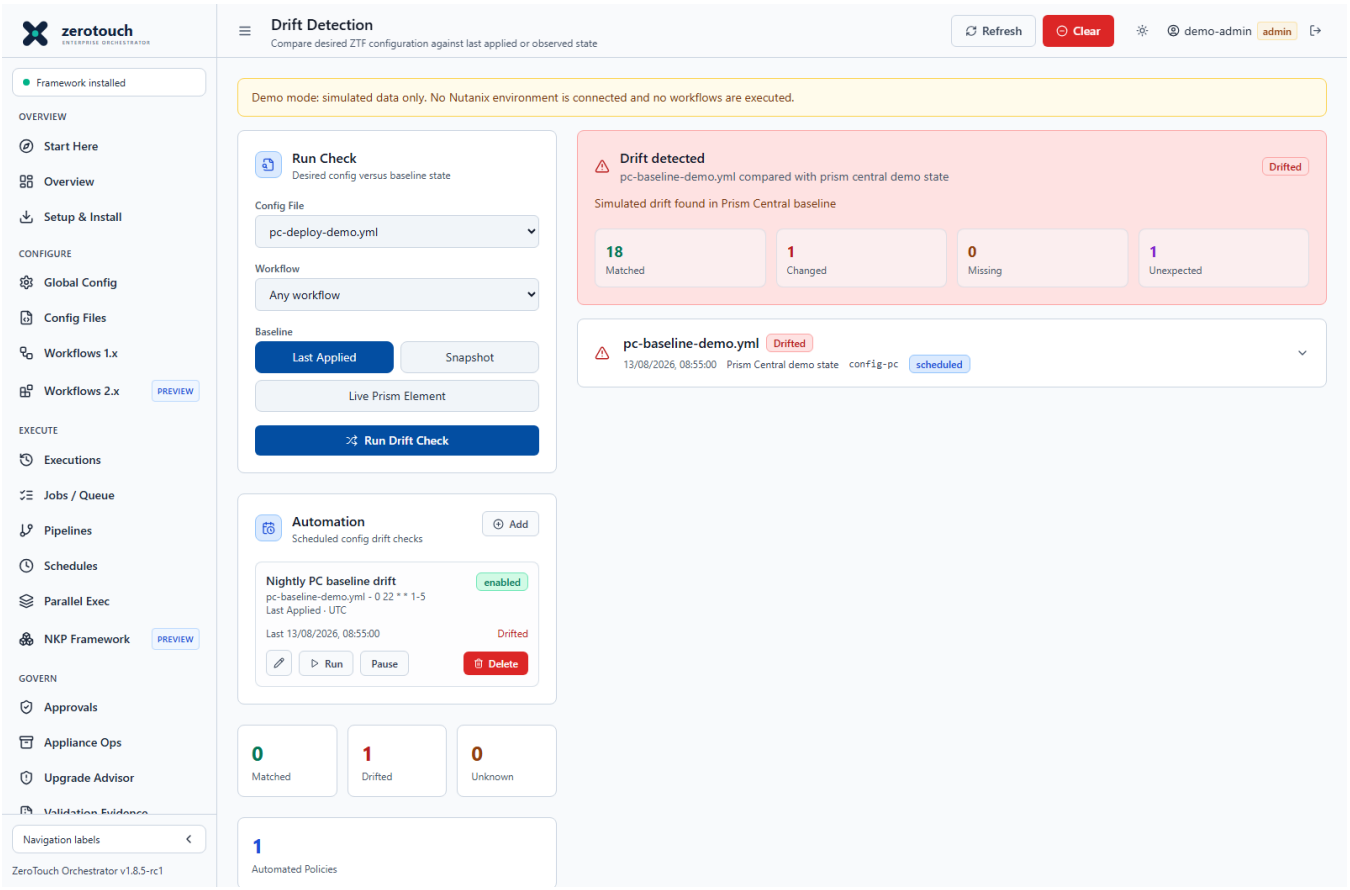


Figure 24. Drift Detection. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Operating procedure

1. Choose a saved desired-state file and the correct comparison baseline.
2. For live monitoring select Live Prism Element; configure trusted HTTPS and a readable PE credential.
3. Run a check and inspect both Drifted and Unknown findings.
4. Add a UTC Automation policy and verify it with Run now.
5. Use the pencil to edit the same policy, then confirm a fresh scheduled collection.

Use Drift Detection to compare intended configuration against a baseline.

Key functions:

Function	Purpose
Config File	Selects the desired saved YAML or JSON config.
Workflow	Optionally associates the drift check with a known workflow.
Last Applied baseline	Compares the desired config to the last successful applied config known to the app.
Snapshot baseline	Compares the desired config to pasted current-state JSON or YAML from Prism Central, Foundation Central, or another source.
Run Drift Check	Records a new comparison.
Latest summary	Shows matched, changed, missing, and unexpected counts.
History	Stores prior drift runs for later review.
Clear	Admin action to clear drift history.

Finding states:

State	Meaning
Matched	Desired and observed values align.
Changed	The same field exists but differs.
Missing	Desired content is absent from the observed baseline.
Unexpected	Observed content was not expected by the desired config.
Unknown	The baseline is unavailable or the comparison cannot be made confidently.

Use this screen before or after changes to understand whether intended configuration still matches the known or observed environment.

Audit Log

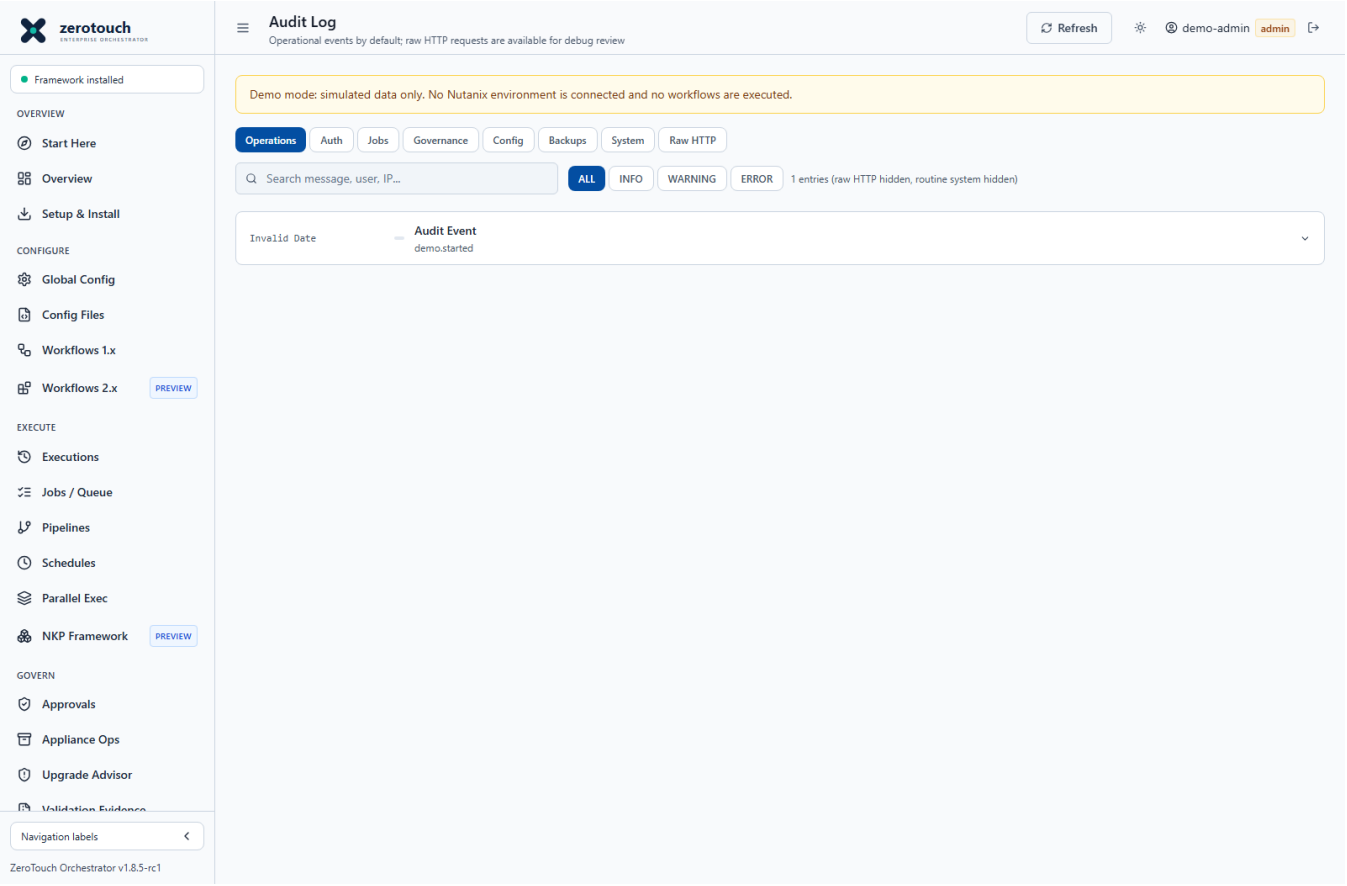


Figure 25. Audit Log. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Operating procedure

1. Open with an administrator account and filter the relevant time or action.
2. Correlate user, target, configuration, approval and job identifiers.
3. Preserve a sanitized record for the change or incident.
4. Apply retention only after required evidence has been retained externally.

Use Audit Log for admin-only operational event review.

Tabs and filters:

View	Function
Operations	Shows normal operator actions without raw routine HTTP noise.
Auth	Focuses on login and identity events.
Jobs	Focuses on job and execution activity.
Governance	Focuses on approval and controlled-change events.
Config	Focuses on config-file and YAML activity.
Backups	Focuses on backup and restore activity.
System	Includes routine system events.

View	Function
Raw HTTP	Includes raw request-style records for debug review.

Other functions:

Function	Purpose
Refresh	Reloads audit data.
Search	Filters records by text.
Expand row	Shows additional structured fields for a record.

Use Audit Log when investigating who changed settings, created approvals, triggered jobs, changed configs, performed backups, or accessed sensitive paths.

Admin Menu

Users

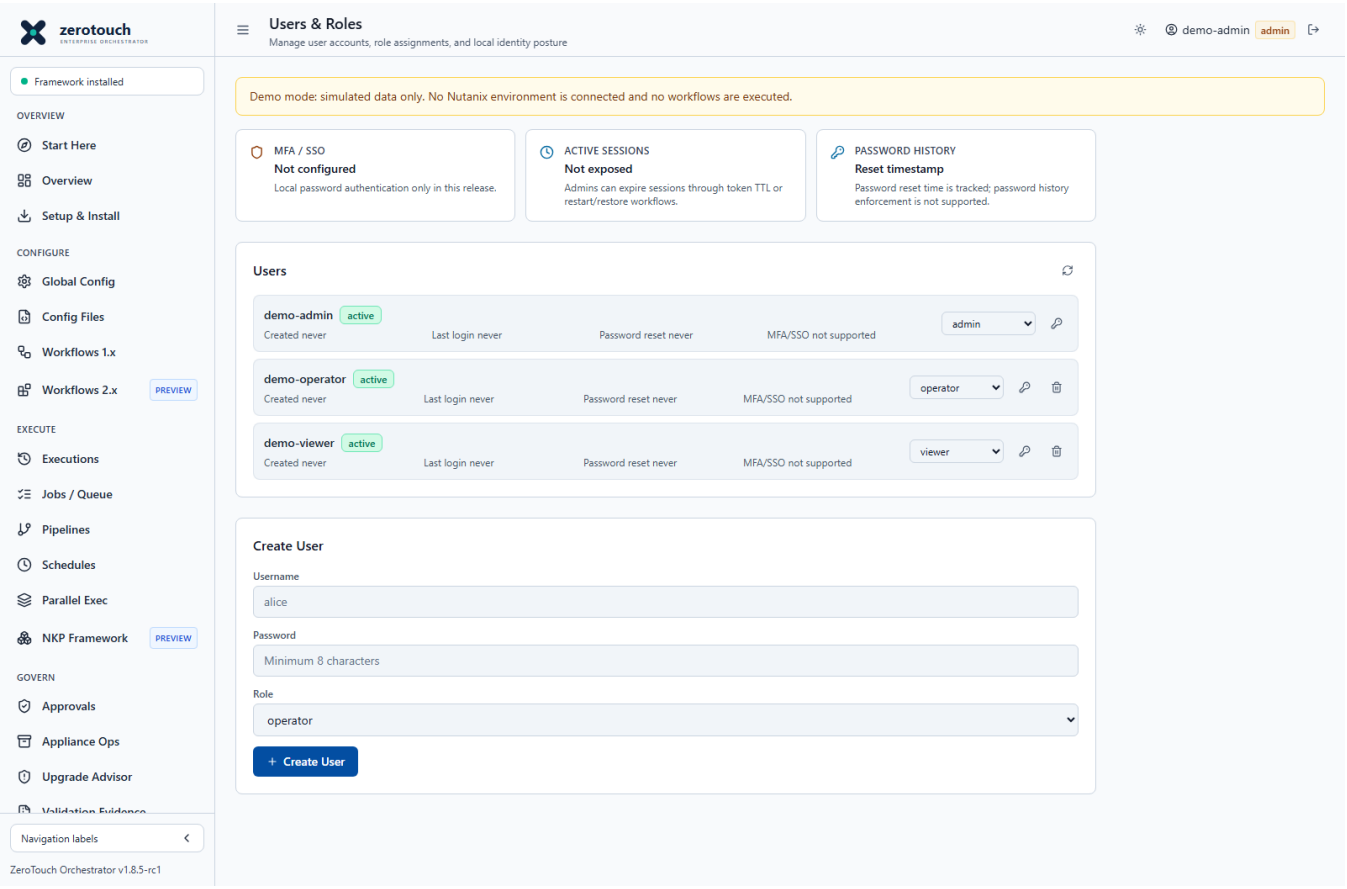


Figure 26. Users. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Operating procedure

1. Create a named account with the least role needed.
2. Provision initial access through a secure channel and change initial passwords.

3. Verify access with the intended role, including denied privileged operations.
4. Review inactive accounts and session/access requirements during offboarding.

Use Users to manage local ZTF-Orchestrator accounts and roles.

Key functions:

Function	Purpose
User list	Shows username, role, creation timestamp, and account metadata.
Create User	Adds a new local account with a selected role.
Reset password	Opens an inline password reset form for the selected user.
Delete user	Removes an account after exact username confirmation.
Refresh	Reloads account records.

Use admin accounts sparingly. Operators should have operator role unless they need user management, backup restore, audit, or destructive record cleanup.

Settings

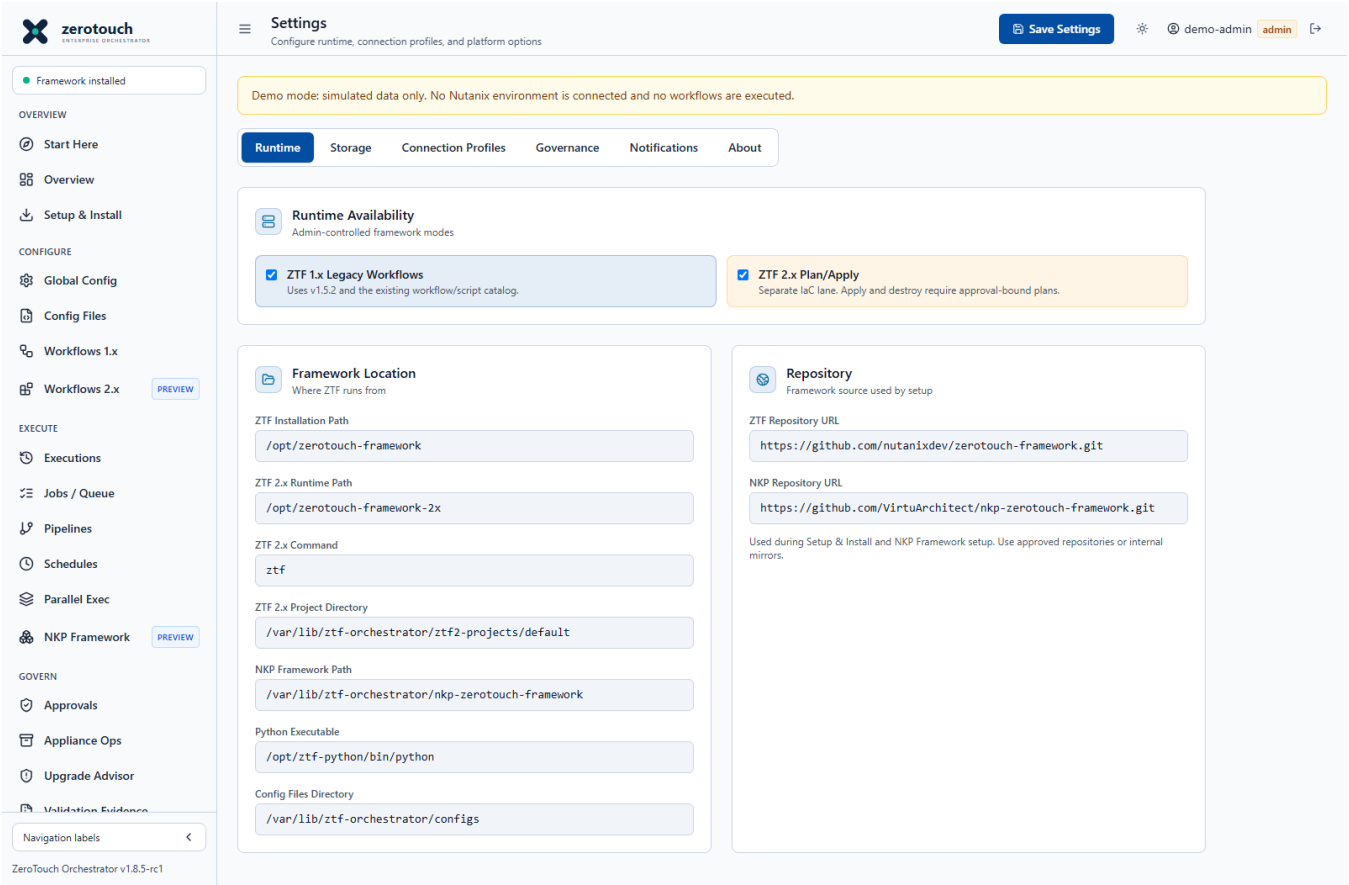


Figure 27. Settings. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Operating procedure

1. Check Runtime paths and execution limits before changing connection profiles.

2. Configure named profiles, governance rules and approved notification destinations.
3. Review Storage and backup posture; rehearse restore in an isolated environment.
4. Use About to copy Installed Build details for update verification and support.
5. Save only authorized changes and validate the affected operation.

Use Settings to configure runtime paths, storage, connection profiles, governance policies, notifications, and installed-build traceability.

Tabs and functions:

Tab	Function
Runtime	Configures the ZTF framework path, Python executable, config directory, NKP path, and NKP repository URL.
Storage	Shows backend mode, data directory or database location, retention settings, and PostgreSQL backup inventory. Admins can create, download, and restore logical backups.
Connections	Manages reusable connection profiles for Prism Central, Foundation Central, Prism Element/CVM, NCM/Calm, directory services, IPAM, DNS, NTP, timezone, and site defaults.
Governance	Selects which workflows require mandatory approved request IDs before direct execution.
Notifications	Configures outbound completion webhook URL.
About	Shows UI version and Installed Build metadata, including source ref, commit, build date, container image, and update package when reported.

Connection profile functions:

Function	Purpose
Add	Creates a new connection profile.
Duplicate	Copies the current profile as a starting point.
Delete	Removes the current profile after confirmation.
Test Connection	Tests TCP reachability for Prism Central, Foundation Central, Prism Element, NCM, directory, or IPAM endpoints.
Copy YAML	Copies generated ZTF defaults for the active profile.

Storage functions:

Function	Purpose
Refresh	Reloads health and backup inventory.
Create Backup	Creates a PostgreSQL logical backup when PostgreSQL storage is active.
Download backup	Downloads a stored backup.
Open Restore Confirmation	Starts a guarded restore flow.

Function	Purpose
Create Safety Backup and Restore	Creates a safety backup first, then restores the selected backup after exact confirmation.

About functions:

Function	Purpose
Refresh	Reloads runtime health and installed build details.
Copy Build Info	Copies version, ref, commit, image, and update-package metadata for support or UAT records.
Open Applied Release	Opens the release URL when an applied update reports one.

Use Installed Build from Settings > About when confirming exactly which patch, container image, or appliance update is running. The footer version alone may not distinguish patched builds that share the same UI version.

ZTF 2 x IaC workspace

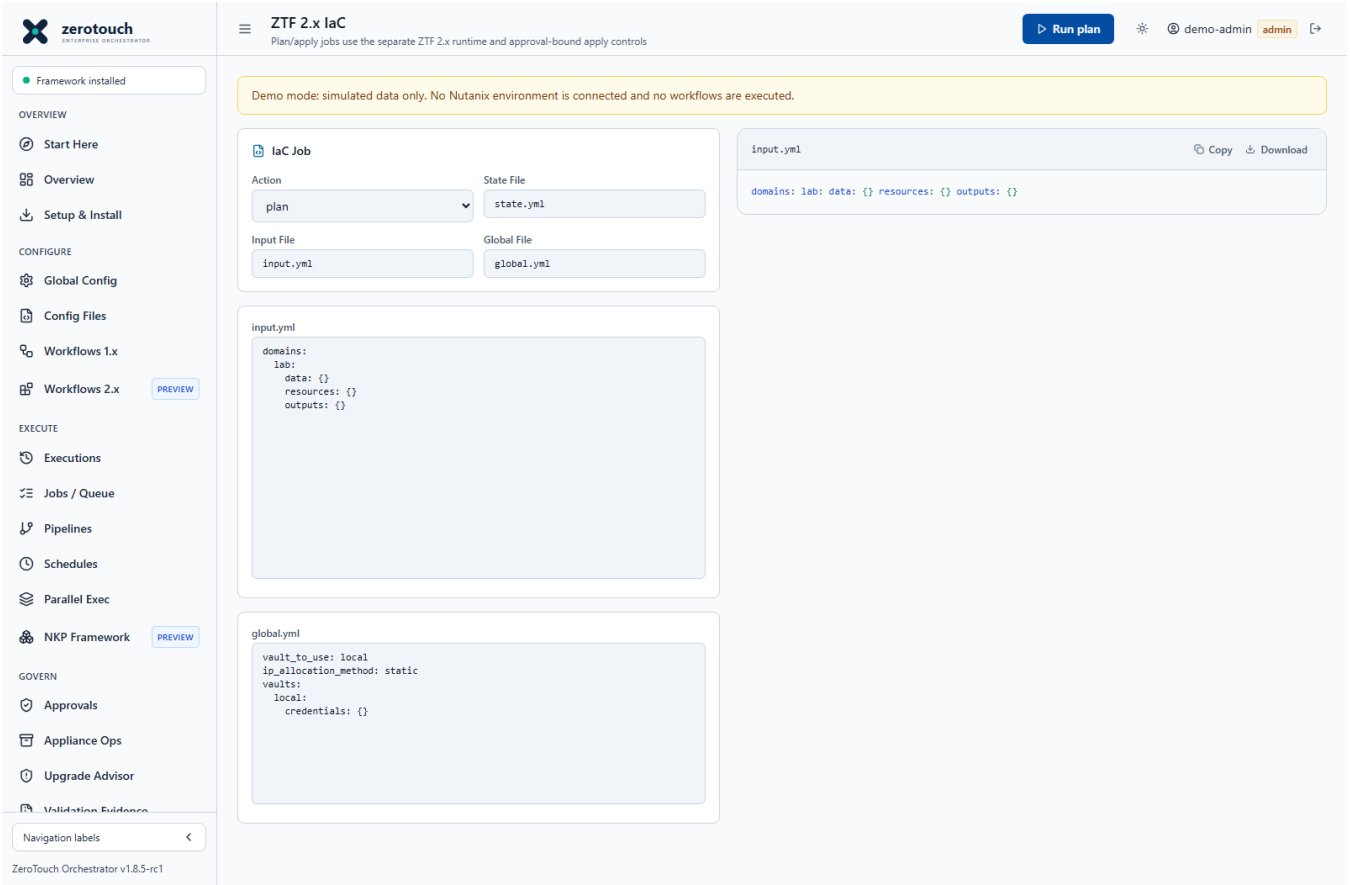


Figure 28. Separate ZTF 2.x plan and apply workspace. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Use this workspace to review generated desired-state input, global configuration, plan output and state identity. Keep plan and apply bound to the reviewed inputs and permitted provider operations. The 2.x

runtime is separate from the legacy workflow/script launcher. Review its Preview status, runtime gates and environment-specific validation before applying changes.

Deploy an AHV HCI cluster with Foundation VM

Use Cluster Create (Foundation VM), not Cluster Create or a Standalone FCA entry. The Foundation VM must already be deployed with a supported image/platform combination and reach the relevant BMC, host and CVM networks. The installer address identifies the Foundation VM, not a future AHV host or cluster VIP. The screenshot initially contains demo defaults; replace them with the actual classic Foundation endpoint, typically http://foundation.example.com:8000.

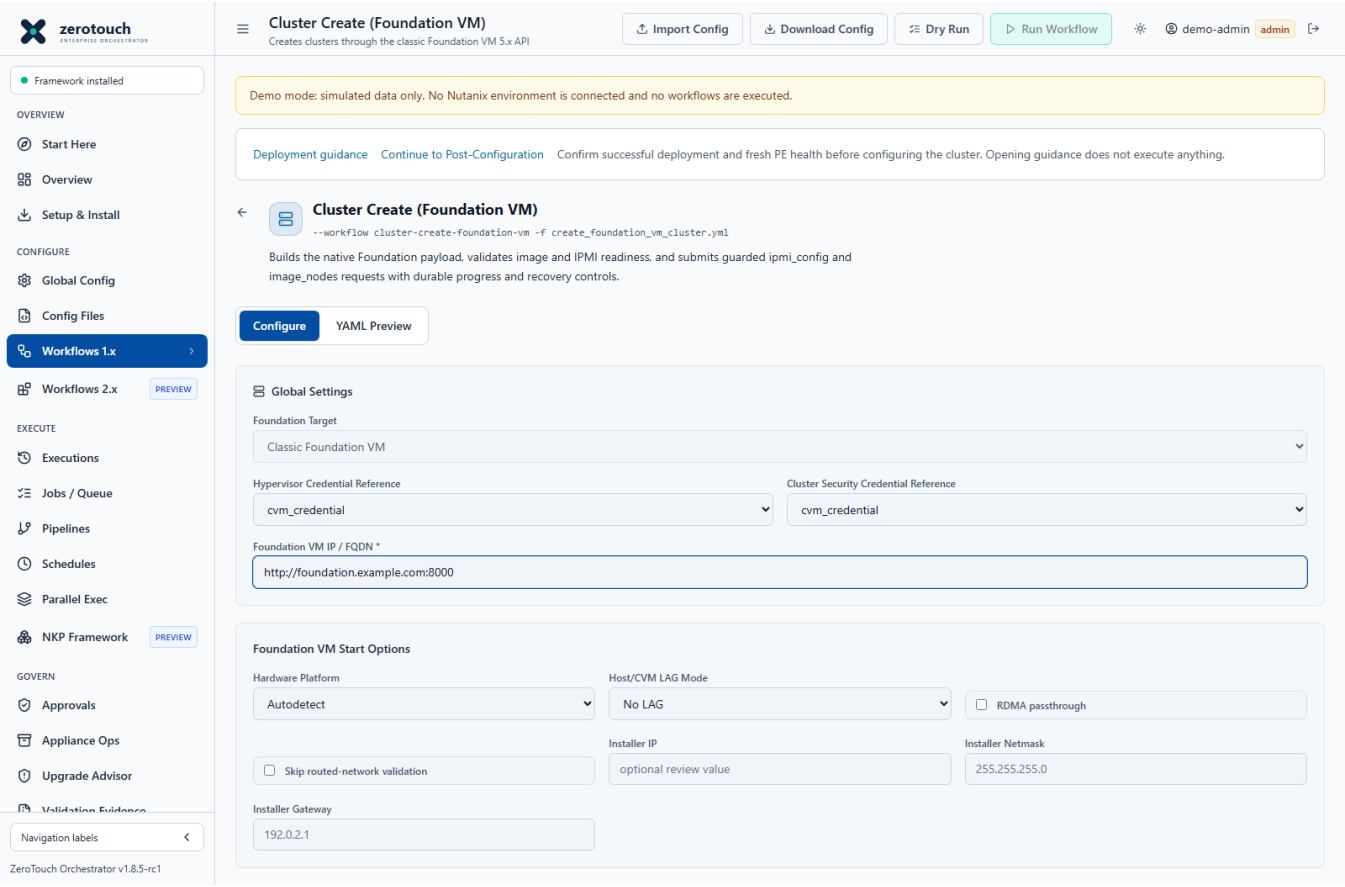


Figure 29. Classic Foundation VM workflow and start options. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Prepare the deployment

1. Confirm physical hardware, firmware, AOS and AHV compatibility through the applicable vendor support information. A Dell platform selector is not an assurance that every Dell server is supported.
2. Reserve unique IPMI, AHV, CVM, cluster VIP and optional data-services addresses. Confirm VLANs, routing, MTU, DNS, NTP, firewall paths and switch configuration.
3. Create separate Global Config references for hypervisor credentials, cluster security credentials and each current BMC account. Use the actual iDRAC username and password; never substitute Prism credentials.
4. Stage the approved AOS package and AHV ISO in Foundation, or use the controlled upload after enabling its independent administrator gate. Refresh inventory and select exact names.

5. Supply the AHV ISO Checksum expected by Foundation. The UI preserves aos_hypervisor_images.hypervisor_checksum through import, export, validation and submission. Accepted hexadecimal lengths are MD5 32, SHA-1 40 and SHA-256 64 characters. An empty field is not a verification result; do not invent a digest.

Complete the form

1. Enter the classic Foundation VM endpoint. Do not use a Prism 9440 URL or /fc/gui/.
2. Select hardware platform, supported LAG mode and RDMA settings. Installer IP, netmask and gateway describe the installer interface for review; they are not the host/CVM address assignment and do not remotely change the VM network.
3. Enter AOS and hypervisor image choices, DNS/NTP and the corresponding subnet, gateway and VLAN inputs. Avoid skipping network validation to hide a real routing problem.
4. Enter each cluster name, VIP, redundancy target and timezone. RF-2 and RF-3 form constraints do not replace hardware-specific resilience guidance.
5. Add each node with real block identity, serial, position, role, AHV hostname/IP, CVM IP, BMC IP and credential reference. Use physical position within the block; do not treat every single-node block as consecutive positions A through D unless that matches the actual hardware layout.
6. Set CVM memory for the supported platform and workload. The form minimum is 20 GB, not a universal sizing recommendation.
7. Enable Configure IPMI network only when the deployment intentionally changes BMC networking. An existing reachable BMC normally does not require this operation. Review cluster security, SSH key/password controls and Skip cluster formation explicitly.

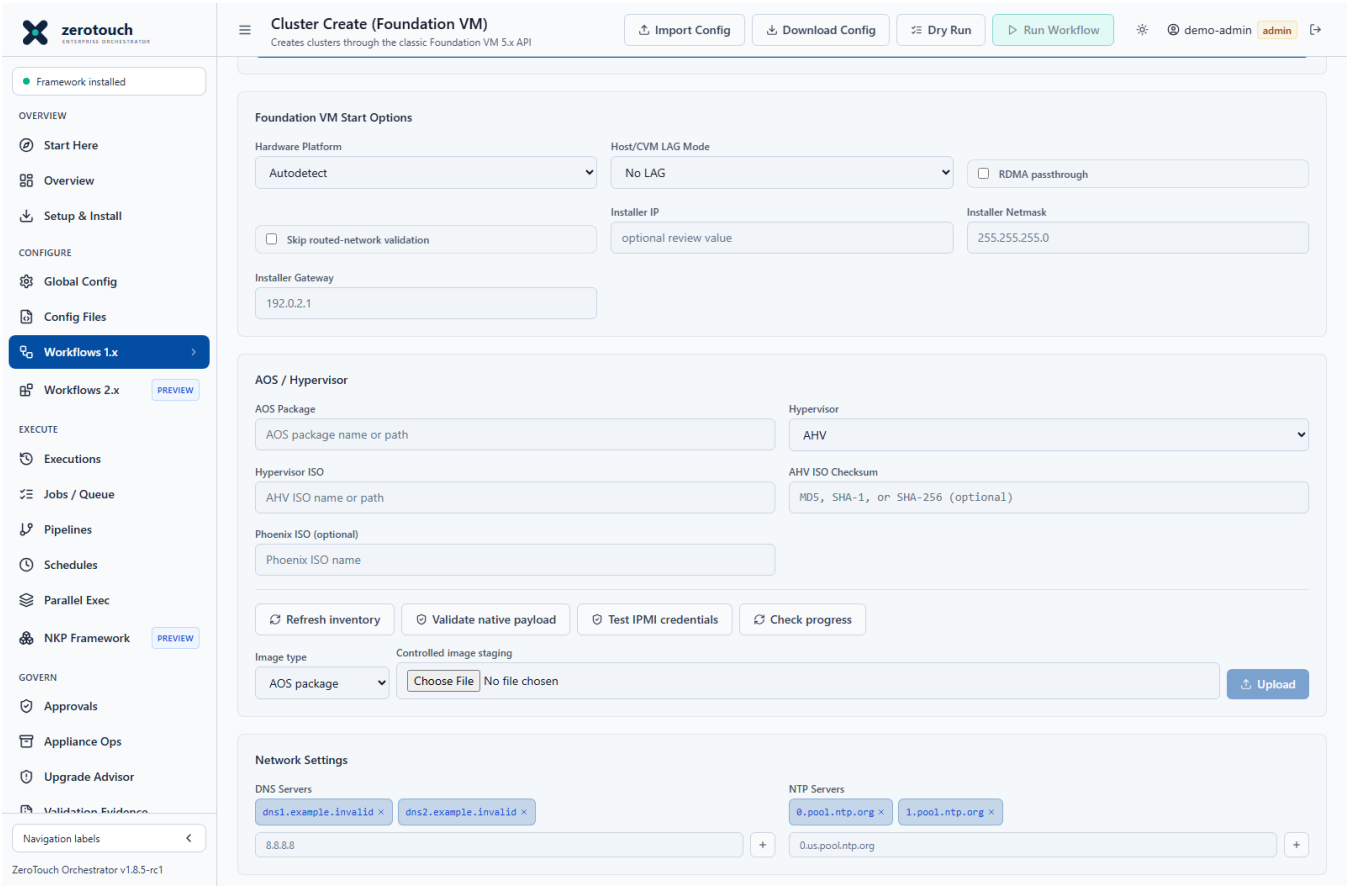


Figure 30. Image inventory controls and AHV ISO checksum. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Image input	Operator check
AOS Package	Select the exact approved installer package available to Foundation. A host-local source path is not automatically a path inside Foundation.
Hypervisor ISO	Match the filename and hypervisor family to Foundation inventory and the supported AOS/platform combination.
AHV ISO Checksum	Use the actual digest expected by Foundation. A valid hexadecimal shape alone does not prove image integrity.
Upload and inventory	Upload is independently administrator-gated. After staging, refresh inventory and validate the selected image before submission.

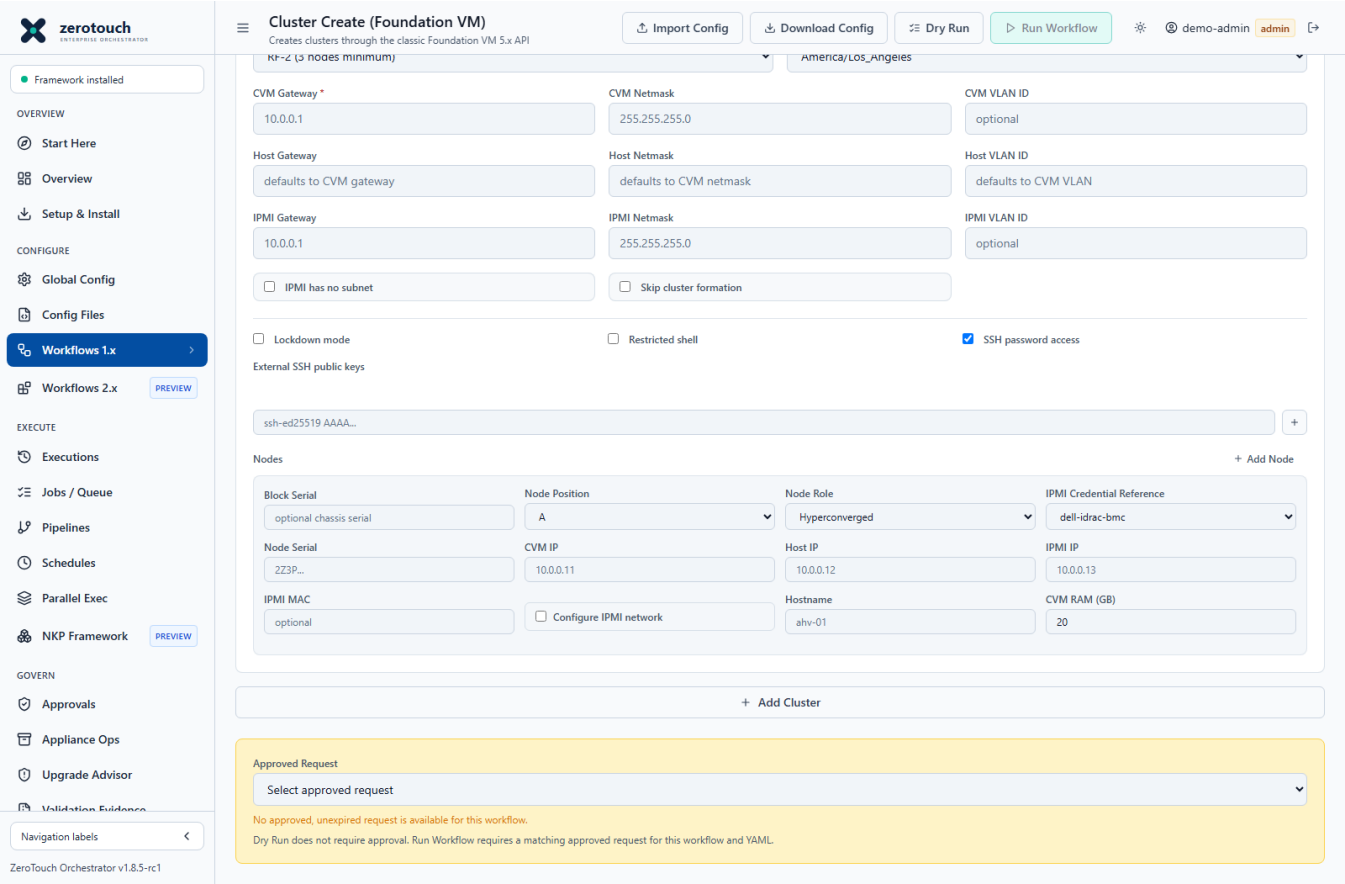


Figure 31. Per-node network and credential inputs. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Validate and submit

1. Save the generated create_foundation_vm_cluster.yml and inspect the complete YAML Preview, including all four node identities if deploying a four-node cluster.
2. Refresh image inventory, Validate native payload and Test IPMI credentials. Testing is read-only but failed logins can trigger BMC lockout policy.
3. Run Dry Run and resolve reference, schema, image, checksum and connectivity errors before mutation. Reference resolution is not proof that the referenced password authenticates.
4. Have the host administrator explicitly configure ZTF_FOUNDATION_VM_ENABLE_MUTATION=true for approved use. Docker Compose

defaults its substitution to false; the server fallback and an appliance profile can differ. Verify effective configuration rather than assuming all builds enable mutation.

5. Enable ZTF_FOUNDATION_VM_ENABLE_IMAGE_UPLOAD=true only when administrator image uploads are needed. Images already present do not require that flag. The default upload size cap is 20 GiB.
6. Obtain the applicable approval, run as administrator, and type DEPLOY FOUNDATION VM. Review the destructive scope: imaging may overwrite the selected nodes.

How the workflow interacts with Foundation

The backend validates intent, queries Foundation version and image inventory, resolves execution credentials and constructs native blocks and clusters JSON. It logs a sanitized payload hash. It calls ipmi_config only for nodes that request BMC network configuration, then submits image_nodes and polls /foundation/progress. It persists sanitized logs and status in the job. Foundation performs the installation and cluster formation; the Orchestrator coordinates and reports the response.

Monitor and recover

Expand deployment status and Node Overview, and use Open Foundation and sanitized-log download. Classic Foundation may report grouped Node imaging / configuration followed by Cluster formation rather than separate AHV and AOS milestones. Not reported means telemetry was absent; never infer historical phase completion from an aggregate percentage alone. Cancelling a running job requests Foundation abort_session. An in-flight target action can finish before cancellation takes effect. Following an Orchestrator restart, inspect interrupted jobs and the actual Foundation session. Administrator restart requires RESTART FOUNDATION VM and records a new job linked to its source; it is not an assurance of idempotent continuation.

Multiple clusters and remote sites

For a single Foundation VM, serialize independent deployments and confirm the previous session is finished before starting another. Session-wide progress and abort operations should not be assumed isolated across concurrent jobs. A Foundation VM per site can provide independent deployment endpoints, but verify network paths, image copies, approval and session ownership separately. The legacy Parallel Exec and pipeline selectors are not a supported substitute for the specialized Foundation VM execution path. Do not assume that choosing cluster-create with Foundation VM YAML selects this backend.

Accept the cluster

Verify Prism Element login, node membership, cluster services, intended AOS/AHV versions, capacity, network and time settings, and vendor health checks. Confirm any optional PC registration independently. Save Foundation logs, job/configuration identifiers and acceptance evidence before handing over. A green console job alone is insufficient.

Initial Prism Element configuration and password changes

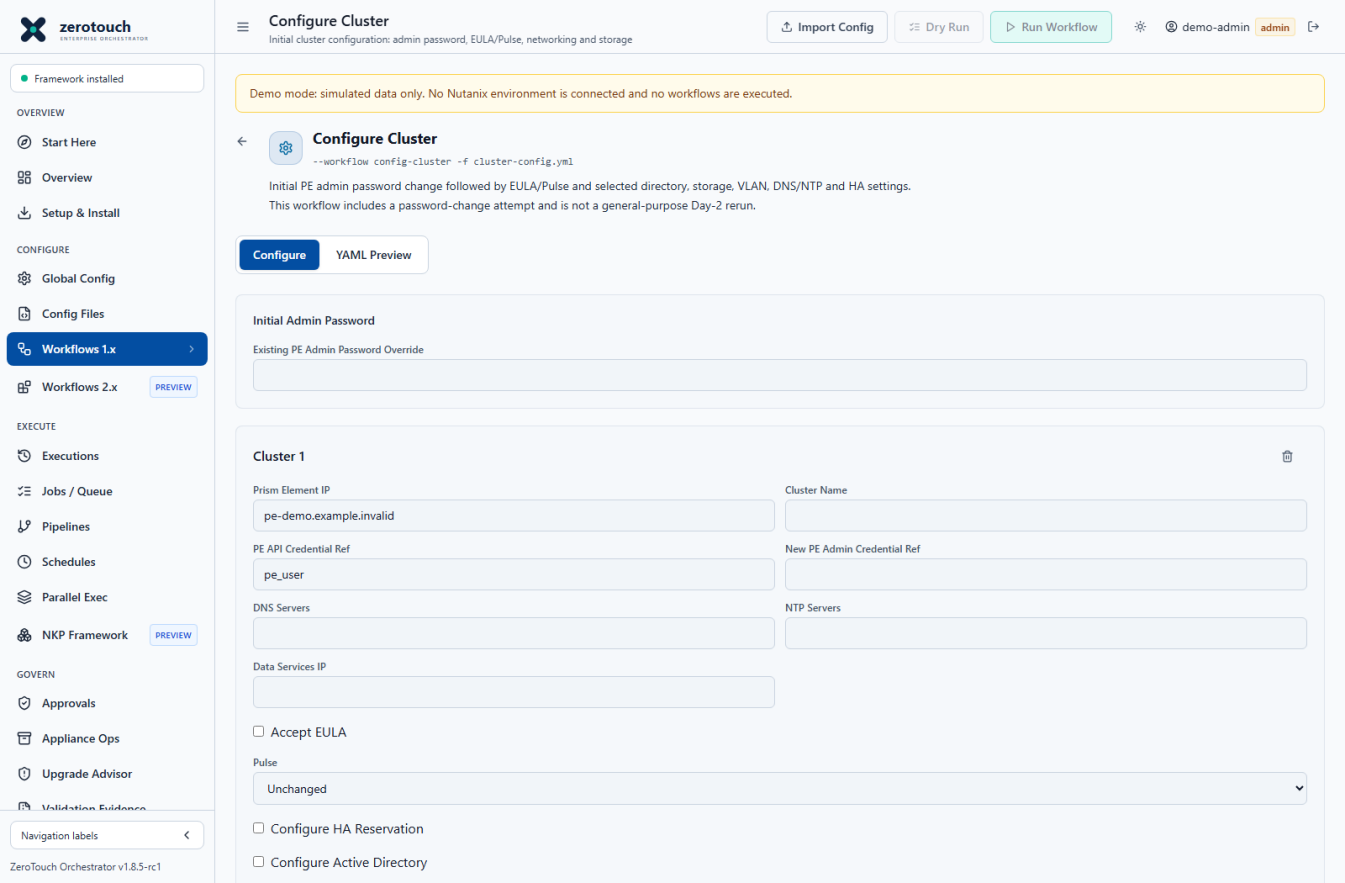


Figure 32. Configure Cluster form. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Configure Cluster (config-cluster) invokes the installed ZTF 1.x ClusterConfig workflow. In updated source-derived RC builds, the password action runs only when new_pe_admin_credential is supplied. Without that field, the password substep explicitly skips and selected EULA/Pulse, network, storage and registration actions can proceed using pe_credential. The workflow remains mutating and requires normal authorization.

Procedure

1. If an initial PE admin password change is required, create a separate Global Config reference whose username is exactly admin and whose value is the approved replacement password. Retain the existing reference until fresh replacement authentication is verified. If already changed, use the working pe_credential and omit new_pe_admin_credential.
2. Open Configure Cluster and add the intended PE VIP and cluster name. For a combined initial password change, select the replacement reference and review both new_pe_admin_credential and pe_credential in the generated YAML. For later narrowly scoped files, omit new_pe_admin_credential entirely; do not assume an empty or reused initial-password field is a repeatability safeguard.
3. Existing PE Admin Password Override uses top-level default_pe_password. Leaving it blank uses the upstream shipped default; pe_credential alone does not supply the old-password argument.

The override is sensitive plaintext in YAML and is shared across targets. Split targets with different old passwords and protect exports, saved YAML and screenshots.

4. Select Accept EULA only with authorization and provide the EULA name/company. Pulse offers Unchanged, Enabled and Disabled; Unchanged omits enable_pulse.
5. Add only the required DNS/NTP, data-services IP, HA, directory services, containers and VLANs. There are no automatically created containers/networks. HA and storage settings still require hardware/version compatibility review.
6. Review/download the YAML and run Dry Run. Password credentials intended for use after the change need not authenticate before the change. Dry Run is not evidence that a real password change or cluster configuration succeeded.
7. Obtain the required approval and exact confirmation. Password PASS requires fresh replacement authentication and a nonempty PE cluster identity before the verified session is handed to later actions. Independently verify new login, EULA/Pulse and selected settings. Update saved connection references; the action does not rewrite other files. Investigate any ambiguous change before retrying.

Native Schema

The form preserves native imported per-cluster and additional settings when editing. Invalid imports, legacy form keys and missing required fields are reported instead of silently converted. New-password and directory service passwords are credential references, not generated plaintext values.

Setting	Native key
New admin password	clusters.<VIP>.new_pe_admin_credential; omit when no password change is requested
API credential after change	clusters.<VIP>.pe_credential
EULA	eula.username, company_name, job_title
Pulse	enable_pulse boolean; omit to leave unchanged
Directory service	directory_services, ad_directory_url, service_account_credential
Role mappings	role_type, entity_type, values
Storage	containers, compression_enabled, on_disk_dedup: ON/OFF
HA	ha_reservation.enable_failover, num_host_failure_to_tolerate
VLAN	networks[].subnet_type: VLAN, vlan_id, ip_config
IP pools	ip_config.pool_list[].range, using space-separated addresses

Previous form keys pulse, storage_containers, active_directory, boolean ha_reservation and flattened network addressing are not valid native inputs. Correct them in the source YAML or recreate the configuration in the form.

The bundle applies an idempotent guard to the upstream role-mapping script so omitting optional directory services does not dereference None.

Pipelines And Validation

Where policy permits, save the generated configuration and select Configure Cluster as one pipeline step. Approval-required workflows remain blocked from pipelines and must use the direct approved-job path. No approval policy is changed by this form correction.

Automated checks cover native form import/export, guarded password handoff, optional skips, requested container-property verification and pipeline failure gates using isolated fixtures. These checks do not establish live PE password-change, EULA/Pulse, storage or TLS compatibility on a particular appliance.

Standalone PE admin password change

The installed ZTF 1.x script ChangeDefaultAdminPasswordPe is exposed under Scripts > Prism Element and in the pipeline step selector. It changes the PE admin password, not AHV root, CVM nutanix, or iDRAC credentials.

1. Create a separate credential reference for the new PE admin password in Global Config. Retain the existing reference until the change is verified.
2. In Scripts, select Change Default Admin Password (PE). Enter the PE VIP, cluster name, existing PE credential reference, and new PE admin credential reference. An optional masked existing-password override is available; leaving it blank uses the upstream shipped default.
3. Generate and review the YAML. An override is stored in that YAML when supplied; protect the editor, downloads and saved file as sensitive material. The new password remains a reference rather than a literal in the generated config.
4. Run the script with exact confirmation RUN ChangeDefaultAdminPasswordPe. Updated source-derived builds create a fresh session with the replacement credential and GET /api/nutanix/v2.0/cluster, requiring cluster identity before PASS. A failed old-password request alone is not success. Independently verify a new Prism login; older unpatched builds lacked this stronger check.
5. Update the credential reference used by subsequent cluster workflows. The script does not automatically modify saved Orchestrator credentials.

For pipelines, save the generated YAML in Config Files, add Change Default Admin Password (PE) (Script), and select that file. Run confirmation is required each time and is not persisted in the pipeline. The step uses --script, while normal workflow steps continue to use --workflow. A failed step stops subsequent steps; ZTF ERROR/CRITICAL output also fails the password step even with exit code zero. Do not automatically retry a password change after an ambiguous network failure.

Do not repeat an initial password-change request in subsequent files. If using the standalone script first, later Configure Cluster files must use the verified replacement pe_credential and omit new_pe_admin_credential. Alternatively, perform the password change once inside the initial combined Configure Cluster step.

Subsequent steps must already reference the new credential, or be run separately after verifying the password and updating the saved reference. Mandatory approval policy cannot be bypassed through a pipeline: approval-required actions must use the direct job path with an approval ID.

Validation includes mocked execution and browser tests. No real cluster password is changed by those checks; environment-specific validation remains required.

Build a safe post deployment pipeline

1. Finish and accept the Foundation deployment first. Preserve access and an independent recovery route.

2. Use one approved initial-password file only when a password change is still required. Store the intended replacement admin reference separately. If the password is already changed, omit that stage and start with working authentication. Keep the password-change action separate when a manual readback gate is required.
3. Where policy permits, build a sequential pipeline from narrowly scoped Configure Cluster files. Use the verified replacement pe_credential in every later file and omit new_pe_admin_credential in those files. Approval-required workflows remain on the approved direct-job path; do not weaken policy to make a pipeline run.
4. Recommended order: optional initial admin password; authorized EULA and approved Pulse setting; DNS/NTP; storage and VM networks; optional HA and PE local administrators; PE registration to PC; independent final acceptance. Separate sensitive stages to limit partial changes. Use one reviewed set of target-specific files per cluster.
5. In an existing six-stage pipeline, keep stage 01 for an initial password change only if needed; stage 02 for EULA/Pulse; stages 03-05 for the three reviewed containers; stage 06 for PC registration. Add DNS/NTP, VM networking, HA or local users only when the actual approved configuration requires them. A filename alone does not prove the included settings.
6. Password SKIP is expected when no replacement is requested. FAIL or CAN'T VERIFY for required password, EULA, Pulse, container or PC registration verification must stop later pipeline steps, including when the subprocess exits zero. Configure Cluster can run internal actions concurrently; a failed step does not reverse changes already started within that step.
7. Independently verify replacement authentication, EULA, Pulse, DNS/NTP, requested container properties, VM networks and exact PE identity in PC. Keep run, config and approval identifiers and sanitized readbacks. Reconcile partial results before rerun; a pipeline does not provide transactional rollback.

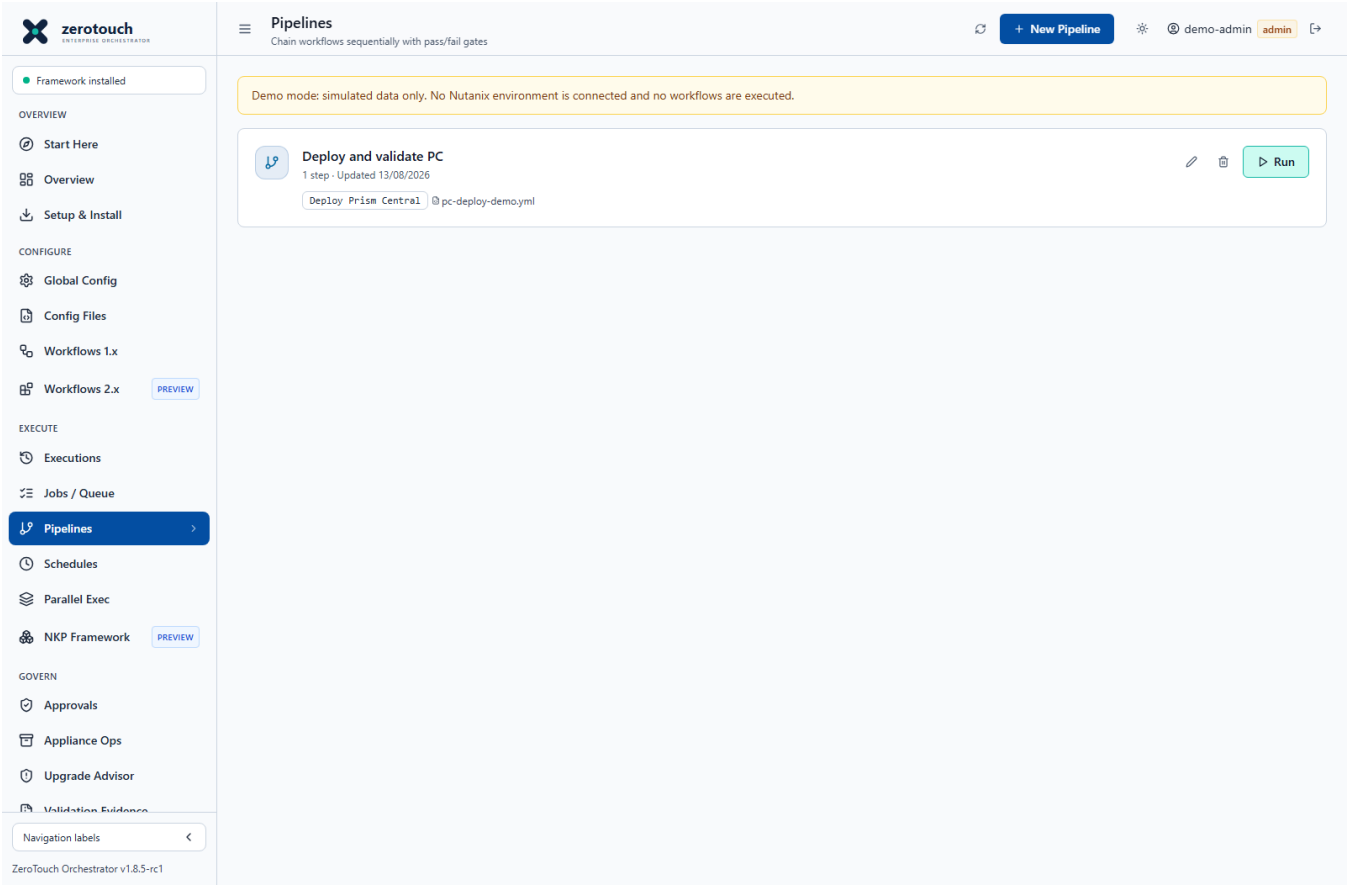


Figure 33. Sequential pipeline definition and execution entry. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Verify requested storage container properties

The patched runtime reads container inventory and verifies every explicitly requested supported field: replication factor, advertised and reserved capacity, compression and delay, erasure coding, fingerprint-on-write and on-disk deduplication, encryption, storage-pool UUID and affinity-host UUID where requested. Unspecified fields are not asserted. Fields ending in _in_gb retain binary units: one GiB is 1,073,741,824 bytes.

Numeric strings normalize without rounding; booleans and ON/OFF values are checked. Unknown property names, invalid values, duplicate request names and nonempty legacy NFS whitelists are rejected before creation. Missing API fields, duplicate readbacks and mismatched values cannot prove success. An existing matching container passes without alteration; mismatches are reported by field name and are not automatically repaired.

For a controlled failure-gate test, use only an approved existing UAT container and a narrowly scoped file with one deliberately mismatched property. Verify that the helper does not alter the container, the step fails, the next pipeline step is skipped and an independent readback is unchanged. Do not use a new or unknown container name, and do not include unrelated desired changes.

Create named local PE administrators

Create Local Users (PE), identifier CreateLocalUsersPe, is an administrator-only Scripts 1.x action and pipeline step. It creates Prism accounts, not Orchestrator, AHV, CVM or iDRAC accounts. Each new user receives ROLE_CLUSTER_ADMIN and ROLE_USER_ADMIN; use these privileges only for

approved individual administrators. The wizard defaults to eight users; the supported list is 1-50 per cluster.

In Global Config create a PE connection reference authorized to manage users and one distinct new-user reference for every account. Each reference username must exactly match the requested username; each password must be unique and satisfy the target policy. Seven users are valid: eight is only the wizard default, and the supported list is 1-50. Use the approved vault provider and never put new passwords in the configuration.

Establish trusted HTTPS and a PE identity covered by the certificate SAN. Open Advanced Automation > Scripts 1.x > Create Local Users (PE), populate the PE endpoint and intended username/reference pairs, generate YAML, review and save it. Dry Run resolves references and validates structure without contacting PE or creating users; it cannot prove permissions or password acceptance.

```
clusters:
  192.0.2.10:
    name: example-cluster
    pe_credential: pe_user_admin
    local_users:
      - {username: ops.admin01, credential_ref: pe_ops_admin01}
      - {username: ops.admin02, credential_ref: pe_ops_admin02}
      - {username: ops.admin03, credential_ref: pe_ops_admin03}
      - {username: ops.admin04, credential_ref: pe_ops_admin04}
      - {username: ops.admin05, credential_ref: pe_ops_admin05}
      - {username: ops.admin06, credential_ref: pe_ops_admin06}
      - {username: ops.admin07, credential_ref: pe_ops_admin07}
      - {username: ops.admin08, credential_ref: pe_ops_admin08}
```

Run as an administrator and enter exactly RUN CreateLocalUsersPe. For a pipeline, choose Create Local Users (PE) (Script) and the saved file, after initial authentication and baseline configuration. If another sensitive script is present, follow the combined exact confirmation shown by the UI. The step uses its submission-time validated private snapshot, not edits made while earlier steps run.

Inspect every account and both assigned roles in PE. Independently test each intended new user login using a new session. The operation verifies account existence and roles, not every password login or expiry state. Scheduled execution is deliberately unsupported and existing privileged schedule entries are rejected at execution.

Existing accounts with both requested roles are verified and skipped without password changes. Existing non-admin, disabled or case-conflicting accounts stop the run; built-in names and duplicate names/references are rejected. Creation and role assignment are separate requests: previously created accounts remain if later work fails. Inspect target state before any retry; do not assume automatic deletion, elevation or rollback.

The dedicated client uses verified HTTPS with fixed PE v1 user and role GET/POST contracts, bounded responses/timeouts and no proxies or redirects. Target API variants fail closed. This is a community extension and requires environment-specific user/role API and partial-failure UAT before broader rollout.

Diagnose local user provisioning safely

Updated builds report [PE-USERS] stage=... target=... category=... failures. Stages distinguish credential resolution, client initialization, existing-account and role reads, creation, role assignment and verification. Categories separate certificate trust/validity, IP or SAN mismatch, TLS negotiation, network failures, HTTP 401 authentication, HTTP 403 authorization, API incompatibility and unsupported response formats.

The log suppresses passwords, Authorization headers, raw response bodies and upstream exception text. Do not weaken TLS or enable raw HTTP debugging to investigate. Use the stage to determine

whether any write was attempted; absence of a success line does not prove that an account was not created. Inspect PE before retrying. Read docs/pe-local-users.md#failure-diagnostics.

Independently verify the requested accounts

For every requested username confirm existence, explicitly enabled state and both Cluster Admin and User Admin. Their role identifiers are ROLE_CLUSTER_ADMIN and ROLE_USER_ADMIN. A read-only verifier can GET /api/nutanix/v1/users and GET /api/nutanix/v1/users/{username}/roles over verified HTTPS; it must not POST, reset passwords or assign roles.

For a seven-account report, expected success is PASS=7, FAIL=0, UNKNOWN=0. The standalone report conventions are exit 0 for all passed, 1 for observed failure without Unknown rows and 2 for incomplete collection, invalid invocation or cancellation. A missing enabled flag, ambiguous duplicate username or unsupported response must remain Unknown rather than assumed enabled. These report codes do not redefine pipeline exit semantics.

Use hidden password entry for the existing verification account and do not place credentials in commands or reports. Account-state/role reads do not test each user password, interactive login or expiry state. Verify those separately. For the site-specific seven-user script and complete commands, use the accompanying read-only verification operator guide; public examples remain generic.

CVM and Dell iDRAC credential rotation

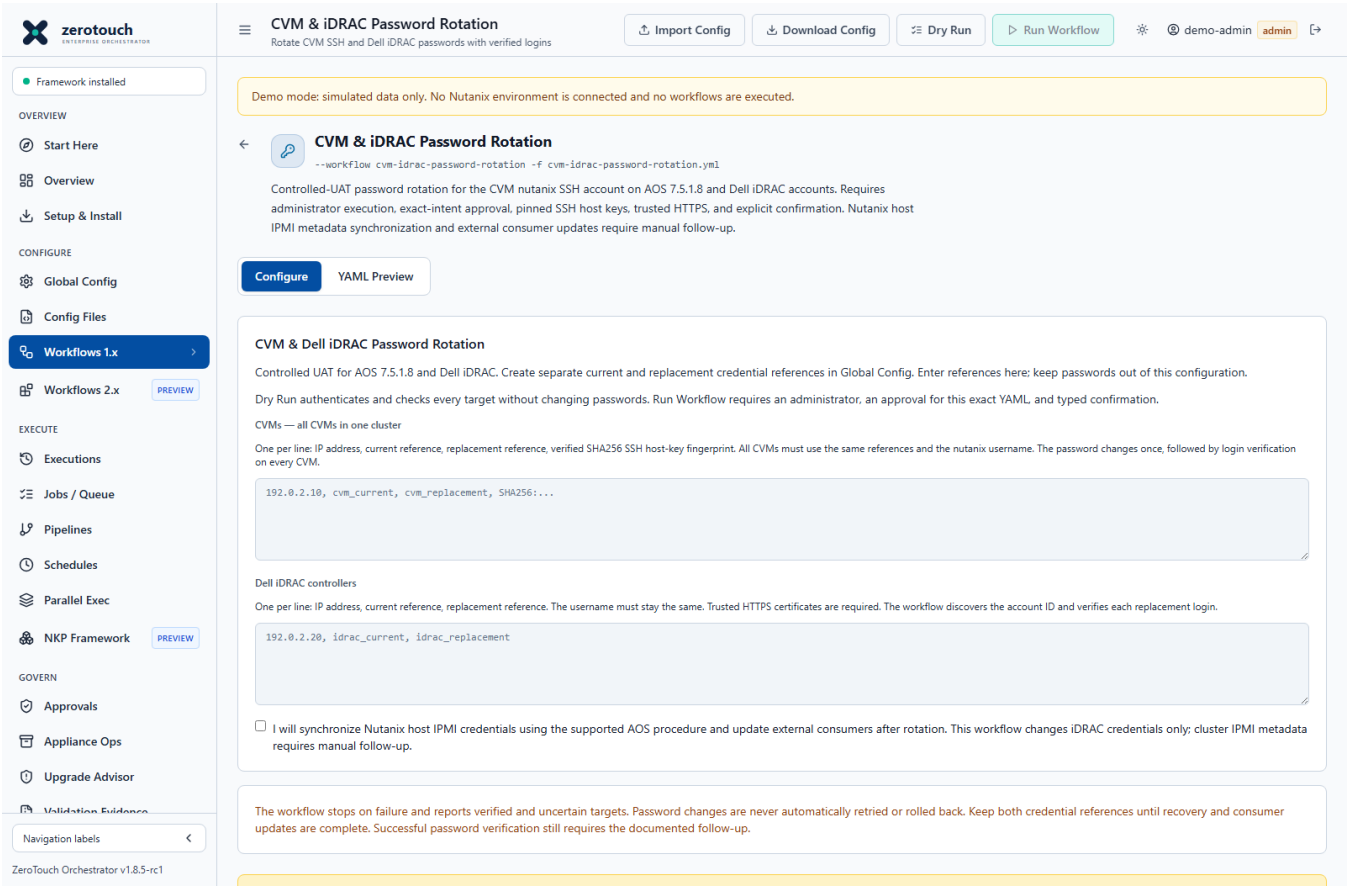


Figure 34. Controlled UAT credential rotation workflow. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

This Orchestrator-managed workflow targets the CVM nutanix SSH account on AOS **7.5.1.8** and existing **Dell iDRAC** local accounts. Implementation and mocked verification are available; live validation on those versions is still required before production use. Prism admin and AHV passwords are outside scope. The AOS version is operator-declared in the intent; installed-version discovery is not performed. Confirm the actual cluster version before approval, and keep referenced credential values unchanged between approval and execution.

Prepare and run

1. In **Global Config > Credentials**, create distinct current and replacement references. Keep the username unchanged; CVM references must use nutanix. Preserve both references until rotation, recovery, and consumer updates finish.
2. Open **Workflows > CVM & iDRAC Password Rotation**. Enter every CVM in one cluster, sharing the same credential references. Supply each CVM's SHA256 SSH host-key fingerprint obtained through a trusted channel. An unauthenticated network key scan alone does not establish identity. Inventory must match svmips before changes are allowed.
3. Enter iDRAC addresses and credential references. The service must trust their HTTPS certificates, including IP address identity. Configure the service trust store appropriately; TLS verification cannot be disabled in this workflow. HTTP proxies and redirects are disabled for these requests.
4. Acknowledge the manual IPMI metadata follow-up when selecting iDRACs. Run **Dry Run** to authenticate and check targets without mutations.
5. Request approval for the exact generated YAML through the existing approval process. Another administrator must approve it. Run as an administrator and type ROTATE CVM AND IDRAC PASSWORDS. Each approval permits one submitted mutation job; general approval settings cannot disable this requirement.
6. Review per-target results. Update connection-profile references and external consumers only after verified results.
7. **Synchronize the corresponding Nutanix host IPMI credentials through the supported procedure for your AOS release.** This workflow changes iDRAC passwords but does not update that cluster metadata or verify cluster power actions. Complete this follow-up before relying on power operations.

Dry Run requires an administrator but no mutation approval. It performs online, read-only authentication checks. Failed authentication can count toward remote account lockout policy.

Behaviour and recovery

All selected targets pass read-only checks before any password changes. The CVM password changes **once**, using interactive sudo passwd nutanix. Replacement password SSH authentication is checked on every CVM, allowing a bounded cluster synchronization interval. iDRAC accounts are discovered by username, changed sequentially through Redfish, and verified by a fresh authenticated account read. The workflow waits ten seconds for CVM synchronization, then attempts one new password login per CVM. It does not repeatedly retry authentication failures.

The configured legacy ZTF Python environment supplies SSH support. The bundled ZTF v1.5.2 requirements already include Paramiko; no runtime dependencies were added. A runtime without SSH support fails CVM preflight before mutations. Passwords travel to the SSH helper through stdin, never shell commands, process arguments, configuration files, or job logs. Remote output and transport exception text are excluded from retained logs.

Success means selected passwords changed and replacement logins were verified. **It does not mean IPMI metadata or external consumers have been updated.** Diagnostics retain followupRequired and nutanixIpmiMetadataUpdated: false.

Failure stops subsequent changes. verified means replacement authentication worked; change_attempted means the final password state is uncertain. A timeout after a write may leave the new password active. Cancellation is checked between operations; an in-flight change can finish before cancellation takes effect. Attempt and verification events are retained as execution advances. Mutations are never automatically retried or rolled back. Inspect the actual target before requesting another approval. Exclude verified iDRAC targets from recovery runs. Recover a partially synchronized CVM cluster through the vendor-supported procedure before rerunning the cluster-wide workflow.

Vendor references and validation boundary

- Nutanix Security Guide documents sudo passwd nutanix and cluster synchronization. This public guide predates AOS 7.5.1.8; release-specific live validation remains required.
- Dell iDRAC Redfish scripting provides vendor-maintained account management examples. Firmware capabilities and password policy require target-environment validation.
- DMTF Redfish specification defines account password updates. This adapter supports Password PATCH; services requiring a ChangePassword action require a separate verified adapter.

Live Prism Element drift monitoring

Live PE monitoring compares approved DNS, NTP and Pulse settings with fresh read-only API results on every manual, run-now and scheduled check. It does not change Prism settings. Existing Last Applied and Stored Snapshot policies keep their configuration-comparison behavior; they do not poll Prism.

Configure a baseline

1. Add a credential reference in Global Config with a PE account authorized to read the three settings. Use the least privileges supported by your release. Never put a password in the drift policy or baseline.
2. Save a YAML configuration such as this example in Config Files:

```
clusters:
  pe.example.com:
    pe_credential: pe_monitor
    name_servers_list:
      - 192.0.2.53
    ntp_servers_list:
      - ntp.example.com
    enable_pulse: false
```

3. In Drift Detection select the file, choose **Live Prism Element**, and run a check. Review every finding and any collection errors.
4. Add an Automation policy. Select **Live Prism Element**, set its UTC schedule, choose notification behaviour and save it enabled. Use **Drift or unknown** to receive alerts for unavailable settings as well as differences. Notifications use the existing configured webhook; they require that webhook to be set up.
5. Use the pencil button (**Edit schedule**) to change the name, file, workflow scope, baseline, schedule, notification mode or enabled state. Saving updates the same policy and re-registers its schedule. Run it now to verify changes.

An explicit HTTPS endpoint with a custom port is also accepted as the cluster key. Bare hosts/IPs use HTTPS port 9440. At most 16 clusters are accepted per baseline. Every cluster needs a credential reference and at least one monitored setting. Omitted settings are not monitored; an empty DNS/NTP list explicitly requests no servers. Other configuration fields are neither compared nor saved in live drift results. DNS/NTP order, duplicates, hostname case and trailing dots are normalized. This is settings drift, not DNS/NTP service-health testing.

Trust and API compatibility

TLS certificate and hostname verification are mandatory. Install your PE CA in the appliance/container trust store, or mount a PEM CA bundle and configure Python's SSL_CERT_FILE. Include the certificate chain and connect using a name or IP present in the certificate SAN. There is no insecure TLS bypass.

The collector uses only GET requests to the installed ZTF legacy PE contracts:

- /PrismGateway/services/rest/v1/cluster/name_servers (JSON string array)
- /PrismGateway/services/rest/v1/cluster/ntp_servers (JSON string array)
- /PrismGateway/services/rest/v1/pulse (enable boolean)

Redirects and environment HTTP proxies are disabled to prevent credential forwarding. Requests have a 10-second timeout and 1 MiB response limit. Credentials are resolved afresh using the existing Global Config reference lookup. No raw response bodies or credentials are retained in drift findings. Admin and operator roles can collect and edit policies; viewers can read results.

Connection, TLS, authentication, unsupported response and parse failures produce **Unknown** findings, not Matched. Confirmed differences remain **Drifted** even when another setting is unknown; inspect the Unknown count and collection errors. Each run records collection-start time, completion time and target endpoints. Schedules persist through application restarts; missed downtime runs are not a continuous-monitoring guarantee. No Prism Central collection or automatic remediation is included.

Interpret live connection diagnostics

A failed live read remains Unknown, not Matched. TLS certificate trust failed means the approved issuing CA chain is missing or untrusted. TLS hostname/IP mismatch means the chosen endpoint is not covered by the certificate SAN; adding a CA alone cannot fix that. Certificate validity failures require checking certificate dates and the appliance clock.

TLS trust configuration could not be loaded indicates an unreadable or invalid PEM bundle. TLS handshake failed indicates protocol/port or TLS compatibility problems. DNS resolution failed, network timeout, connection refused and network unreachable require checks from the container network context. Other network connection failures remain separately classified. HTTP status failures are distinct from transport failures.

Repair the reported issue and rerun: an initial trust failure can mask a later SAN mismatch. Errors are sanitized and exclude credentials, raw exceptions, certificate contents and local paths. Do not disable verification or replace Unknown with a matched baseline to suppress alerts.

Acceptance test

On an authorized test cluster, establish a matched baseline, change one monitored setting in Prism, wait for a scheduled run and verify the specific difference. Restore the approved setting and verify a later fresh run match. Test an unreachable endpoint or insufficient permissions and verify Unknown. Restart

the appliance and confirm the enabled policy runs again. Validate these legacy API response contracts on your installed AOS version before operational use.

Automated tests cover collection contracts, error handling, scheduling callbacks, policy persistence/editing and UI flows using test doubles. They do not establish live AOS compatibility or vendor support.

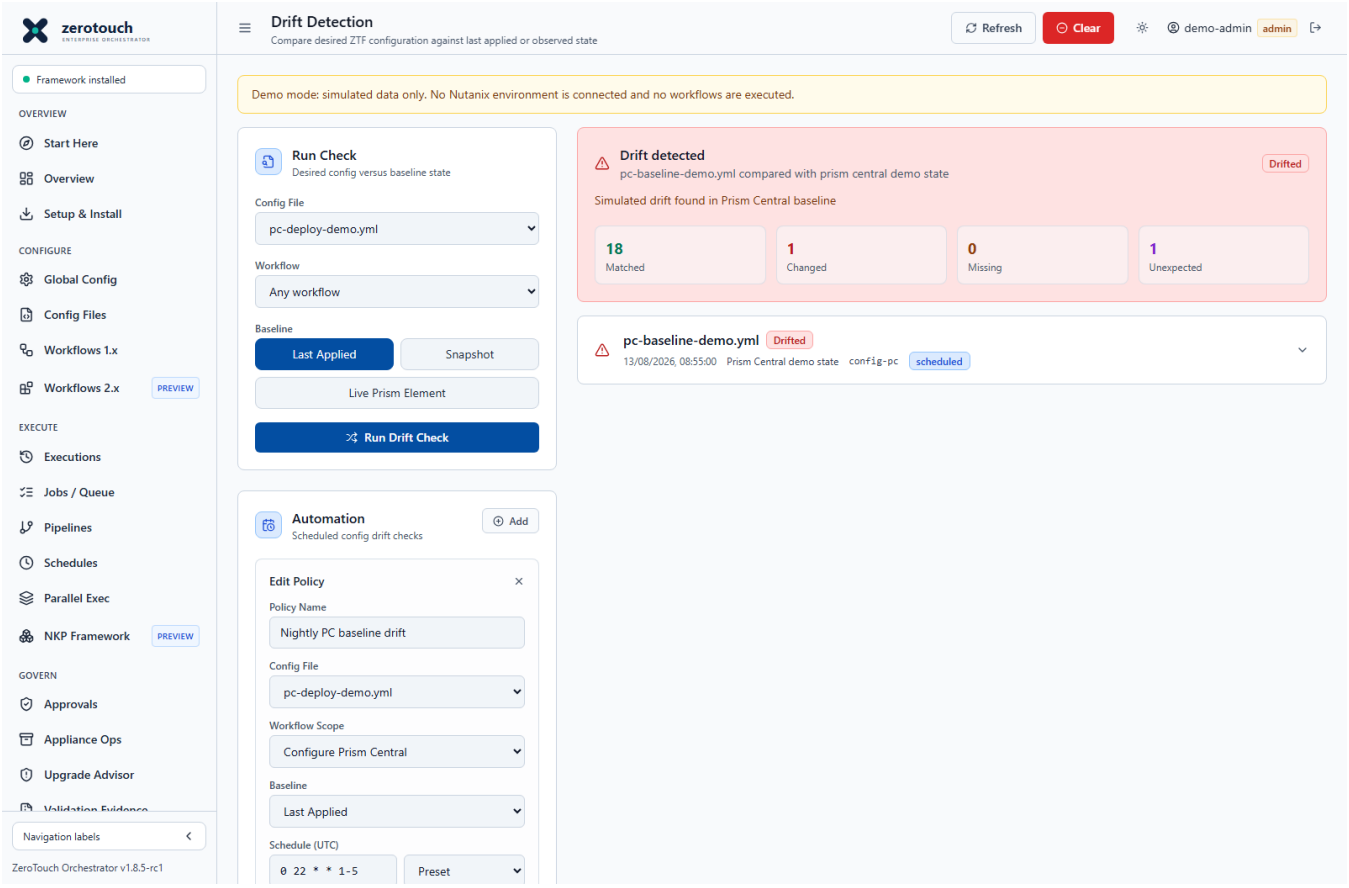


Figure 35. Editing a persisted drift schedule. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Establish persistent certificate trust

Use a CA-issued PE certificate with a DNS name or IP included in its Subject Alternative Name. Obtain the signing CA chain from your PKI administrator through a trusted channel. A certificate downloaded from the network must not be trusted solely because the endpoint returned it.

1. From a trusted browser open the intended PE endpoint, inspect the certificate and export the certificate chain in PEM or Base-64 form. Export certificates only, never private keys.
2. Verify SHA-256 fingerprints, issuer, validity and SAN through an independent trusted source. For a self-signed lab certificate, explicitly approve the trust anchor and ensure it identifies the exact endpoint.
3. Save the approved CA chain on the appliance host under /opt/ztf-orchestrator/certs. Create a dedicated combined PEM bundle containing the application runtime existing CA bundle plus the approved private CA certificates. Do not replace all public trust with a single PE certificate.
4. Merge the read-only combined-bundle mount and SSL_CERT_FILE environment entry into the host Compose service actually used for startup and updates. On the appliance the helper

explicitly uses /opt/ztf-orchestrator/appliance/docker-compose.appliance.yml; an unreferenced override file is not sufficient.

```
services:
  ztf-orchestrator:
    environment:
      SSL_CERT_FILE: /etc/ztf/pe-ca-bundle.pem
    volumes:
      - /opt/ztf-orchestrator/certs/pe-ca-bundle.pem:/etc/ztf/pe-ca-bundle.pem:ro
```

5. Merge this fragment into the existing service definition and preserve its environment and persistent volumes. Back up that file and .env securely first. Confirm Jobs / Queue and pipelines are idle, validate Compose with config --quiet, then recreate only the app using the explicit commands below. Keep PostgreSQL and data volumes intact.

On the appliance host, validate the effective configuration and recreate only the pinned application image:

```
sudo docker compose --env-file /opt/ztf-orchestrator/.env \
  -f /opt/ztf-orchestrator/appliance/docker-compose.appliance.yml config --quiet
sudo docker compose --env-file /opt/ztf-orchestrator/.env \
  -f /opt/ztf-orchestrator/appliance/docker-compose.appliance.yml \
  up -d --no-deps --no-build --pull never --force-recreate ztf-orchestrator
```

6. Test from the application container with Python ssl.create_default_context and server_hostname equal to the endpoint identity. Then run Live Prism Element drift monitoring. A TLS connection test does not prove account permission or API schema compatibility.

Keep the PEM bundle on the appliance host and bind-mount it read-only; ensure the runtime user can read it. Use the same explicit Compose/environment files for maintenance and updates, and recheck the mount after a Compose-file upgrade. Include existing public and private roots needed by other clients. Editing a running container trust file alone is not persistent. Follow docs/pe-certificate-trust.md for export, fingerprint review, activation and rollback.

Trust scope matters: live drift, PE local-user creation and Upgrade Advisor retain verified-TLS clients. The legacy ZTF Configure Cluster transport is not replaced by the password-readback fix; secured=True selects HTTPS, not a newly guaranteed trust implementation. Keep legacy HTTP debug logging disabled during password operations because upstream payload logging is outside sanitized verification exception handlers.

7. Retain certificate expiry ownership and renewal procedures. If a SAN does not match the IP, use the correct resolvable DNS name or replace the certificate; do not disable verification.

Validate the schedule end to end

1. Confirm an enabled Automation policy, a five-field UTC cron expression and a successful fresh Run now result. Use Drift or unknown notifications when failed collection also needs attention, and verify the configured webhook separately.
2. At an authorized test PE change one monitored setting and wait for the next scheduled run. Confirm a new collection timestamp and the exact difference.
3. Restore the approved setting and confirm a later run returns Matched. Simulate an unavailable endpoint or insufficient read permission and confirm Unknown rather than Matched.
4. Restart the Orchestrator during maintenance and confirm the enabled policy runs again. Scheduling is not continuous monitoring and does not guarantee replay of missed runs.
5. Use Edit schedule to modify the existing policy rather than creating duplicates. Review the new file, target scope, baseline, cron expression, notification mode and enabled state, then run it again.

Administration security and service operations

Security Administration

ZTF-Orchestrator is designed for trusted internal networks. It should not be directly exposed to the internet.

Core controls:

- Session-based authentication with expiring bearer tokens.
- bcrypt password hashing.
- Role-based protected routes.
- API rate limits on login, execution, and install-sensitive paths.
- Workflow and script allowlists.
- YAML safe parsing.
- Path traversal protections around config file operations.
- Subprocess execution through argument lists rather than shell strings.
- Security headers including CSP-related controls.
- Docker Compose host binding to localhost by default.

Admin hardening checklist:

1. Put nginx or an equivalent reverse proxy with TLS in front of team deployments.
2. Restrict network access to trusted administration networks.
3. Rotate first-run credentials immediately.
4. Use PostgreSQL for durable shared state.
5. Back up database and VM/container state before updates.
6. Keep secrets out of exported docs, screenshots, public issues, release notes, and generated artifacts.
7. Review webhook host restrictions before enabling notifications.
8. Run controlled UAT before representing a workflow as production validated.

Backup Restore and Disaster Recovery

ZTF-Orchestrator stores operational state that must be recoverable:

- Users and sessions.
- Settings and runtime configuration.
- Config files and automatic backups.
- Execution history and job logs.
- Approvals and schedules.
- Validation evidence.
- Audit events.
- PostgreSQL backups.

Minimum DR objectives:

Objective	Admin decision
RPO	How much Orchestrator state loss is acceptable.
RTO	How quickly the admin console must be restored.

Objective	Admin decision
Backup location	Where database, VM/container, and exported evidence backups are stored.
Restore environment	Where the service will be restored if the primary host is unavailable.
Secret rotation	Which credentials must rotate after restore or incident.
Evidence retention	Which evidence exports must survive outside the app.

Use cases:

- Restore the Orchestrator after appliance failure.
- Roll back from a failed update.
- Recover PostgreSQL-backed state after database corruption.
- Rebuild a disconnected deployment from update packages and retained backups.
- Preserve evidence after an infrastructure incident.

Related runbooks:

- docs/runbooks/RB-002-backup-restore.md
- docs/runbooks/RB-003-upgrade-rollback.md
- docs/runbooks/RB-010-database-recovery.md
- docs/governance/DISASTER-RECOVERY.md

Disaster Recovery Cluster Rebuild Use Case

For preparation, capability limits, acceptance gates, and rehearsal evidence, see the Infrastructure Recovery Guide and Recovery Rehearsal Checklist. Recovery of the Orchestrator itself is covered separately in Disaster Recovery of ZTF-Orchestrator.

ZTF-Orchestrator can be part of a broader disaster recovery process for rebuilding affected Nutanix infrastructure from known-good baselines.

Appropriate positioning:

ZTF-Orchestrator automates the infrastructure rebuild and validation phase of a broader disaster recovery process. It does not perform malware remediation, forensic investigation, backup recovery, or ransomware detection.

Example scenario:

1. A cluster is affected by ransomware, malware, severe misconfiguration, or a destructive operational event.
2. Security and infrastructure teams isolate the environment, preserve evidence, and decide that the cluster should be rebuilt rather than trusted in place.
3. The recovery lead approves a predefined rebuild process and selects a reviewed ZTF-Orchestrator workflow or pipeline.
4. ZTF-Orchestrator uses approved configuration to coordinate imaging, reinstallation, baseline configuration, and validation where supported.
5. Supported workflow mappings reapply reviewed settings such as DNS, NTP, VM networks, storage containers, and Prism registration. Operators complete manual monitoring, certificate, and hardening controls where executable mappings are unavailable; evidence-only steps do not apply those controls.

6. Validation gates confirm cluster health, API reachability, time sync, expected services, and configuration posture.
7. Backup, DR, or application teams restore workloads only after the rebuilt infrastructure is accepted.

Recommended workflow pattern:

Phase	Human process	ZTF-Orchestrator capability
Declare and contain	Incident commander freezes changes and confirms containment.	Disable schedules and confirm no jobs are running.
Authorize rebuild	Change owner approves rebuild scope and target baseline.	Approval Gate records authorization and binds reviewed config.
Reinstall or reimage	Infrastructure owner confirms hardware, images, and network data.	Cluster Create, Imaging Only, Site Deploy, FCA handoff, or Native Foundation controlled-UAT planning where applicable.
Apply baseline	Platform owner selects known-good config.	Configure Cluster, Post-Foundation Baseline, PE Network Baseline, Monitoring Baseline, Certificate Baseline, and Security Hardening evidence.
Validate	Operators and reviewers confirm readiness.	Dry run, Jobs / Queue, Execution History, Drift Detection, Validation Evidence.
Handover	Backup or application team starts workload restore.	Evidence bundle exported for change record and recovery handover.

Controls to require:

- Explicit incident and rebuild authorization.
- Known-good configuration source.
- Backup and workload restore owner identified.
- Secrets rotated where compromise is possible.
- Evidence captured before and after rebuild.
- Target-side tasks verified in Prism, Foundation Central, FCA, or NKP tools as applicable.

Backup and restore procedure

Release channel: v1.8.5-rc1 controlled UAT; v1.8.4 remains Latest stable. Match the active build and data-store profile to the backup procedure.

Metadata

Field	Value
Runbook ID	RB-002
Title	Backup and restore
Version	1.0
Owner	Platform operations

Field	Value
Approver	Platform lead
Classification	Internal operational procedure
Status	Reference procedure requiring local authorization

Purpose

Create and restore ZTF-Orchestrator operational state backups with explicit approval, evidence capture, and post-restore validation.

Scope

Covers PostgreSQL logical backups exposed by **Settings > Storage** and the maintenance backup APIs. File-backed backup handling is limited to copying ZTF_DATA_DIR through an approved host procedure while the service is stopped.

Preconditions

- App version, storage backend, and deployment mode are known.
- No workflow, NKP, schedule, pipeline, or restore job is running.
- Restore has explicit approval and a selected backup file.

Required Role RBAC

App admin for backup/restore actions. Host or database administrator for direct database recovery.

Required Inputs

- Backup filename or destination.
- Restore approval ID for restore operations.
- Expected restore point.
- Current change ticket.

Dependencies

- PostgreSQL mode requires pg_dump and pg_restore in the application container or host path.
- ZTF_DATA_DIR/backups/postgres/ must be writable.
- Restore requires healthy database connectivity.

Risk Impact

Backup is low impact. Restore is high impact because it can roll back users, sessions, settings, jobs, approvals, evidence, and audit data.

Procedure

1. Confirm **Jobs / Queue** has no running or cancelling jobs.
2. Open **Settings > Storage**.
3. Confirm storage backend and latest backup posture.

4. For backup, click **Create Backup** and wait for completion.
5. For restore, review the selected backup, obtain approval, type the required confirmation, and start restore.
6. Restart the application after restore so in-memory sessions and workers load restored state.
7. Reopen **Settings > Storage**, **Jobs / Queue**, **Approvals**, and **Audit Log**.

Validation

- Backup file is listed and downloadable.
- Restore creates a safety backup first.
- App returns healthy after restart.
- Expected users, jobs, approvals, configs, evidence, and audit records are present for the selected restore point.

Expected Result

A backup is available or the application state is restored to the approved point without running jobs or orphaned operational state.

Failure Conditions

- Backup creation fails.
- Restore is attempted while jobs are running or cancelling.
- Restore completes but application health fails.
- Restored data does not match the approved restore point.

Recovery Rollback

If restore fails, stop the service and escalate to RB-010. Use the safety backup created by the restore flow when a rollback is required.

Evidence To Capture

- Operator and approver.
- Backup filename and checksum if exported.
- Restore approval ID.
- Health check after restoring.
- Storage page screenshot or exported backup inventory.
- Audit event for backup or restore.

Audit Requirements

Restore requires a change ticket, approval record, and post-restore validation record. Backups created before high-risk work must be referenced by later job evidence.

Escalation

Escalate before retrying restore if database errors, missing state, or running job conflicts appear.

References

- PostgreSQL Backend

- PostgreSQL Backup Restore Drill
- Runbook Index

Evidence Mapping

Evidence	Source	Required
Backup file	Settings/API	Yes
Restore approval	Approvals/change ticket	Restore only
Safety backup	Settings/API	Restore only
Audit event	Audit Log	Yes

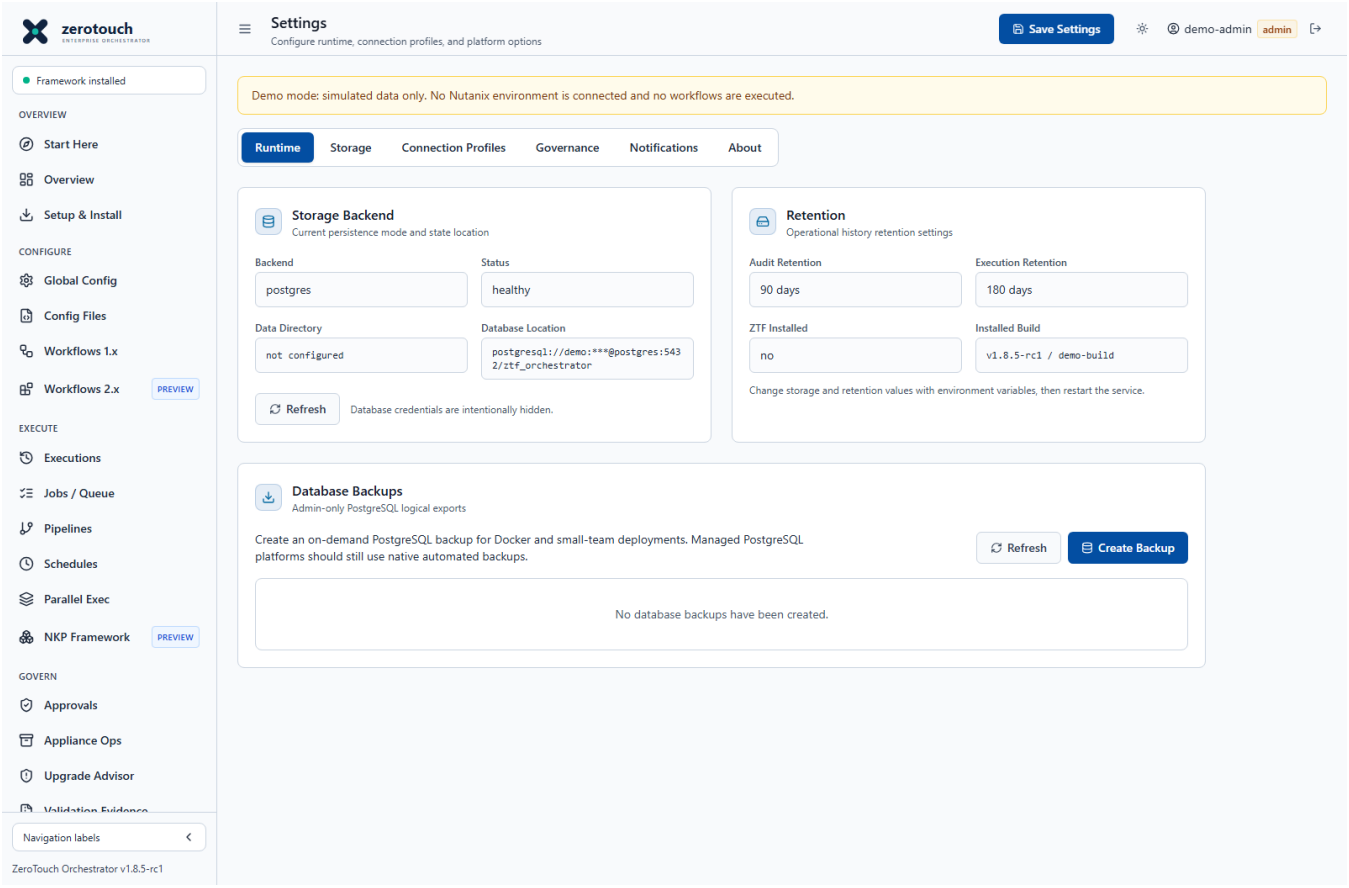


Figure 36. PostgreSQL backup and restore controls. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Notifications governance and build identity

Review these screens when preparing a team deployment. Connection profiles supply reusable defaults; they neither create accounts nor prove credential validity. Preserve approval binding and notification restrictions. Settings > About and the actual image/commit distinguish patch builds; historical screenshot footers are not active-build evidence.

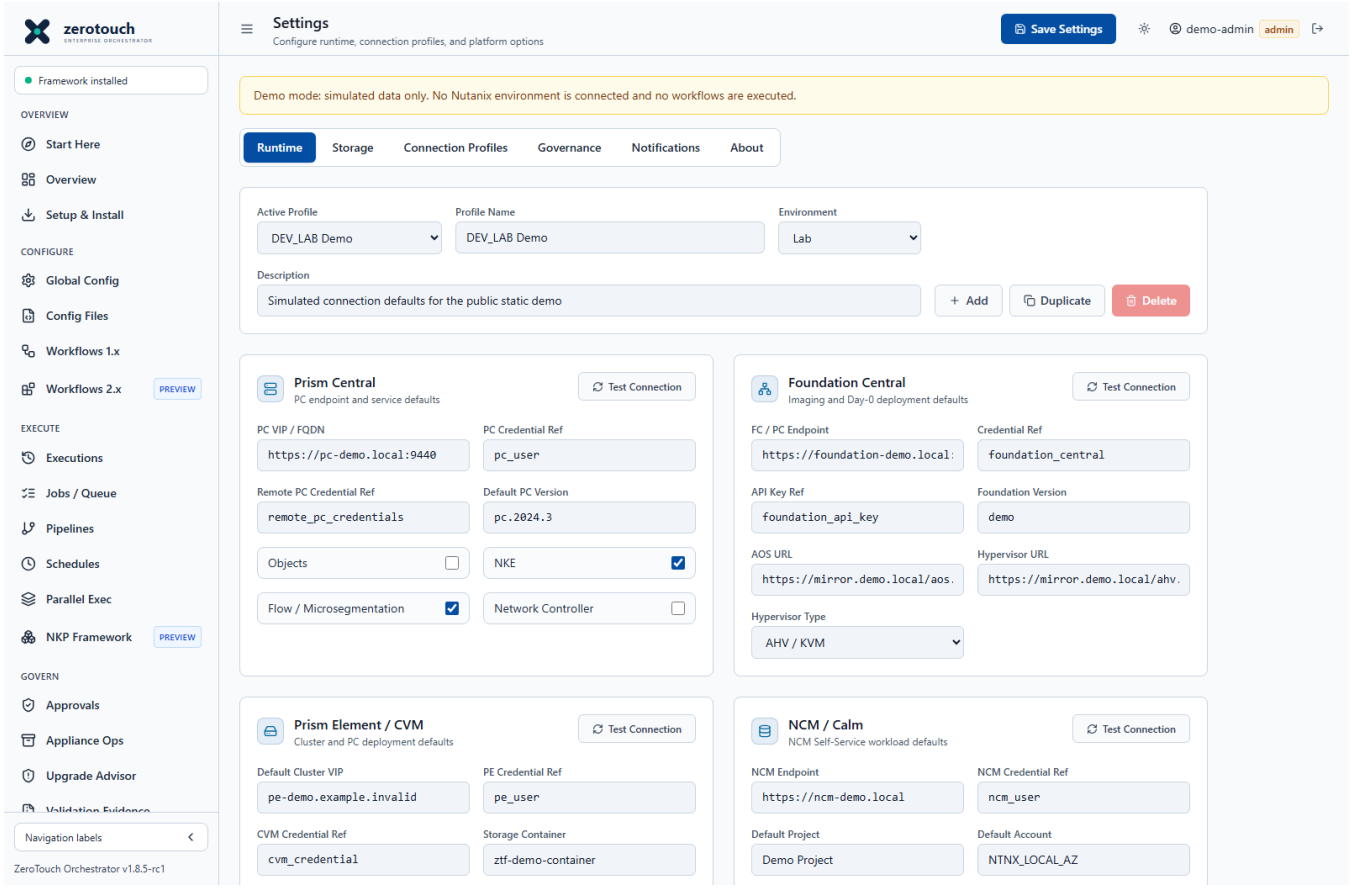


Figure 37. Named connection profiles. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Select or create a named profile for the intended environment. Keep the profile name, target endpoints and credential references distinct for each site. Copying a profile copies defaults, not target accounts.

Review Prism Central, Foundation, PE/CVM, directory, IPAM and registry sections relevant to the operation. A TCP connection check establishes reachability only; it does not validate authenticated API access, TLS trust or deployment readiness.

Save, regenerate the relevant workflow YAML, and inspect which defaults were applied. Switching a profile does not retroactively rewrite every saved configuration.

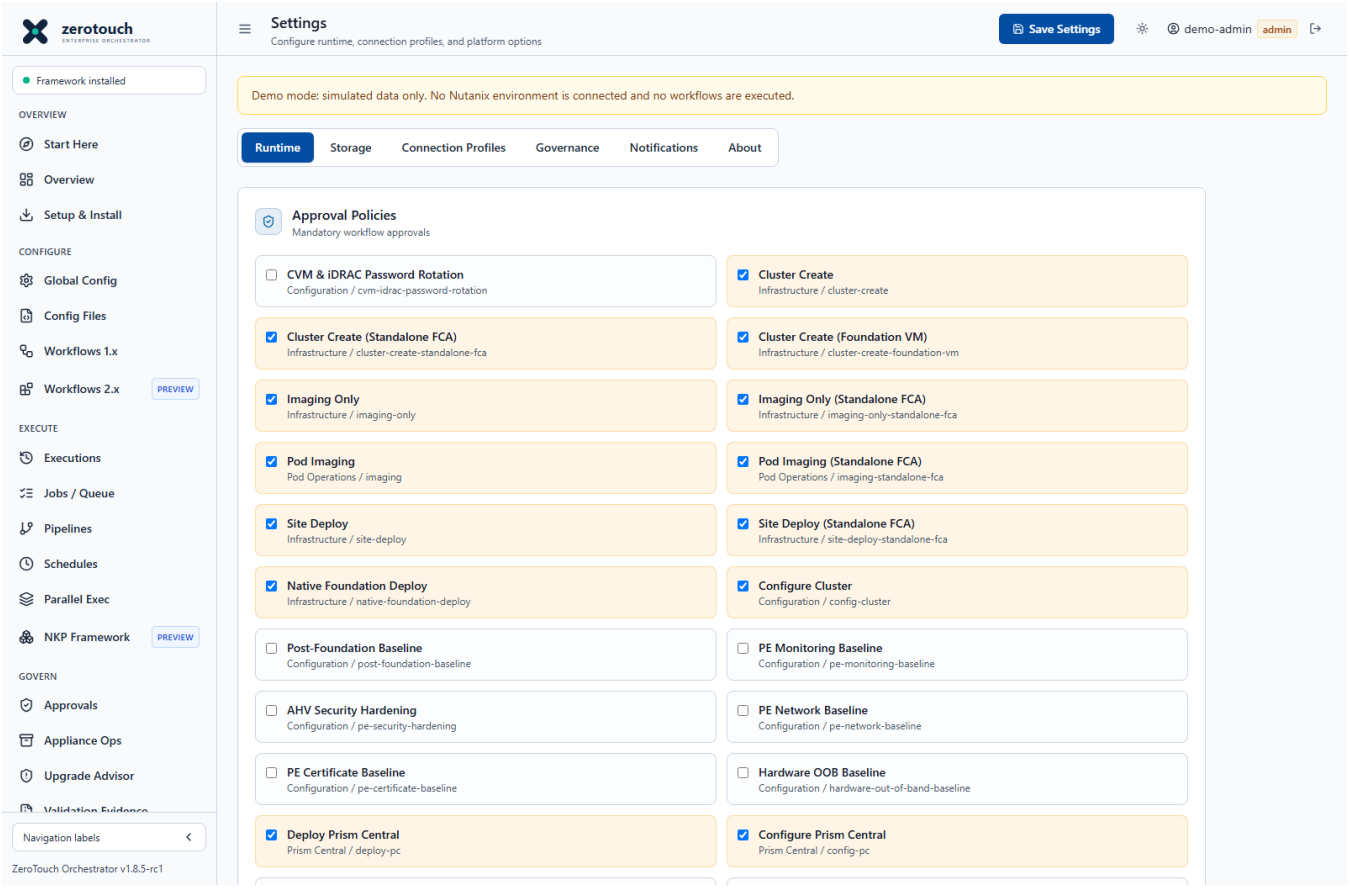


Figure 38. Governance and approval policy. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Review each selected workflow that requires an approval and retain the change owner and approver responsibilities. High-risk actions can have mandatory server-side controls independent of the general checkbox policy.

Use another administrator for review where separation of duties is required. Ensure the approved YAML is exactly the one submitted; changed targets or credential references require a fresh review.

Test the approved direct-job path and an unauthorized attempt. If pipelines or schedules cannot carry the required approval binding, use the direct path rather than weakening the policy.

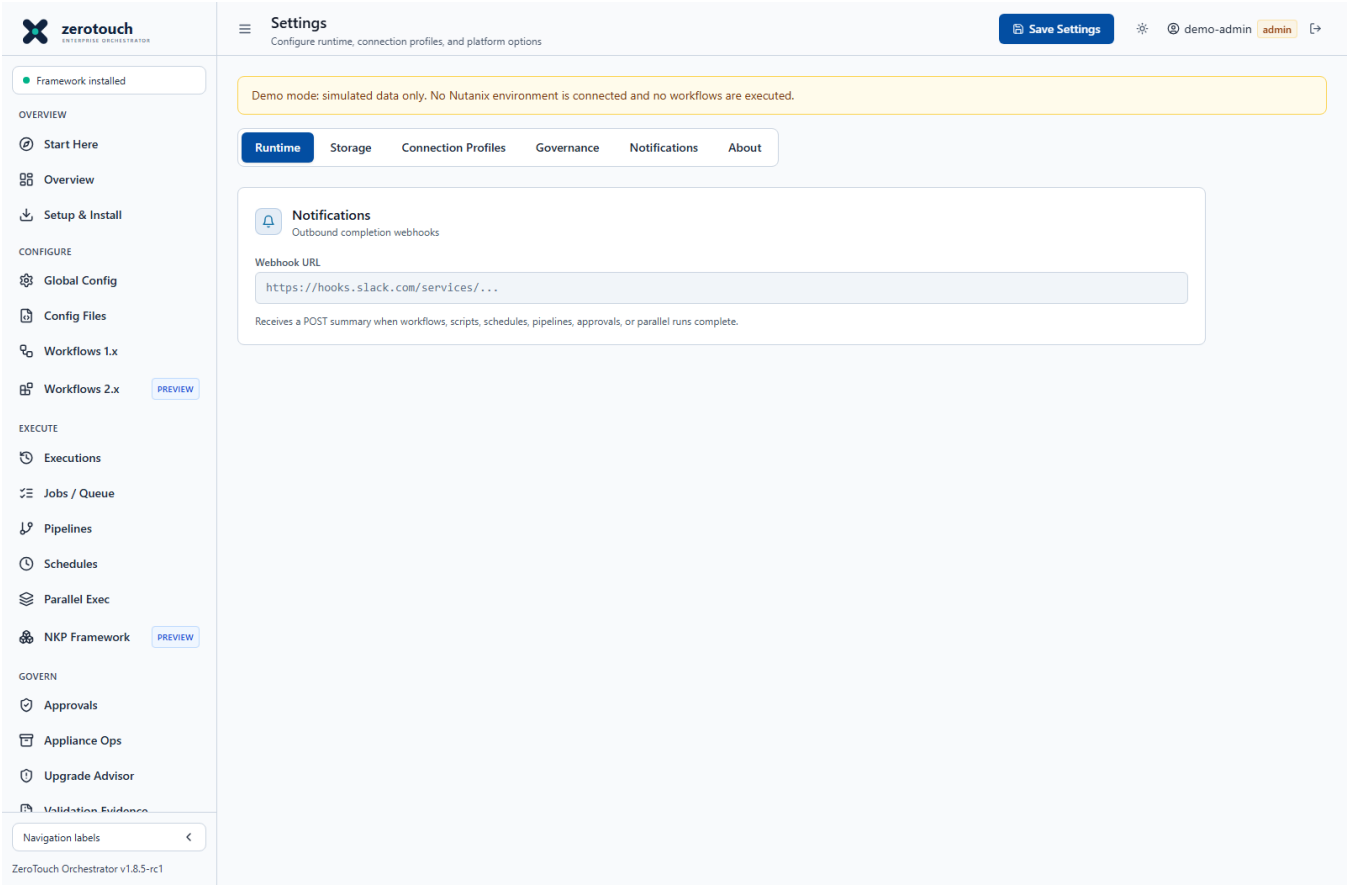


Figure 39. Notification configuration. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Enter only an approved webhook destination and verify the application network can reach it. Treat webhook URLs containing tokens as secrets. Restrict the remote endpoint to the intended receiving service.

Verify delivery with an authorized test operation, then confirm the receiving team sees the expected identifier and result. Do not include raw credentials or sensitive configuration in notification content.

For drift schedules select the desired notification condition separately. A notification setting does not itself schedule checks, and a silent webhook does not prove that no drift occurred.

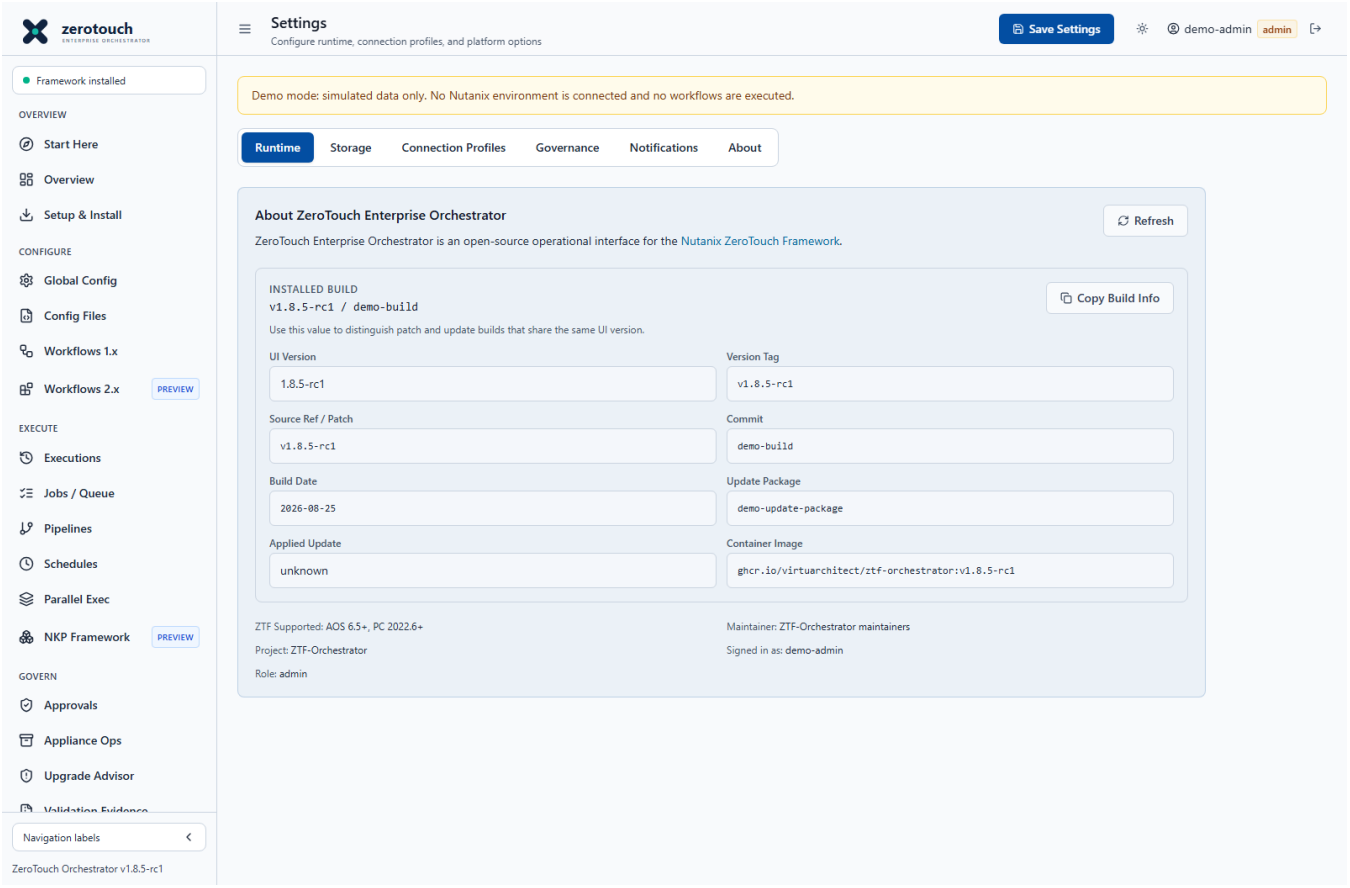


Figure 40. Installed build identity. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Refresh and record the full Installed Build, source commit, image and update-package fields when available. The screenshot is deliberately marked demo and is not the running appliance identity.

Compare the actual values after an update with the manifest and running container image. A successful Git commit or imported package does not prove that the container was recreated from that image.

Copy sanitized build details into the support or change record. Report missing identity fields as unknown instead of inferring a build from the common release footer.

Keep Docker rebuild identity aligned

Local Compose source rebuilds use the versioned ztf-orchestrator:v1.8.5-rc1 image and forward release/build identity to Docker. A guard rejects stale version overrides that disagree with the source release. This does not change the stable GitHub release. Use a clean reviewed checkout, preserve unrelated local work and review effective image pins before rebuilding.

```
docker compose config --images
docker compose build ztf-orchestrator
docker compose up -d --no-deps --no-build ztf-orchestrator
docker inspect ztf-orchestrator --format '{{.Config.Image}}'
```

These commands describe a source checkout, not the appliance host update path. A local override can intentionally pin a reviewed image and prevent a new build from becoming active. Inspect that pin before cutover; do not delete it blindly. Confirm no active jobs, retain backups and compare exact image/health identity after recreation. See docs/docker-build-identity.md.

Appliance update and rollback procedure

1. Identify the installed build, host Compose profile, volumes and exact intended update image. Preserve .env, certificate mounts and local runtime settings.
2. Download the approved version-controlled bundle and verify its manifest and SHA256SUMS. An image tar, update ZIP and complete QCOW2 appliance are separate artifacts.
3. Budget free space for the retained image tar, unpacked image layers and a complete pre-update PostgreSQL/data backup pair. Imported appliance-update-packages inside the data volume are included in its snapshot and can make backups large. Pause schedules and confirm no active or cancelling jobs or pipelines. Retain a verified complete backup pair and rollback image before targeted retention cleanup.
4. Import the package in Appliance Ops > Updates, review target and version, verify and stage it. Importing or staging does not update the running container by itself.
5. On the appliance host run the approved apply-update-request.sh helper with the correct request-file and image-tar paths. Keep backups required. If a backup fails for lack of space, stop; retain the latest verified complete pair and the new image tar, free only specifically approved obsolete artifacts, then retry. Never prune volumes or bypass the backup merely to finish an update.
6. Verify the exact running image tag and image ID, /health installed.commit and installed.containerImage, Settings > About, retained configurations, users, approvals, schedules and backups. Historical appliedUpdate/sourceRef metadata can describe an earlier request; it is not a substitute for the active installed build.
7. Record helper output and checksums. If acceptance fails, use the approved rollback runbook and preserved backups; do not improvise a database rollback while jobs are running.

Apply and accept a consolidated RC appliance bundle

For the consolidated v1.8.5-rc1-4a01072 package, intermediate RC patch installations are not required. The application image is ghcr.io/virtuarchitect/ztf-orchestrator:v1.8.5-rc1-4a01072; source commit 4a010727d5db3496d17a52e4ce681818dcc56200. It includes merged earlier fixes, PE local-user diagnostics, Docker identity and Start Here guidance, and excludes unfinished deployment-engine work. Verify the transferred manifest and SHA256SUMS; do not relabel an old request without matching approved metadata.

```
df -h / /opt/ztf-orchestrator
df -i /opt/ztf-orchestrator
sudo docker ps --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}'
sudo du -sh /opt/ztf-orchestrator/update-artifacts /var/lib/docker /var/lib/containerd
```

Expected: sufficient bytes and inodes for a full new backup pair, retained rollback image and image extraction. containerd space may be separate from Docker system df reporting. Stop when space is insufficient. Never use blanket prune, volume removal or deletion based only on the newest filename; a readable database dump without its completed data snapshot is not a complete pair.

```
sudo env \
  ZTF_UPDATE_REQUEST_FILE=/opt/ztf-orchestrator/appliance_update_request.json \
  ZTF_UPDATE_IMAGE_TAR=/opt/ztf-orchestrator/update-artifacts/ztf-orchestrator-v1.8.5-rc1-4a01072-image.tar \
  /opt/ztf-orchestrator/appliance/scripts/apply-update-request.sh
```

Use the freshly exported staged request path for this appliance; the sample path is not proof that the request exists on a PostgreSQL-backed appliance. Keep pre-update backup requirements enabled. Expected helper results include a completed PostgreSQL backup, data snapshot, image load and service recreation. Backup failure stops the change; import, verify and stage do not activate the image.

```
sudo docker inspect ztf-orchestrator --format '{{.Config.Image}} {{.Image}} {{.State.Health.Status}}'
curl -fsS http://127.0.0.1:5001/health
sudo docker exec ztf-orchestrator pg_dump --version
sudo docker exec ztf-orchestrator pg_restore --version
```

Expected: the exact requested image is running and healthy; /health installed.commit identifies 4a010727d5db3496d17a52e4ce681818dcc56200 and installed.containerImage matches its tag. The general frontend release may still read v1.8.5-rc1; use full installed identity to distinguish patches. PostgreSQL client major 16 must match the server. Use the configured host HTTP port if different from 5001. Verify saved state, trust mounts, schedules and selectors after login, then test Start Here, the seven guidance stages and the post-configuration links without deploying infrastructure.

Acceptance for this fix requires fresh replacement authentication on an authorized disposable target, password SKIP in subsequent files, field-by-field container readback, expected stopping on mismatches and independently verified PE identity in PC. A successful Docker load, healthy container or passed Dry Run does not establish those target outcomes.

Rollback without reversing target changes

Before applying, capture the previous container image tag and ID, .env, service/Compose overrides and trust mounts. Retain that exact local image and a verified paired PostgreSQL/data backup. If acceptance fails, stop new work, restore the captured .env and original overrides, validate Compose configuration and restart through the approved service procedure. Confirm exact old image ID, application health and retained records.

If only the application build failed, do not automatically restore the database or delete volumes. Data recovery requires a separately authorized paired restore while the application is stopped, with a safety backup and subsequent verification. An application rollback does not reverse a PE password, accepted EULA, Pulse update, created user/container or PC registration. Reconcile actual target state and active credentials before resuming automation.

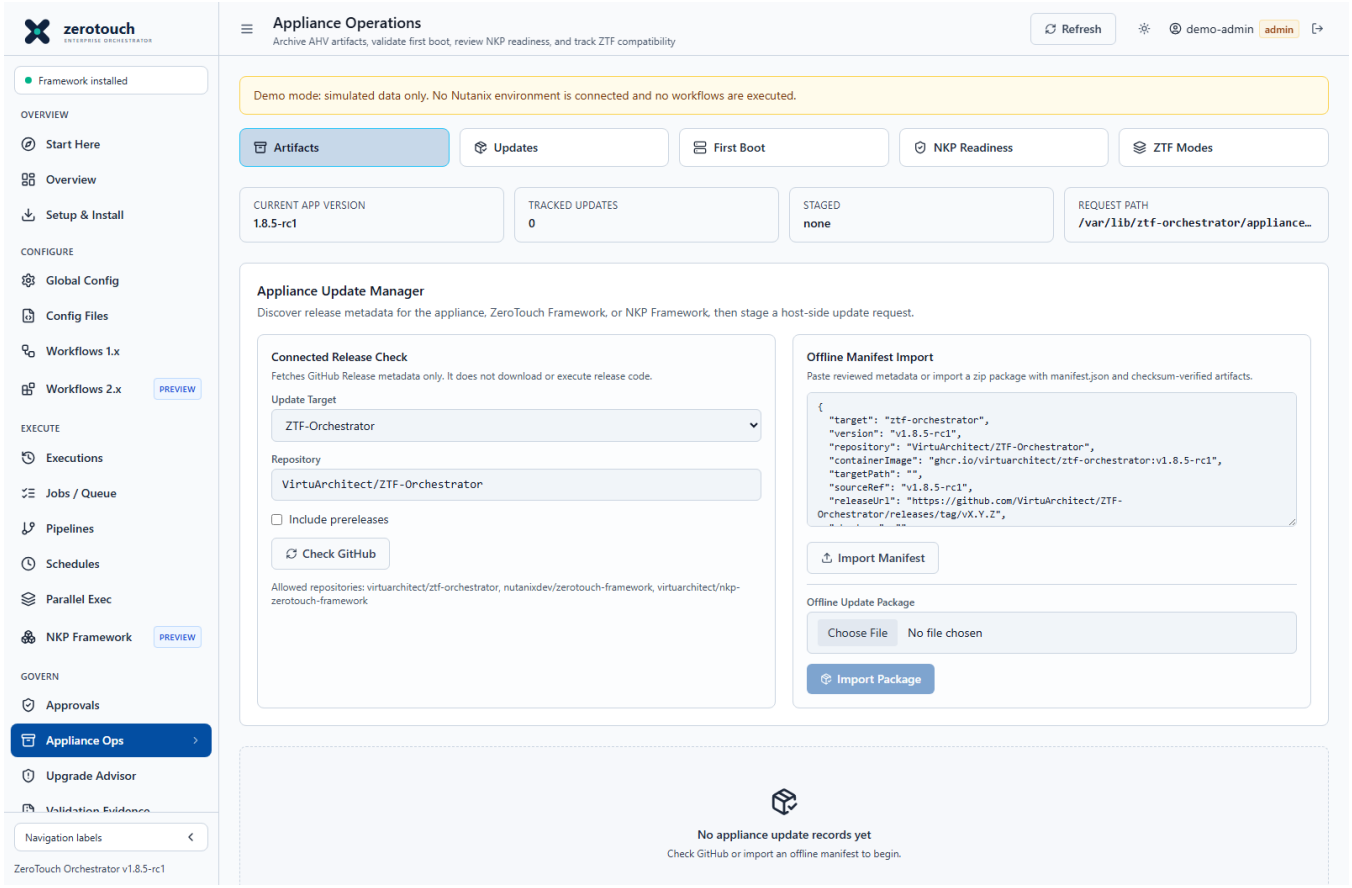


Figure 41. Appliance update discovery and package import. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Connected checking discovers release metadata; it does not download and execute arbitrary release code. For offline updates use the intended package, review the target/version and validate integrity before staging.

Inspect the staged request and have a privileged host administrator apply it through the correct helper. Retain before/after identity and backup evidence. Marking a record applied is not independent proof of the running image.

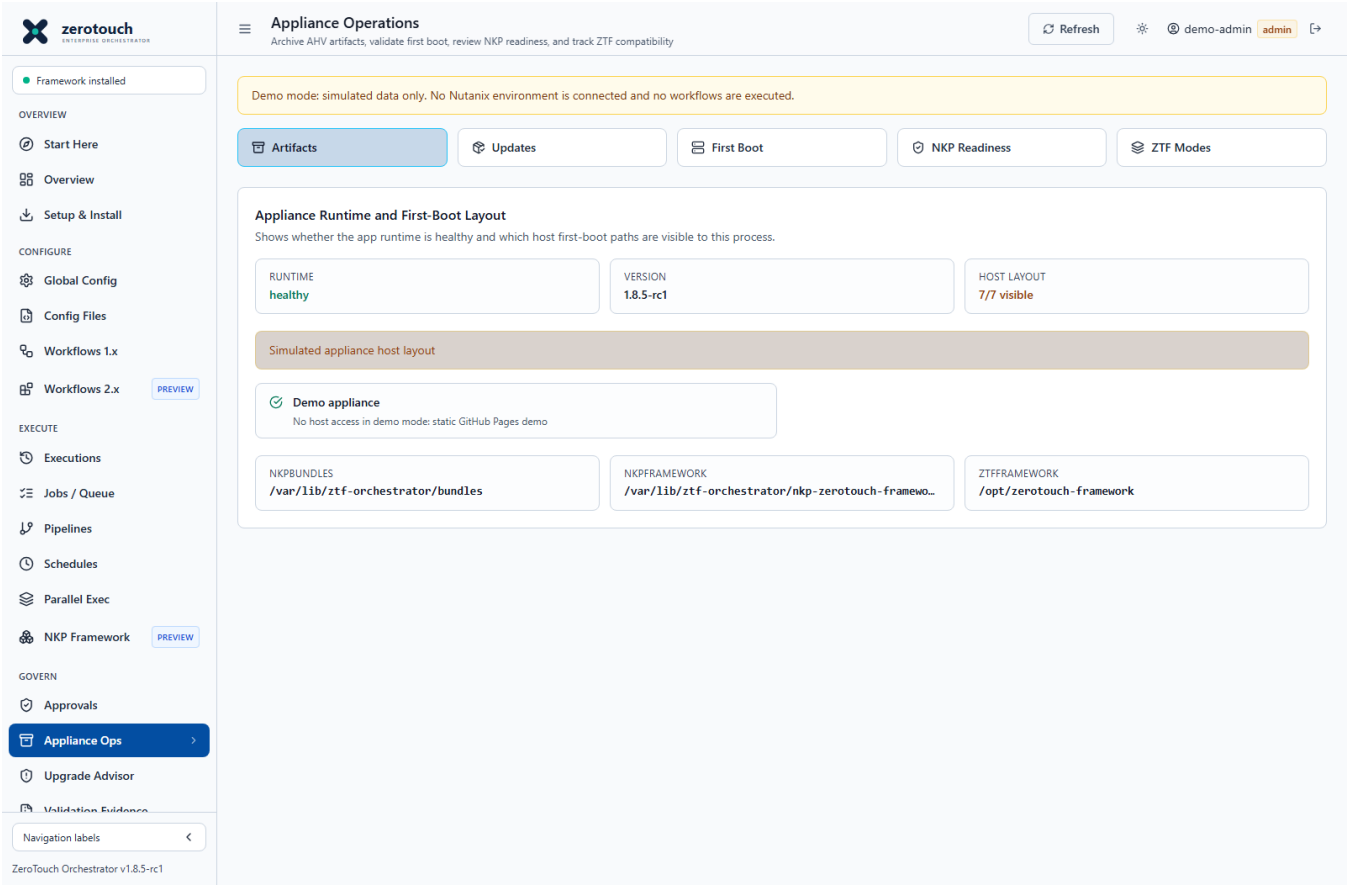


Figure 42. First boot checks. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Review application runtime health separately from host-layout visibility. Paths expected on the VM host may not be visible inside the container; investigate the deployment profile rather than treating all invisible paths as missing software.

Check systemd first-boot logs on the host when bootstrap failed. Confirm persistent directory ownership, Compose environment, database health and framework locations before rerunning setup. Preserve the generated .env because it contains database credentials.

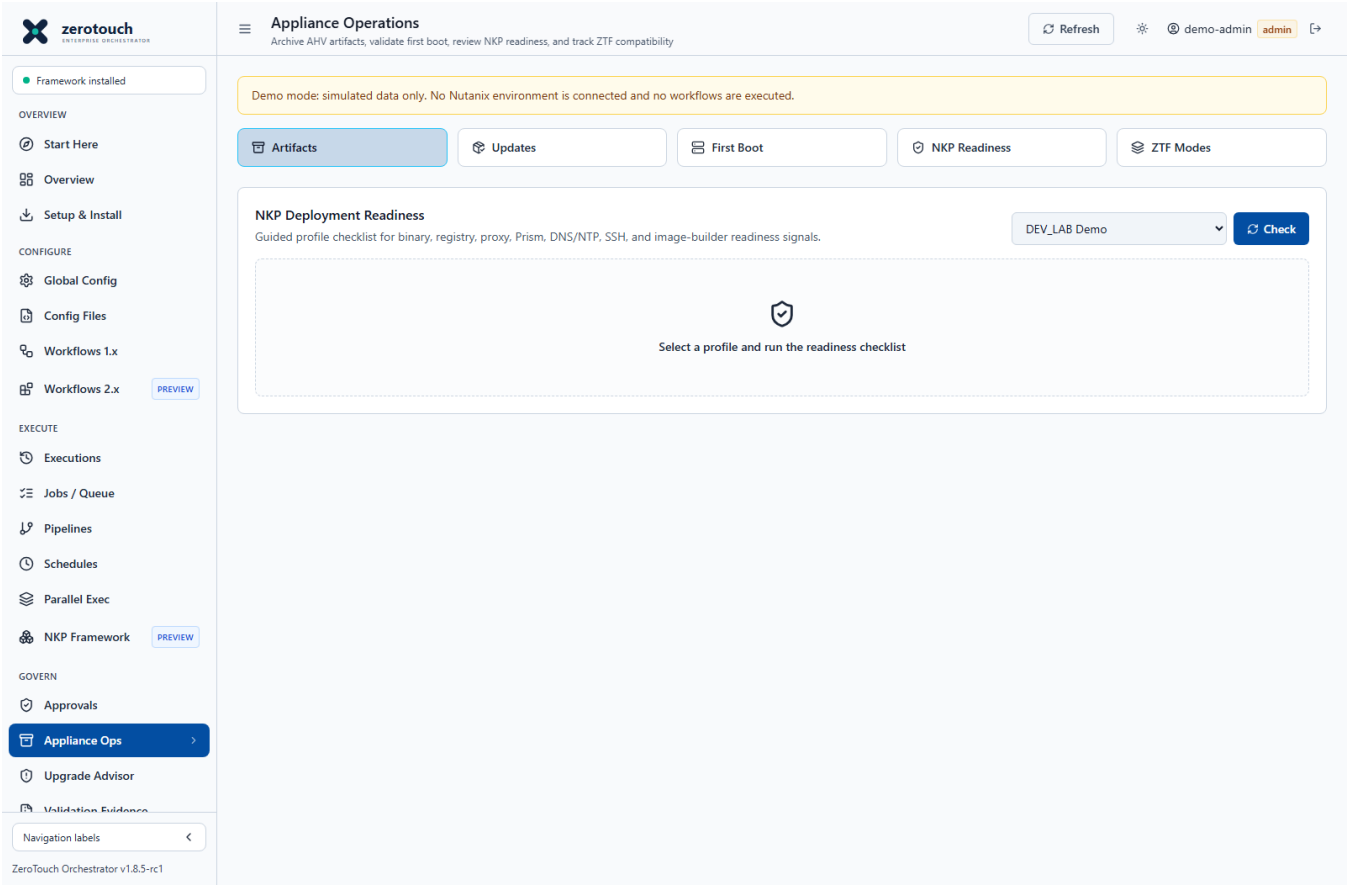


Figure 43. NKP readiness review. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Select the intended saved NKP profile and run its readiness check. Review binary, registry, network, DNS/NTP, image and bundle requirements according to the installed integration.

A passed checklist or profile score is supporting evidence, not proof of deployed Kubernetes workloads. Resolve Unknown and blocked findings and retain the actual CLI/version and target checks for the change record.

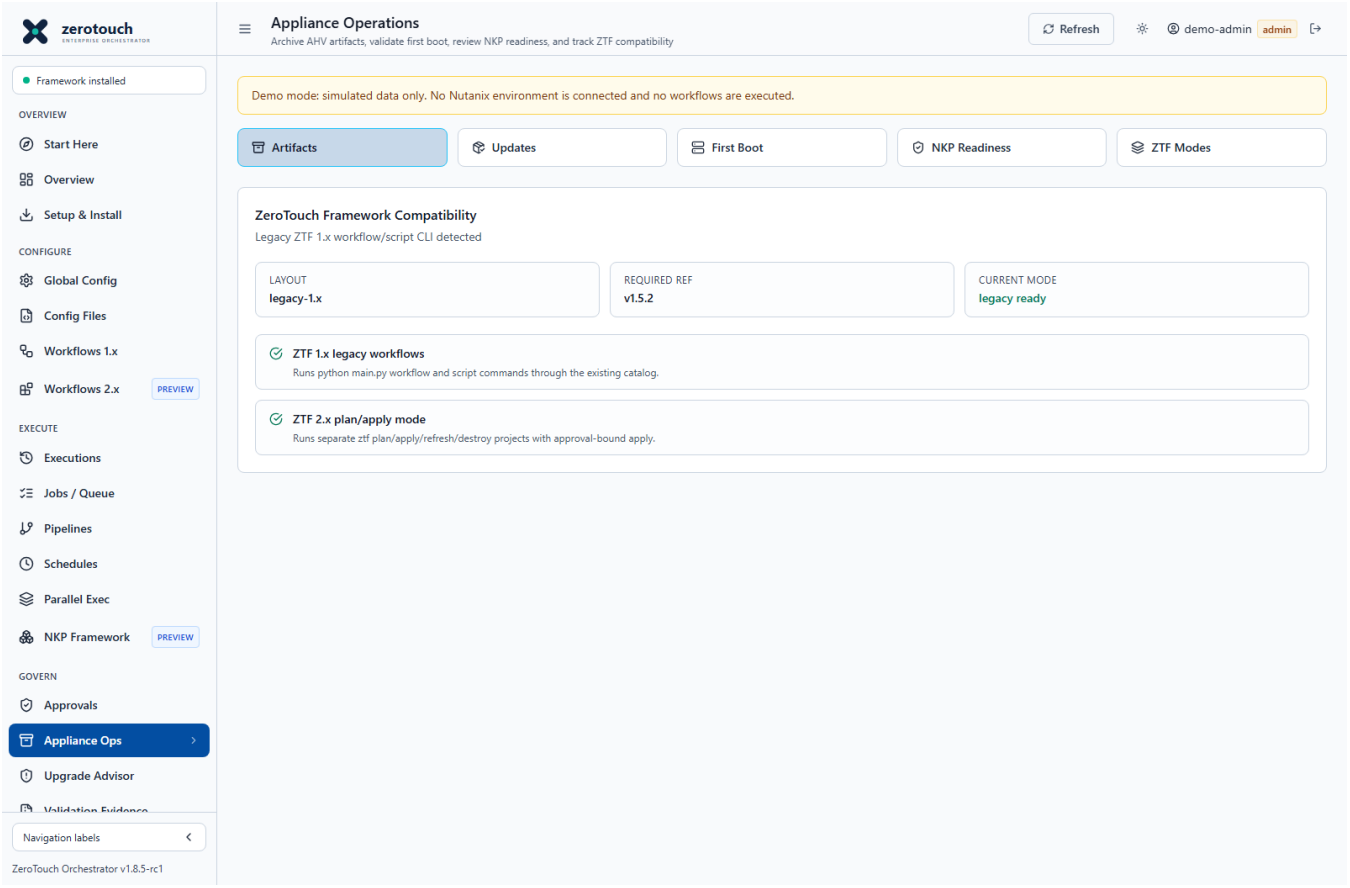


Figure 44. Separate framework runtime modes. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Confirm the legacy 1.x workflow/script layout and required ref before using legacy launchers. Configure the separate 2.x checkout and executable before generating or running IaC plans.

Do not fix an incompatible legacy runtime by pointing its path at a 2.x checkout. Verify framework mode in Settings and test the correct non-mutating entry point; mode recognition alone does not validate provider compatibility.

Daily weekly and maintenance checks

Frequency	Checks
Daily	Runtime health, failed/interrupted jobs, pending approvals, schedule execution timestamps, drift Unknown/Drifted findings and backup failures.
Weekly	Test backup readability, review disk growth and retained image tars, inspect inactive accounts, certificate expiry and notification delivery.
Before change	Record build and target scope, verify backup and recovery route, obtain approval, validate configuration and suspend conflicting automation.
After change	Verify target state, reconcile saved credentials/baselines, resume approved schedules and retain sanitized evidence.
Periodically	Rehearse isolated restore and recovery; review least privilege, runtime pins, compatibility, retention and operational ownership.

Troubleshooting and support

Symptom	Check	Next action
Workflow missing after rebuild	Installed Build, image tag, current frontend assets and catalogue	Confirm exact running image and health; do not infer deployed code from a source commit or browser footer.
YAML valid but workflow not runnable	Selected family, template, schema, references and adapter gate	Use the matching workflow generator; validation syntax and execution readiness are separate.
HTTP 401	Target account, active password, credential reference and permission	Inspect target-side authentication and lockout; do not repeatedly retry password changes.
Network unreachable or timeout	Container routing, VLAN, firewall, target endpoint and port	Test from the application execution context, not only a workstation or PCVM.
Certificate verification failed	Trust chain, SAN identity, validity and persistent mounted bundle	Install verified CA trust and use matching endpoint identity; do not turn TLS verification off.
HTTP 503	Actual API route and upstream service state	A browser redirect or root-page response does not prove the authenticated API is healthy.
Foundation checksum error	Exact ISO inventory record and expected digest	Supply the correct expected checksum and revalidate native payload before resubmission.
Progress Not reported	Raw API response shape, stale timestamp and job outcome	Use grouped classic telemetry when available; inspect Foundation and download sanitized logs.
Schedule does not run	Enabled state, UTC cron, server uptime, approval restrictions and latest job	Use Run now, inspect server/job logs and verify the next actual trigger.
No space left on device	Host filesystem, update-artifacts and complete backup inventory	Retain valid backup pairs; approve explicit old-file cleanup or expand storage before retry.

Support packet. Include Installed Build, deployment profile, storage backend, affected workflow/script identifier, sanitized YAML, timestamps in UTC, job/approval IDs, relevant sanitized logs, target software versions and what was verified independently. Exclude passwords, tokens, private keys and unredacted configuration exports. Preserve evidence before retrying, cleanup or restore.

Native Foundation planning and post Foundation baselines

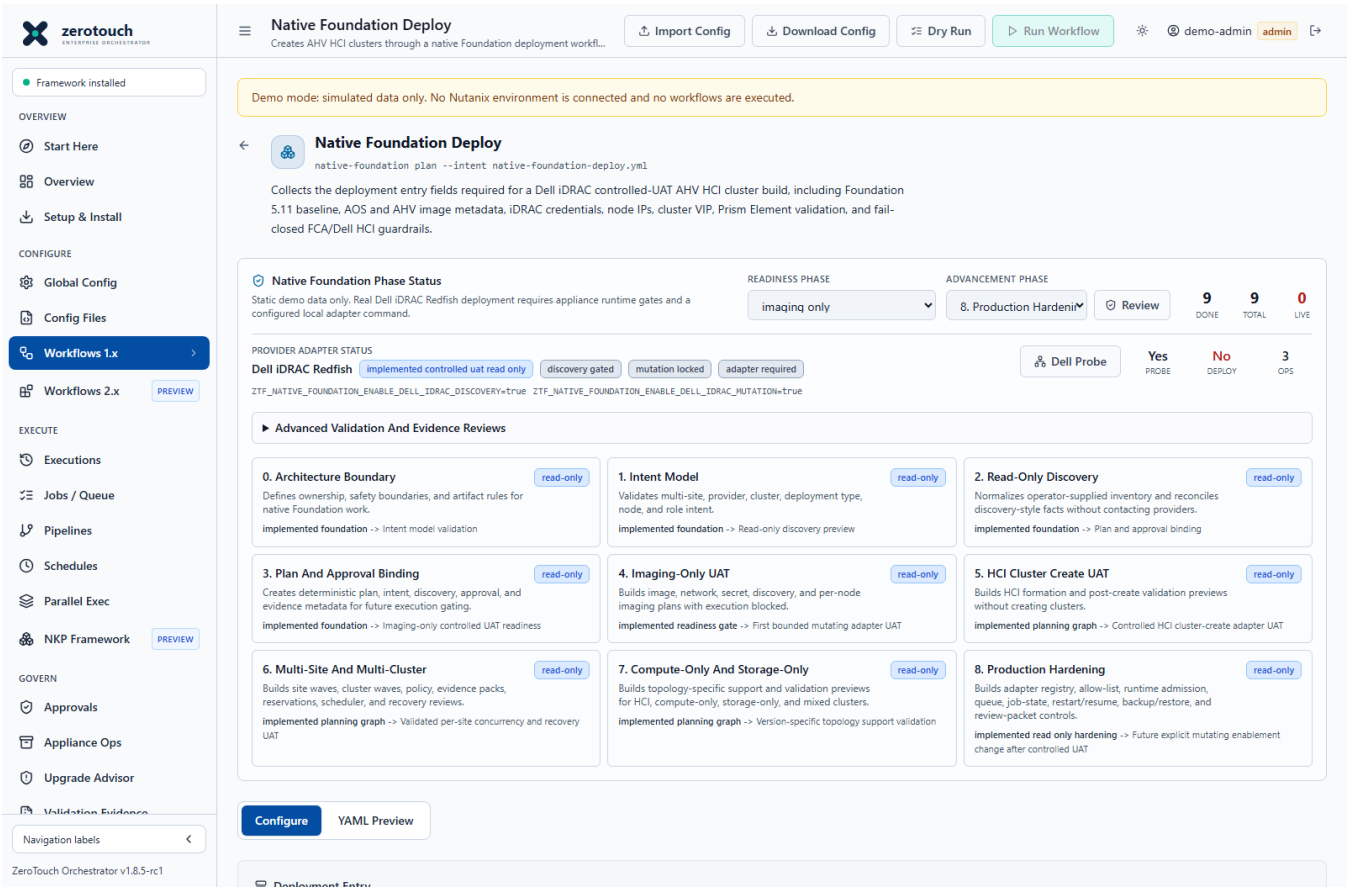


Figure 45. Native Foundation planning and provider readiness. Current v1.8.5-rc1 source capture with simulated demo data; not live deployment evidence.

Native Foundation is a separate planning and controlled-adapter lane. Its numerous review buttons generate deterministic manifests, plans, readiness gates, policy records and evidence. Most review actions do not discover live hardware, reserve windows, persist authorization or perform deployment. Follow each returned blocker and read-only flag. A Dell Probe checks connectivity/authentication and selected conditions; its passed count is a count of checks, not necessarily a count of nodes. A successful probe is not a completed imaging operation.

For Post-Foundation Baseline, Monitoring, Certificate, Network, Security Hardening and Hardware OOB entries, review the mapping between executable actions and evidence-only tasks. Complete manual controls in the target console where no verified executable mapping exists. Do not describe an evidence receipt as proof that a security or network control was applied.

Workflow and script reference

The following identifiers are transcribed from the pinned catalogue. Availability in a selector does not establish that an action is supported through pipelines, schedules or the specialized Foundation VM backend. Read the relevant entry details and installed runtime contract before use. Scripts are grouped by category; every script must use its own wizard/schema, target credential and independent verification rather than a generic sample YAML.

Workflow catalogue

Category	Name and identifier	Catalogue purpose
Configuration	CVM & iDRAC Password Rotation cvm-idrac-password-rotation	Rotate CVM SSH and Dell iDRAC passwords with verified logins
Infrastructure	Cluster Create cluster-create	Creates clusters using Foundation Central with full node imaging
Infrastructure	Cluster Create (Standalone FCA) cluster-create-standalone-fca	Creates clusters through standalone Foundation Central Appliance Lifecycle APIs
Infrastructure	Cluster Create (Foundation VM) cluster-create-foundation-vm	Creates clusters through the classic Foundation VM 5.x API
Infrastructure	Imaging Only imaging-only	Images nodes without creating a cluster (bare metal prep)
Infrastructure	Imaging Only (Standalone FCA) imaging-only-standalone-fca	Images nodes through standalone Foundation Central Appliance Lifecycle APIs
Pod Operations	Pod Imaging imaging	Pod workflow combining node imaging and cluster creation
Pod Operations	Pod Imaging (Standalone FCA) imaging-standalone-fca	Runs pod imaging through standalone Foundation Central Appliance Lifecycle APIs
Infrastructure	Site Deploy site-deploy	Full site deployment: imaging, cluster creation, and basic config
Infrastructure	Site Deploy (Standalone FCA) site-deploy-standalone-fca	Runs site deployment through standalone Foundation Central Appliance Lifecycle APIs
Infrastructure	Native Foundation Deploy native-foundation-deploy	Creates AHV HCI clusters through a native Foundation deployment workflow
Configuration	Configure Cluster config-cluster	Initial cluster configuration: admin password, EULA/Pulse, networking and storage
Configuration	Post-Foundation Baseline post-foundation-baseline	Plans safe Prism Element day-1 settings after cluster creation
Configuration	PE Monitoring Baseline pe-monitoring-baseline	Plans Prism Element monitoring, alerting, and evidence checks
Configuration	AHV Security Hardening pe-security-hardening	Plans post-cluster Prism Element, CVM, and AHV hardening controls
Configuration	PE Network Baseline pe-network-baseline	Plans Prism Element virtual switch and VM network settings
Configuration	PE Certificate Baseline pe-certificate-baseline	Plans Prism Element CSR and certificate replacement steps
Configuration	Hardware OOB Baseline hardware-out-of-band-baseline	Plans out-of-band hardware baseline tasks through vendor APIs

Category	Name and identifier	Catalogue purpose
Prism Central	Deploy Prism Central deploy-pc	Deploys Prism Central VM instances on target clusters
Prism Central	Configure Prism Central config-pc	Configures PC: AD, SAML, security policies, NKE, Objects
Prism Central	ZTF 2.x Prism Category ztf2-prism-category	Generates ZTF 2.x IaC for multi-category Prism Central intent
Prism Central	ZTF 2.x Project ztf2-project	Generates ZTF 2.x IaC for Prism Central project intent
Configuration	ZTF 2.x Subnet Intent ztf2-subnet-intent	Generates ZTF 2.x IaC for multi-subnet/VLAN resource intent
Workloads	ZTF 2.x Image Registration ztf2-image-registration	Generates ZTF 2.x IaC for VM image registration intent
Workloads	ZTF 2.x VM Deployment ztf2-vm-deployment	Generates ZTF 2.x IaC for virtual machine deployment intent
Configuration	ZTF 2.x Security Groups ztf2-security-groups	Generates ZTF 2.x IaC for address and service group intent
Prism Central	ZTF 2.x Protection Policy ztf2-protection-policy	Generates ZTF 2.x IaC for protection policy intent
Prism Central	ZTF 2.x Recovery Plan ztf2-recovery-plan	Generates ZTF 2.x IaC for disaster recovery plan intent
Pod Operations	Pod Config pod-config	Configures pod and edge site clusters at scale
Pod Operations	Deploy Management PC deploy-management-pc	Deploys Prism Central and NCM management instances
Pod Operations	Configure Management PC config-management-pc	Initializes and configures Prism Central and NCM
Workloads	Calm VM Workloads calm-vm-workloads	Creates and deploys VM workloads via Calm DSL blueprints
Workloads	Edge AI Workload calm-edgeai-vm-workload	Deploys Edge-AI application workloads
Services	NDB Deploy ndb	Deploys and configures Nutanix Database Service
System	LCM Update lcm-update	Runs lifecycle management update workflows

ZTF 1 x script catalogue

Category	Name and identifier	Catalogue purpose
Authentication	Create Local Users (PE) CreateLocalUsersPe	Administrator-only named PE user creation with separate credential references; verified HTTPS, role readback and no scheduled execution
Authentication	Add AD Server (PE) AddAdServerPe	Adds Active Directory server configuration to Prism Element
Authentication	Add AD Server (PC) AddAdServerPc	Adds Active Directory server configuration to Prism Central
Authentication	Create Role Mapping (PE) CreateRoleMappingPe	Creates AD role mappings in Prism Element
Authentication	Create Role Mapping (PC) CreateRoleMappingPc	Creates AD role mappings in Prism Central
Authentication	Create SAML IDP (PC) CreateIdp	Configures a SAML identity provider in Prism Central
Authentication	Add Local Users (PC) AddLocalUsers	Adds local IAM users in Prism Central
Authentication	Import Users (PC) ImportUsers	Imports users into Prism Central
Authentication	Add User Groups (PC) AddUserGroups	Adds IAM user groups in Prism Central
Authentication	Add Roles (PC) AddRoles	Adds IAM roles in Prism Central
Authentication	Add Directory Services (PC) AddDirectoryServices	Adds directory service objects in Prism Central
Networking	Create Subnet (PE) CreateSubnetPe	Creates a network subnet on Prism Element
Networking	Create Subnets (PC) CreateSubnetsPc	Creates network subnets on Prism Central
Networking	Delete Subnets (PE) DeleteSubnetsPe	Removes network subnets from Prism Element
Networking	Delete Subnets (PC) DeleteSubnetsPc	Removes network subnets from Prism Central
Networking	Create VPC (PC) CreateVPC	Creates a Virtual Private Cloud in Prism Central
Networking	Update VPC (PC) UpdateVPC	Updates a Virtual Private Cloud in Prism Central
Networking	Delete VPC (PC) DeleteVPC	Deletes a Virtual Private Cloud in Prism Central

Category	Name and identifier	Catalogue purpose
Networking	Enable Network Controller (PC) EnableNetworkController	Enables Network Controller in Prism Central
Networking	Disable Network Controller (PC) DisableNetworkController	Disables Network Controller in Prism Central
Networking	Add DNS Servers (PE) AddNameServersPe	Adds DNS name servers in Prism Element
Networking	Add DNS Servers (PC) AddNameServersPc	Adds DNS name servers in Prism Central
Networking	Add NTP Servers (PE) AddNtpServersPe	Adds NTP servers in Prism Element
Networking	Add NTP Servers (PC) AddNtpServersPc	Adds NTP servers in Prism Central
Storage	Create Storage Container (PE) CreateContainerPe	Creates a storage container in Prism Element
Storage	Delete Storage Container (PE) DeleteContainerPe	Removes a storage container from Prism Element
Storage	Create Object Store (PC) CreateObjectStore	Creates an S3-compatible object store
Storage	Delete Object Store (PC) DeleteObjectStore	Deletes an object store
Storage	Create Bucket (PC) CreateBucket	Creates a bucket in an object store
Storage	Share Bucket (PC) ShareBucket	Shares a bucket with users
Compute	Create VM (PE) CreateVmPe	Creates virtual machines directly through Prism Element
Compute	Create VMs (PC) CreateVmsPc	Creates virtual machines through Prism Central
Compute	Delete VM (PE) DeleteVmPe	Deletes virtual machines directly through Prism Element
Compute	Delete VM (PC) DeleteVmPc	Deletes virtual machines through Prism Central
Compute	Power Transition VM (PE) PowerTransitionVmPe	Changes VM power state through Prism Element
Compute	Power On VM (PC) PowerOnVmPc	Powers on virtual machines through Prism Central
Images	Upload Image (PE) UploadImagePe	Uploads disk or ISO images to Prism Element

Category	Name and identifier	Catalogue purpose
Images	Upload Image (PC) PcImageUpload	Uploads disk images through Prism Central
Images	Upload OVA (PC) PcOVAUpload	Uploads OVA images through Prism Central
Images	Delete Image (PC) PcImageDelete	Deletes disk images through Prism Central
Images	Delete OVA (PC) PcOVADelete	Deletes OVA images through Prism Central
Security	Create Security Policy (PC) CreateNetworkSecurityPolicy	Creates a Flow network security policy
Security	Delete Security Policy (PC) DeleteNetworkSecurityPolicy	Deletes a Flow network security policy
Security	Create Address Groups (PC) CreateAddressGroups	Creates network address groups for security policies
Security	Delete Address Groups (PC) DeleteAddressGroups	Deletes network address groups
Security	Create Service Groups (PC) CreateServiceGroups	Creates service groups for security policies
Security	Delete Service Groups (PC) DeleteServiceGroups	Deletes service groups
Security	Create Category (PC) CreateCategoryPc	Creates categories for VM grouping and policies
Security	Delete Category (PC) DeleteCategoryPc	Deletes categories in Prism Central
Kubernetes	Create NKE Cluster (PC) CreateKarbonClusterPc	Creates a Nutanix Kubernetes Engine cluster
Kubernetes	Enable NKE (PC) EnableNke	Enables Nutanix Kubernetes Engine on Prism Central
Database	Configure NDB NdbConfig	Runs the NDB configuration script
Database	Register NDB Cluster RegisterInitClusterNdb	Registers and initializes a cluster with NDB
Prism Central	Deploy Prism Central DeployPC	Deploys a Prism Central VM
Prism Central	Register PE to PC RegisterToPc	Registers Prism Element clusters with Prism Central
Prism Central	Enable Microsegmentation (PC) EnableMicrosegmentation	Enables Flow microsegmentation on Prism Central

Category	Name and identifier	Catalogue purpose
Prism Central	Disable Microsegmentation (PC) DisableMicrosegmentation	Disables Flow microsegmentation on Prism Central
Prism Central	Enable Objects (PC) EnableObjects	Enables Nutanix Objects on Prism Central
Prism Central	Enable DR (PC) EnableDR	Enables Disaster Recovery capabilities
Prism Central	Create Protection Policy (PC) CreateProtectionPolicy	Creates a VM protection policy with RPO/RTO settings
Prism Central	Delete Protection Policy (PC) DeleteProtectionPolicy	Deletes a protection policy
Prism Central	Create Recovery Plan (PC) CreateRecoveryPlan	Creates a disaster recovery plan
Prism Central	Delete Recovery Plan (PC) DeleteRecoveryPlan	Deletes a disaster recovery plan
Prism Central	Connect Availability Zone (PC) ConnectToAz	Connects a remote availability zone for DR
Prism Central	Disconnect Availability Zone (PC) DisconnectAz	Disconnects a remote availability zone
Prism Central	Enable Foundation Central (PC) EnableFC	Enables Foundation Central in Prism Central
Prism Central	Generate FC API Key GenerateFcApiKey	Generates a Foundation Central API key
Prism Central	Enable Marketplace (PC) EnableMarketplace	Enables Marketplace in Prism Central
Prism Element	Change Default Admin Password (PE) ChangeDefaultAdminPasswordPe	Changes the Prism Element admin password using a new credential reference; does not rotate AHV, CVM, or iDRAC accounts
Prism Element	Accept EULA (PE) AcceptEulaPe	Accepts the EULA in Prism Element
Prism Element	Accept EULA (PC) AcceptEulaPc	Accepts the EULA in Prism Central
Prism Element	Update Pulse (PE) UpdatePulsePe	Updates Pulse settings in Prism Element
Prism Element	Update Pulse (PC) UpdatePulsePc	Updates Pulse settings in Prism Central
Prism Element	Set HA Reservation (PE) HaReservation	Configures high-availability host reservation
Prism Element	Set Rebuild Capacity (PE) RebuildCapacityReservation	Configures rebuild capacity reservation

Category	Name and identifier	Catalogue purpose
Prism Element	Update DSIP (PE) UpdateDsip	Updates DSIP in Prism Element
System	Update CVM Foundation UpdateCvmFoundation	Updates Foundation on CVM nodes

ZTF 2 x script catalogue

Category	Name and identifier	Catalogue purpose
Prism Central	Create Category (PC) ztf2-create-category-pc	Generates ZTF 2.x IaC for Prism Central category intent
Prism Central	Create Project (PC) ztf2-create-project-pc	Generates ZTF 2.x IaC for Prism Central project intent
Configuration	Create Subnets (PC) ztf2-create-subnets-pc	Generates ZTF 2.x IaC for Prism Central subnet/VLAN intent
Workloads	Upload Image (PC) ztf2-upload-image-pc	Generates ZTF 2.x IaC for VM image registration intent
Workloads	Create VMs (PC) ztf2-create-vms-pc	Generates ZTF 2.x IaC for virtual machine deployment intent
Security	Create Security Groups (PC) ztf2-create-security-groups-pc	Generates ZTF 2.x IaC for address and service group intent
Data Protection	Create Protection Policy (PC) ztf2-create-protection-policy-pc	Generates ZTF 2.x IaC for protection policy intent
Data Protection	Create Recovery Plan (PC) ztf2-create-recovery-plan-pc	Generates ZTF 2.x IaC for recovery plan intent

Acceptance checklists and source reference

Administrator handover checklist

1. Named owner, service URL, management network restrictions and least-privilege accounts documented.
2. Full build identity, runtime refs, storage backend and effective mutation gates recorded.
3. Verified backups retained independently, restore rehearsed, and retention/free-space monitoring assigned.
4. Target certificate trust, account permissions and renewal/rotation ownership documented.
5. Approval policy, emergency stop, maintenance procedure and notification routing tested.
6. Each enabled schedule tested manually and at its scheduled time, including post-restart behavior.

Operator job checklist

1. Correct backend and target identities selected; hardware/software compatibility and destructive scope reviewed.
2. YAML and credentials reviewed; no unexpected plaintext secrets in exports.
3. Non-mutating validation completed, approval obtained and required confirmation entered deliberately.
4. Per-target results reviewed, partial state reconciled, and actual Prism or Foundation completion verified.
5. Sanitized logs, configuration hash/build identity and acceptance evidence attached to the change record.

Validation limits of this document

Operational text and simulated UI captures were reviewed against merged application revision 4a010727d5db3496d17a52e4ce681818dcc56200 on 27 October 2026. No live Nutanix passwords, accounts, EULA/Pulse settings, physical hardware, workloads or running appliance configuration were changed while updating this guide. Source, fixture and documentation evidence do not close site-specific UAT, recovery, soak or stable-promotion gates.

Source documents

Repository https://github.com/VirtuArchitect/ZTF-Orchestrator. Application and screenshot source revision 4a010727d5db3496d17a52e4ce681818dcc56200. The appliance example v1.8.5-rc1-4a01072 uses that revision and remains controlled UAT. All screenshots were recaptured from the isolated static demo on 27 October 2026. Paths below are repository-relative; use commit-pinned documents for reproducibility.

docs/installation-guide.md

docs/user-guide.md

docs/cluster-initial-configuration.md

docs/pe-admin-password-change.md

docs/credential-rotation.md

docs/live-pe-drift-monitoring.md

docs/admin-guide.md

docs/runbooks/RB-002-backup-restore.md

src/data.ts

src/App.tsx

src/pages/Pipelines.tsx

src/pages/DriftDetection.tsx

src/components/forms/ClusterCreateForm.tsx

docker-compose.yml

foundation_vm_backend.py

foundation_vm_progress.py

prism_drift.py

docs/appliance-update-manager.md

docs/runbooks/RB-003-upgrade-rollback.md

docs/runbooks/RB-007-airgapped-update.md

docs/nginx-tls.md

docs/production-readiness-boundary.md

docs/pe-post-configuration-verification.md

docs/pe-local-users.md

docs/nutanix-upgrade-risk-advisor.md

docs/releases/README.md

docs/releases/v1.8.5-rc1-readiness.md

Release document index: https://virtuarchitect.github.io/ZTF-Orchestrator/releases/

Current application source reference: https://github.com/VirtuArchitect/ZTF-Orchestrator/tree/4a010727d5db3496d17a52e4ce681818dcc56200

Historical release notes and installation checklists are preserved under docs/releases/archive using commit-pinned references in the index. The repository root remains clean. Existing external blob/main URLs to removed files are not automatically redirected by GitHub; use the published index or the documented pinned archive URL.

docs/start-here.md

docs/docker-build-identity.md

docs/pe-certificate-trust.md

docs/manuals/README.md

docs/assets/guide/v1.8.5-rc1/README.md

Screenshot index

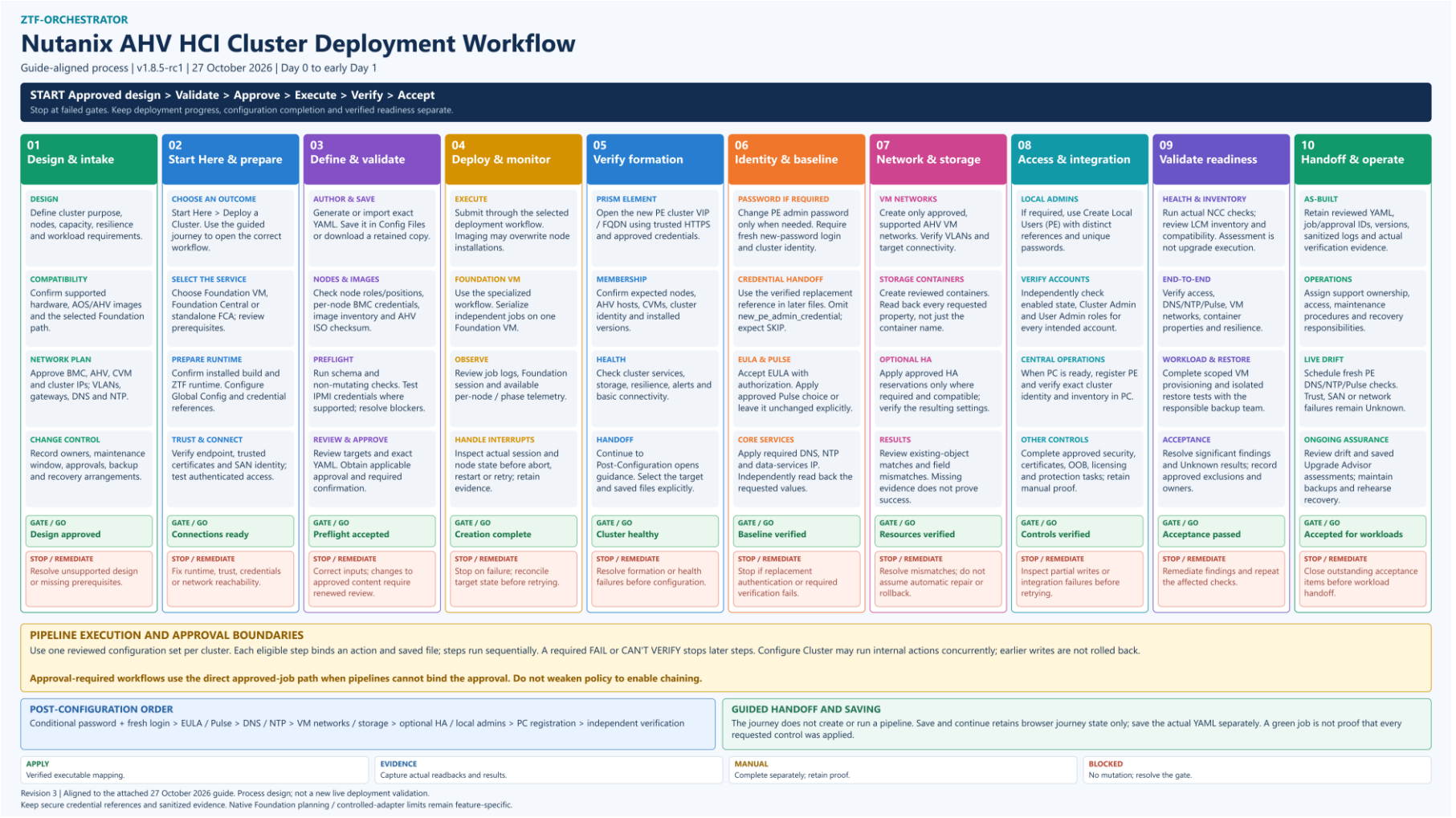
Figure	Screen
1	Operations dashboard and navigation
2	Start Here and the four operator choices
3	Deployment methods and seven guidance stages
4	Post-configuration guidance and pipeline handoff
5	Dashboard
6	Setup & install
7	Workflows 1.x
8	Workflows 2.x
9	Global Config

Figure	Screen
10	Config Files
11	YAML Studio
12	Scripts 1.x
13	Scripts 2.x
14	Execution History
15	Jobs / Queue
16	Pipelines
17	Schedules
18	Parallel Exec
19	NKP Framework
20	Approvals
21	Appliance Ops
22	Upgrade Advisor
23	Validation Evidence
24	Drift Detection
25	Audit Log
26	Users
27	Settings
28	Separate ZTF 2.x plan and apply workspace
29	Classic Foundation VM workflow and start options
30	Image inventory controls and AHV ISO checksum
31	Per-node network and credential inputs
32	Configure Cluster form
33	Sequential pipeline definition and execution entry
34	Controlled UAT credential rotation workflow
35	Editing a persisted drift schedule
36	PostgreSQL backup and restore controls
37	Named connection profiles
38	Governance and approval policy

Figure	Screen
39	Notification configuration
40	Installed build identity
41	Appliance update discovery and package import
42	First boot checks
43	NKP readiness review
44	Separate framework runtime modes
45	Native Foundation planning and provider readiness

Cluster deployment process flow revision 3

End-to-end overview | v1.8.5-rc1 | 27 October 2026



Process flow overview. Gates separate deployment completion, configuration results and verified readiness. This diagram describes the process; it is not new live deployment evidence.

Full-resolution image, editable SVG and text equivalent: https://github.com/VirtuArchitect/ZTF-Orchestrator/blob/main/docs/cluster-deployment-process.md

Stages 01 to 05 preparation and deployment



Design approval; Start Here and trusted connectivity; saved YAML, images and preflight; governed deployment; independent formation and PE health checks.

Eligible pipeline steps run sequentially; required verification failures stop later steps. Use direct approved jobs where policy requires approval binding. Inspect partial writes before retry; there is no transactional rollback.

Stages 06 to 10 configuration and acceptance



Fresh password authentication; EULA/Pulse and core settings; network/storage readback; local roles and PC integration; readiness, acceptance and ongoing monitoring.

Eligible pipeline steps run sequentially; required verification failures stop later steps. Use direct approved jobs where policy requires approval binding. Inspect partial writes before retry; there is no transactional rollback.